BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC SƯ PHẠM KỸ THUẬT TP. HCM
KHOA ĐIỆN - ĐIỆN TỬ
BỘ MÔN KỸ THUẬT MÁY TÍNH - VIỄN THÔNG



TIỂU LUẬN MÔN HỌC
PHÁT TRIỂN ỨNG DỤNG DI DỘNG

ỨN DỤNG EDGE AI TRONG HỆ THỐNG CHO THÚ CƯNG ĂN THÔNG MINH

Giáo viên hướng dẫn: Th.S. Huỳnh Hoàng Hà
Nhóm thực hiện: 06
Mã lớp học: MBAD436364

TP. Hồ Chí Minh, tháng 09 năm 2026

BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC SƯ PHẠM KỸ THUẬT TP. HCM
KHOA ĐIỆN - ĐIỆN TỬ
BỘ MÔN KỸ THUẬT MÁY TÍNH - VIỄN THÔNG



TIỂU LUẬN MÔN HỌC
PHÁT TRIỂN ỨNG DỤNG DI DỘNG

ỨN DỤNG EDGE AI TRONG HỆ THỐNG CHO THÚ CƯNG ĂN THÔNG MINH

GVHD: Th.S. Huỳnh Hoàng Hà

SVTH: Nhóm 06

1. Hoàng Ngọc Dung 23139006
2. Hoàng Mạnh Hùng 23139017
3. Trần Hữu Dương 23139009
4. Phạm Quang Phúc 23139034
5. Đoàn Minh Duy Bình 23139005

TP. Hồ Chí Minh, tháng năm 2026

DANH SÁCH NHÓM VIẾT BÁO CÁO CUỐI KỲ

HỌC KỲ 1 NĂM HỌC 2026–2027

1. **Mã lớp môn học:** MBAD436364
2. **Giảng viên hướng dẫn** : Th.S. Huỳnh Hoàng Hà
3. **Tên đề tài:** Ứng dụng edge AI trong hệ thống cho thú cưng ăn thông minh
4. **Danh sách nhóm và phân công nhiệm vụ**

STT	Họ và tên sinh viên	MSSV	Nội dung thực hiện	Ký tên
1	Hoàng Ngọc Dung	23139006	Phát triển mô hình Edge AI, huấn luyện, đánh giá, quantization và triển khai trên ESP32-S3 CAM.	
2	Hoàng Mạnh Hùng	23139017	Thiết kế hệ thống, xây dựng phần cứng và phát triển ứng dụng React Native kết nối Firebase.	
3	Phạm Quang Phúc	23139034	Phát triển ứng dụng, Firebase, chức năng quản lý cho ăn, lịch sử và kiểm thử hệ thống.	
4	Trần Hữu Dương	23139009	Chuẩn bị dataset, hỗ trợ huấn luyện mô hình, triển khai Edge AI và tích hợp điều khiển cho ăn.	
5	Đoàn Minh Duy Bình	23139005	Lắp ráp phần cứng ESP32-S3 CAM, servo, load cell/HX711; kiểm thử và tối ưu hệ thống.	

Nhận xét của giáng viên

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

Ngày …. tháng …. năm 2026

Giảng viên chấm điểm

LỜI CAM ĐOAN

Chúng em xin cam đoan đề tài nghiên cứu là trung thực. Những kết quả số liệu, thông tin phục vụ cho quá trình xử lý và hoàn thành bài nghiên cứu đều được thu thập từ các nguồn khác nhau, có ghi rõ nguồn gốc. Tất cả các thông tin số liệu, bảng, hình ảnh và kết quả nghiên cứu được đều được trình bày một cách minh bạch và chân thực, không chứa bất kỳ sự biến đổi hay chỉnh sửa nào có thể làm ảnh hưởng đến tính chính xác của kết quả.

Các nguồn dữ liệu và thông tin được sử dụng trong nghiên cứu đã được thu thập từ các nguồn đáng tin cậy và đã được ghi rõ nguồn gốc. Bất kỳ tài liệu tham khảo, nghiên cứu trước đây hoặc ý kiến cá nhân được sử dụng đều đã được đặc tả một cách rõ ràng để đảm bảo tính minh bạch và khách quan. Chúng em xin chịu hoàn toàn trách nhiệm, kỷ luật của bộ môn và nhà trường nếu như có bất cứ vấn đề nào.

LỜI CẢM ƠN

Lời đầu tiên, chúng em xin gửi lời cảm ơn chân thành đến Th.S. Huỳnh Hoàng Hà giảng viên bộ môn Lập trình ứng dụng di động**.** Trong suốt quá trình học tập, Thầy đã rất tâm huyết dạy và hướng dẫn cho chúng tôi nhiều điều bổ ích trong môn học và kỹ năng làm một bài nghiên cứu để chúng tôi có đủ kiến thức thực hiện bài báo cáo này.

Tuy nhiên vì kiến thức bản thân còn nhiều hạn chế và sự tìm hiểu chưa sâu sắc nên không tránh khỏi những thiếu sót. Mong Thầy sẽ châm chước và cho chúng em những lời góp ý để bài nghiên cứu của chúng tôi sẽ hoàn thiện hơn. Một lần nữa, chúng tôi xin gửi lời cảm ơn sâu sắc đến Thầy và chúc Thầy luôn mạnh khỏe, hạnh phúc và thành công trong sự nghiệp.

TP. Hồ Chí Minh, tháng năm 2026

MỤC LỤC

DANH MỤC CÁC CHỮ VIẾT TẮT x
CHƯƠNG 1: GIỚI THIỆU TỔNG QUAN VỀ ĐỀ TÀI 1
1. Tình hình nghiên cứu 1
2. Đặt vấn đề 2
3. Mục tiêu của đề tài 3
4. Nội dung nghiên cứu 5
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT 7
2.1. Tổng quan hệ thống IoT và Edge AI 7
2.2. ESP32-S3 và ESP32-S3 CAM 7
2.3. Cơ sở lý thuyết về phân loại ảnh 7
2.4. Mô hình AI nhẹ cho Edge Device 9
2.5. Tối ưu mô hình trên ESP32-S3 10
2.6. Servo Motor và cơ cấu cấp thức ăn 11
2.7. Load Cell và HX711 12
2.8. Firebase 13
2.9. React Native 15
CHƯƠNG 3: PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG 17
3.1. Yêu cầu hệ thống 17
3.1.1. Yêu cầu chức năng 17
3.1.2. Yêu cầu phi chức năng 19
3.2. Kiến trúc tổng thể hệ thống 20
3.3. Thiết kế phần cứng 23
3.3.1. ESP32-S3 CAM 23
3.3.2. Camera 25
3.3.3. Servo Motor 25
3.3.4. Load Cell và HX711 26
3.3.5. Nguồn cấp 28
3.3.6. Sơ đồ kết nối phần cứng 29
3.4. Thiết kế mô hình Edge AI 31
3.4.1. Tổng quan mô hình 31

3.4.2. Nền tảng toán học 34
3.4.3. Encoder hướng phần cứng: từng lớp, từng công thức 38
3.4.4. Shared–Specific Sparse Unrolling Head 43
3.4.5. Hàm mất mát 50
3.4.6. Chưng cất tri thức 53
3.4.7. Quy trình tối ưu hoá 55
3.4.9. Phân tích độ phức tạp 56
3.4.10. Chỉ số đánh giá 58
33.5. Thiết kế chức năng cho ăn theo loài 58
3.6. Thiết kế thuật toán cấp thức ăn 60
3.7. Thiết kế điều khiển dựa trên phản hồi khối lượng 62
3.8. Thiết kế Firebase 65
3.8.1. Phân chia quyền đọc và ghi dữ liệu 65
3.8.2. Cấu trúc cây dữ liệu Realtime Database 67
3.8.3. Luồng đồng bộ Feed Now 68
3.8.4. Cấu hình theo loài, lịch và cooldown 69
3.8.5. Heartbeat và đánh giá thiết bị sẵn sàng 69
3.8.6. Tính toàn vẹn dữ liệu và xử lý tranh chấp 70
3.8.7. Bảo mật và giới hạn hiện tại 70
3.9. Thiết kế ứng dụng React Native 70
3.9.1. Kiến trúc phần mềm ứng dụng 71
3.9.2. FeederContext và vòng đời dữ liệu 71
3.9.3. FirebaseFeederRepository 72
3.9.4. Device contract và chuẩn hóa dữ liệu 72
3.9.5. Thiết kế các màn hình chức năng 72
3.9.6. Kiểm soát thao tác và an toàn phía ứng dụng 74
3.9.7. Luồng dữ liệu từ giao diện đến thiết bị 74
CHƯƠNG 4: PHÂN TÍCH KẾT QUẢ 75
4.1. Môi trường thực nghiệm 75
4.2. Kết quả mô hình AI 76
4.3. Kết quả triển khai Edge AI 77

4.4. Kết quả Firebase và ứng dụng 79
4.4.1. Kết quả kiểm tra ứng dụng 79
4.4.2. Kết quả các chức năng Firebase và mobile 80
4.4.3. Kết quả tích hợp với firmware.....81
4.4.4. Đánh giá luồng Feed Now end-to-end.....82
4.4.5. Những phần chưa được xem là đồng bộ hoàn toàn 82
4.4.6. Kết luận phần Firebase và ứng dụng 83
KẾT LUẬN 84
TÀI LIỆU THAM KHẢO.....86

DANH MỤC CÁC CHỮ VIẾT TẮT

AAHA	Hiệp hội Bệnh viện Động vật Hoa Kỳ
AI	Trí tuệ nhân tạo
CAM	Camera
CNN	Mạng nơ-ron tích chập
DB	Cơ sở dữ liệu
DL	Học sâu
ESP32	Vi điều khiển ESP32
ESP32-S3	Vi điều khiển ESP32-S3
FR	Yêu cầu chức năng
HX711	Bộ chuyển đổi tín hiệu cho cảm biến khối lượng
ID	Mã định danh
INT8	Số nguyên 8 bit
IoT	Internet vạn vật
MCU	Bộ vi điều khiển
ML	Học máy
RTDB	Cơ sở dữ liệu thời gian thực
TFLM	TensorFlow Lite dành cho vi điều khiển
UID	Mã định danh người dùng
UI	Giao diện người dùng

CHƯƠNG 1: GIỚI THIỆU TỔNG QUAN VỀ ĐỀ TÀI

1. Tình hình nghiên cứu

Trong đời sống hiện đại, chó và mèo không chỉ được nuôi để trông giữ nhà mà còn trở thành thành viên trong nhiều gia đình. Vì vậy, việc bảo đảm chế độ dinh dưỡng phù hợp có ảnh hưởng trực tiếp đến sức khỏe, thể trạng và chất lượng sống của thú cưng. Các hướng dẫn dinh dưỡng của AAHA và WSAVA đều nhấn mạnh rằng khẩu phần cần được đánh giá theo từng cá thể, dựa trên loài, khối lượng, giai đoạn sống, mức độ vận động và tình trạng sức khỏe; việc cho ăn thiếu kiểm soát có thể dẫn đến thừa cân, béo phì hoặc thiếu dinh dưỡng. Tuy nhiên, chủ nuôi không phải lúc nào cũng có mặt tại nhà để cho thú cưng ăn đúng giờ và đúng lượng. Lịch học tập, làm việc hoặc những chuyến đi ngắn ngày dễ làm gián đoạn thói quen cho ăn. Phương pháp cho ăn thủ công phụ thuộc hoàn toàn vào người chăm sóc, trong khi các máy cho ăn hẹn giờ thông thường chủ yếu chỉ mở cơ cấu cấp thức ăn theo thời gian cài đặt. Các thiết bị này thường chưa xác nhận được con vật nào đang tiếp cận, lượng thức ăn đã cấp thực tế là bao nhiêu hoặc lần cho ăn có diễn ra thành công hay không.

Trong gia đình nuôi nhiều thú cưng, hạn chế trên trở nên rõ rệt hơn. Một con vật có thể ăn nhầm khẩu phần của con khác; thú cưng lạ cũng có thể tiếp cận máng ăn; đồng thời sự cố kẹt cơ cấu, hết thức ăn hoặc mất kết nối có thể không được phát hiện kịp thời. Do đó, hệ thống chỉ điều khiển theo bộ hẹn giờ chưa đủ để đáp ứng yêu cầu chăm sóc chính xác và có khả năng kiểm chứng. Cần kết hợp cơ cấu cấp thức ăn với Load Cell, cảm biến hiện diện và camera để xác nhận đối tượng, đo lượng thức ăn và lưu lại kết quả sau mỗi lần hoạt động. Sự phát triển của Internet vạn vật tạo điều kiện liên kết bộ điều khiển, cảm biến, cơ sở dữ liệu đám mây và ứng dụng di động thành một hệ thống thống nhất. Nghiên cứu của Castillo-Arceo và cộng sự cho thấy việc kết hợp IoT, cảm biến khối lượng, cảm biến tiếp cận, camera và mô hình học sâu trong máy cho thú cưng ăn có thể hỗ trợ nhận diện đối tượng và kiểm soát khẩu phần [3]. Đối với đề tài này, ứng dụng di động giúp người dùng thiết lập lịch, ra lệnh cho ăn, theo dõi trạng thái và xem lịch sử; Firebase Realtime Database đóng vai trò đồng bộ dữ liệu giữa ứng dụng và thiết bị gần như theo thời gian thực. Bên cạnh kết nối IoT, trí tuệ nhân tạo giúp hệ thống chuyển từ cơ chế cho ăn tự động đơn thuần sang cơ chế có khả năng nhận biết đối tượng. Camera trên ESP32-S3 CAM thu nhận hình ảnh, mô hình AI xác định chó hoặc mèo và

trả về độ tin cậy trước khi hệ thống quyết định cấp thức ăn. Tuy nhiên, vi điều khiển có tài nguyên bộ nhớ và khả năng tính toán hạn chế, vì vậy mô hình phải được tối ưu về số tham số, dung lượng và thời gian suy luận. Dự án ESP-Detection của Espressif cung cấp các mô hình phát hiện đối tượng siêu nhẹ dựa trên YOLOv11 và quy trình triển khai cho ESP32-S3, cho thấy hướng xử lý AI tại biên là khả thi đối với phần cứng nhúng [5]. Các nghiên cứu về triển khai mô hình phát hiện đối tượng trên ESP32-CAM cũng khẳng định nhu cầu tối ưu mô hình khi xử lý ảnh theo thời gian thực.

Từ những vấn đề trên, việc xây dựng hệ thống cho thú cưng ăn thông minh kết hợp ứng dụng di động, cơ sở dữ liệu đám mây, cảm biến và AI là cần thiết. Đề tài hướng đến một giải pháp có chi phí hợp lý, dễ sử dụng, cho phép người nuôi kiểm soát việc cho ăn từ xa nhưng vẫn có cơ chế tự động xác nhận đối tượng và lượng thức ăn. Hệ thống đồng thời tạo ra dữ liệu lịch sử để người dùng theo dõi thói quen ăn uống, phát hiện tình trạng bất thường và cải thiện quá trình chăm sóc thú cưng.

2. Đặt vấn đề

Bài toán đặt ra là thiết kế một hệ thống có khả năng cho chó và mèo ăn đúng thời điểm, đúng khẩu phần và đúng đối tượng, đồng thời cung cấp thông tin giám sát cho người dùng trên điện thoại. Hệ thống phải phối hợp giữa ba khối chính: ứng dụng di động React Native, nền tảng Firebase và thiết bị nhúng gồm ESP32, ESP32-S3 CAM, servo, Load Cell, HX711 cùng cảm biến phát hiện. Mỗi khối thực hiện một nhiệm vụ riêng nhưng phải trao đổi dữ liệu thống nhất để lệnh cho ăn có thể được theo dõi từ lúc tạo đến khi hoàn tất.

Để giải quyết bài toán tổng thể, đề tài tập trung vào các yêu cầu sau:

- Quản lý tài khoản và hồ sơ thú cưng, bao gồm loài, khối lượng, loại thức ăn và khẩu phần khuyến nghị.
- Cho phép người dùng ra lệnh cho ăn thủ công hoặc thiết lập lịch cho ăn trên ứng dụng di động.
- Nhận diện chó hoặc mèo từ hình ảnh camera, kèm theo độ tin cậy để hỗ trợ quyết định cho ăn.
- Điều khiển servo cấp thức ăn, sử dụng Load Cell và HX711 để đo lượng thức ăn thực tế sau mỗi lần cấp.
- Cập nhật trạng thái thiết bị, kết quả nhận diện và lịch sử cho ăn lên Firebase để ứng dụng theo dõi gần thời gian thực.

- Cảnh báo khi thức ăn sắp hết, thiết bị mất kết nối, lệnh thực hiện thất bại hoặc phát hiện đối tượng không phù hợp.

Luồng xử lý dự kiến bắt đầu khi người dùng nhấn Feed Now, khi đến thời điểm của một lịch đã lưu hoặc khi cảm biến phát hiện thú cưng trước máng ăn. Yêu cầu cho ăn được ghi vào Firebase với mã lệnh, thời gian, khẩu phần và trạng thái pending. ESP32 theo dõi lệnh mới, kiểm tra điều kiện an toàn và điều khiển servo. Trong trường hợp cần nhận diện, ESP32-S3 CAM xử lý ảnh và gửi về speciesId cùng confidence. Khi kết quả đạt ngưỡng tin cậy và phù hợp với hồ sơ đã cấu hình, bộ điều khiển mới cấp thức ăn. Load Cell đo khối lượng thực tế, sau đó thiết bị cập nhật trạng thái completed hoặc failed và ghi dữ liệu vào lịch sử.

Về tổ chức dữ liệu, thông tin cá nhân, hồ sơ loài, lịch cho ăn và thông báo được lưu theo mã uid của người dùng. Trạng thái thiết bị, kết quả nhận diện, lệnh cho ăn và lịch sử hoạt động được lưu theo mã thiết bị, chẳng hạn feeder-01. Cách phân tách này giúp ứng dụng quản lý dữ liệu riêng của từng tài khoản, đồng thời tạo một vùng dữ liệu chung để thiết bị và ứng dụng trao đổi lệnh. Khi triển khai thực tế, hệ thống cần bổ sung quan hệ sở hữu giữa uid và mã thiết bị, xác thực riêng cho thiết bị và các quy tắc Firebase giới hạn quyền đọc, ghi.

Trong phạm vi hiện tại, ứng dụng React Native và các chức năng đăng nhập, đọc, ghi dữ liệu Firebase được sử dụng làm nền tảng giao tiếp thật. Khối phần cứng và AI được thiết kế để tích hợp thông qua các trường dữ liệu đã xác định; kết quả nhận diện, trạng thái servo và số đo Load Cell cần được kiểm thử trên thiết bị thật trước khi kết luận độ chính xác toàn hệ thống. Vì vậy, quá trình đánh giá không chỉ xem xét giao diện ứng dụng mà còn phải đo sai số khẩu phần, thời gian phản hồi, độ chính xác nhận diện, độ ổn định kết nối và khả năng xử lý tình huống lỗi.

3. Mục tiêu của đề tài

Đề tài hướng đến xây dựng một hệ thống cho thú cưng ăn thông minh, trong đó khả năng nhận diện và điều khiển được thực hiện trực tiếp trên thiết bị biên.

Trong phạm vi triển khai, hệ thống tập trung vào hai loài là chó và mèo. ESP32-S3 CAM đóng vai trò bộ xử lý trung tâm, tiếp nhận hình ảnh từ camera và thực hiện suy luận mô hình AI để trả về loài dự đoán cùng độ tin cậy. Kết quả này được firmware sử dụng để lựa chọn bộ cấp thức ăn tương ứng. Quá trình cấp thức ăn được thực hiện bằng

servo, trong khi Load Cell kết hợp HX711 liên tục phản hồi khối lượng để dừng quá trình xả khi đạt giá trị mục tiêu. Đồng thời, firmware tích hợp các cơ chế như timeout, phát hiện không tăng khối lượng và thời gian chờ giữa hai lần cho ăn nhằm hạn chế lỗi vận hành và tránh cấp thức ăn lặp lại trong thời gian ngắn.

Bên cạnh chức năng điều khiển tại thiết bị, hệ thống còn kết nối với Firebase để đồng bộ cấu hình, lệnh điều khiển, trạng thái thiết bị và lịch sử cho ăn với ứng dụng Android phát triển bằng React Native. Qua đó, đề tài xây dựng được một quy trình hoàn chỉnh từ nhận diện thú cưng, ra quyết định cấp thức ăn, điều khiển phần cứng đến giám sát và quản lý dữ liệu trên ứng dụng di động, đồng thời đánh giá khả năng ứng dụng Edge AI trong một hệ thống IoT nhúng thực tế.

ID	Chức năng	Yêu cầu
FR-01	Chụp ảnh	ESP32-S3 CAM phải chụp ảnh khi luồng cho ăn được kích hoạt.
FR-02	Phân loại loài thú cưng	Mô hình Edge AI phải phân loại ảnh đã chụp vào một trong các lớp loài được hỗ trợ.
FR-03	Chọn cấu hình cho ăn	Hệ thống phải chọn cấu hình cho ăn tương ứng với loài được AI dự đoán.
FR-04	Cấp thức ăn tự động	Bộ điều khiển phải điều khiển servo để cấp lượng thức ăn đã cấu hình.
FR-05	Kiểm tra khối lượng	Hệ thống phải đo lượng thức ăn đã cấp bằng load cell/HX711 và dừng khi đạt mục tiêu trong phạm vi sai số được định nghĩa khi triển khai.
FR-06	Cho ăn thủ công	Ứng dụng di động phải có lệnh cho ăn thủ công để kiểm thử và vận hành có kiểm soát.
FR-07	Cài đặt theo loài	Ứng dụng phải cho phép cấu hình loại thức ăn, khẩu phần, lịch, giới hạn theo ngày và trạng thái auto-feed cho từng loài được hỗ trợ.
FR-08	Đồng bộ dữ liệu	Firebase phải lưu và đồng bộ cấu hình, trạng thái thiết bị và lịch sử cho ăn.
FR-09	Lịch sử & giám sát	Ứng dụng phải hiển thị các lần cho ăn gần đây, loài được nhận diện, lượng thức ăn và trạng thái thiết bị.
FR-10	Thông báo	Ứng dụng cần hỗ trợ thông báo cho các sự kiện quan trọng như hoàn tất cho ăn, sắp hết thức ăn hoặc trạng thái lỗi/thiết bị khi khả thi.

Thiết kế Edge AI

AI là chức năng cốt lõi của dự án. Mô hình chỉ chịu trách nhiệm phân loại loài thú cưng xuất hiện trước camera; quyết định cho ăn sau đó dựa trên cấu hình đã thiết lập cho loài tương ứng.

Nhiệm vụ	Phân loại ảnh đa lớp

Số lớp mục tiêu	Tối đa khoảng 5 loài thú cưng định nghĩa trước; (danh sách chính xác sẽ chốt trước khi thu thập dataset)
Vị trí suy luận	ESP32-S3 CAM (Edge AI)
Input	Ảnh camera, được resize và normalize cho mô hình
Output	Loài dự đoán + confidence score
Đề xuất mô hình	Tiny CNN hoặc mô hình MobileNet gọn nhẹ
Hướng tối ưu	INT8 quantization / TensorFlow Lite Micro hoặc stack inference tương thích ESP32

Quy tắc thiết kế quan trọng: thêm một thú cưng mới thuộc loài đã hỗ trợ không cần huấn luyện lại mô hình. Chỉ cần huấn luyện lại khi bổ sung một loài mới mà mô hình hiện tại chưa hỗ trợ.

Màn hình / Chức năng	Mục đích
Dashboard	Hiển thị trạng thái thiết bị, kết quả nhận diện mới nhất, lần cho ăn gần nhất và điều khiển nhanh.
Cài đặt theo loài	Cấu hình profile cho ăn cho từng loài được hỗ trợ.
Cho ăn thủ công	Gửi lệnh cho ăn thủ công có kiểm soát.
Lịch sử cho ăn	Hiển thị thời gian, loài, lượng mục tiêu, lượng thực tế và kết quả.
Thống kê	Tổng hợp lượng thức ăn và số lần cho ăn theo thời gian.
Thông báo	Hiển thị các cảnh báo quan trọng của hệ thống và quá trình cho ăn.
Cài đặt thiết bị	Cấu hình kết nối/thiết bị cơ bản khi cần.

4. Nội dung nghiên cứu

- Để đạt được các mục tiêu đã đề ra, nội dung nghiên cứu tập trung vào việc xây dựng và tích hợp hoàn chỉnh các thành phần của hệ thống.
- Xác định kiến trúc tổng thể và luồng dữ liệu giữa camera, ESP32-S3 CAM, mô hình Edge AI, khối điều khiển cấp thức ăn, servo, Load Cell/HX711, Firebase và ứng dụng React Native.
- Chuẩn bị dữ liệu ảnh chó và mèo, huấn luyện mô hình phân loại nhẹ, tối ưu mô hình để phù hợp với tài nguyên của ESP32-S3 CAM và tích hợp chức năng suy luận vào firmware.

- Phần cứng được thiết kế với hai bộ cấp thức ăn độc lập, trong đó gồm Servo và Load Cell dành riêng cho từng loại thú cưng. LED và buzzer được bổ sung để phản hồi trạng thái hoạt động. Firmware trên ESP32-S3 CAM được xây dựng theo dạng module, bao gồm camera, servo, HX711, hiệu chuẩn cảm biến, Feeding Controller, Wi-Fi, Firebase, quản lý trạng thái và các cơ chế xử lý lỗi. Thuật toán cấp thức ăn được thiết kế theo trình tự nhận diện loài, kiểm tra điều kiện cho ăn, lấy khẩu phần tương ứng, đo khối lượng ban đầu, mở servo và theo dõi lượng thức ăn tăng thêm cho đến khi đạt giá trị mục tiêu thì đóng servo.
- Xây dựng Firebase Realtime Database để lưu trữ và đồng bộ cấu hình, lệnh điều khiển, trạng thái thiết bị và lịch sử cho ăn. Ứng dụng Android phát triển bằng React Native được sử dụng để cấu hình hệ thống, gửi lệnh cho ăn thủ công, theo dõi trạng thái và xem lịch sử hoạt động.
- Tổ chức quá trình nhận diện thú cưng, cấp thức ăn, cập nhật dữ liệu lên Firebase đến hiển thị kết quả trên ứng dụng di động.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

2.1. Tổng quan hệ thống IoT và Edge AI

2.2. ESP32-S3 và ESP32-S3 CAM

2.3. Cơ sở lý thuyết về phân loại ảnh

Phân loại ảnh là bài toán trong lĩnh vực thị giác máy tính, trong đó mục tiêu là xác định ảnh đầu vào thuộc về một trong các lớp đã được định nghĩa trước. Với một ảnh đầu vào x, mô hình phân loại thực hiện ánh xạ: $f_\theta: x \rightarrow$ ŷ trong đó fθ là mô hình có tham số θ, còn ŷ là nhãn dự đoán của ảnh. Đối với bài toán phân loại C lớp, mô hình thường tạo ra một vector xác suất:

$$p = [p_1, p_2, \dots, pC]$$

với: $\sum_{i=1}^{c} p_i = 1$. Lớp dự đoán cuối cùng được xác định bằng:

$$\hat{y} = arg\ max_i\ p_i$$

Trong các hệ thống phân loại ảnh hiện đại, mạng nơ-ron tích chập (Convolutional Neural Network – CNN) được sử dụng rộng rãi nhờ khả năng tự động học các đặc trưng trực tiếp từ dữ liệu ảnh thay vì phải thiết kế đặc trưng thủ công. Ảnh đầu vào thường được biểu diễn dưới dạng tensor $X \in \mathbb{R}^H \times^W \times^c$. Trong đó H và W lần lượt là chiều cao và chiều rộng của ảnh, còn C là số kênh màu. Đối với ảnh RGB, C = 3. Trong CNN, lớp tích chập thực hiện phép toán giữa ảnh hoặc feature map đầu vào với các kernel có thể học được. Với một kernel K, giá trị tại vị trí (i, j) của feature map đầu ra biểu diễn:

$$Y(i,j) = \sum_m \sum_n X(i+m, j+n) K(m,n) + b$$

trong đó b là hệ số bias. Các kernel được cập nhật trong quá trình huấn luyện để phát hiện những đặc trưng có ý nghĩa trong ảnh. Ở các tầng đầu, mạng thường học các đặc trưng đơn giản như cạnh, đường thẳng và màu sắc. Khi đi qua các tầng sâu hơn, các đặc trưng này được kết hợp để tạo thành những đặc trưng phức tạp hơn như hình dạng, cấu trúc hoặc các bộ phận đặc trưng của đối tượng. Sau lớp tích chập, hàm kích hoạt phi tuyến thường được sử dụng để tăng khả năng biểu diễn của mạng. Một trong những hàm phổ biến nhất là ReLU:

$$ReLU(x) = max(0, x)$$

Ngoài ra, các lớp pooling có thể được sử dụng nhằm giảm kích thước không gian của feature map, giảm số lượng phép tính và tăng tính bất biến đối với sự thay đổi nhỏ về vị trí của đối tượng. Max Pooling được xác định bằng cách lấy giá trị lớn nhất trong vùng:

$$Y(i,j) = max_{(m,n)} \in R\ X(i+m, j+n)$$

với R là cửa sổ pooling. Sau nhiều lớp trích xuất đặc trưng, mạng thu được tensor đặc trưng $F \in \mathbb{R}^{H' \times W' \times D}$. trong đó D là số lượng kênh đặc trưng. Tensor này cần được chuyển thành vector đặc trưng trước khi thực hiện phân loại. Một phương pháp phổ biến là Global Average Pooling (GAP). Với mỗi kênh k, GAP tính giá trị trung bình trên toàn bộ không gian:

$$z_k = 1/(H'W') \sum_{i=1}^{H'} \sum_{j=1}^{W'} F_{ijk}$$

Kết quả thu được vector $z \in \mathbb{R}^D$. Vector đặc trưng sau đó được đưa qua lớp Fully Connected để tạo ra các giá trị đầu ra, còn gọi là logits:

$$o = Wz + b$$

trong đó W và b là các tham số được học trong quá trình huấn luyện. Đối với bài toán phân loại nhiều lớp, các logits được chuyển thành xác suất bằng hàm Softmax:

$$p_i = \frac{e^{oi}}{\sum_{j=1}^{c} e^{oj}}$$

Trong quá trình huấn luyện, Cross-Entropy Loss thường được sử dụng để đánh giá sai lệch giữa nhãn thực tế và xác suất dự đoán:

$$L = -\sum_{i=1}^{c} y_i\ log(p_i)$$

trong đó y_i là nhãn thực tế dưới dạng one-hot và p_i là xác suất dự đoán của lớp thứ i. Các tham số của mạng được tối ưu bằng thuật toán lan truyền ngược (Backpropagation). Gradient của hàm mất mát đối với từng tham số được tính toán và cập nhật theo quy tắc:

$$\theta_{t+1} = \theta_t - \eta \nabla \theta L$$

trong đó η là learning rate. Toàn bộ quá trình phân loại ảnh có thể được mô tả theo chuỗi:

Ảnh đầu vào → Trích xuất đặc trưng → Feature Map → Vector đặc trưng → Bộ phân loại → Xác suất → Nhãn dự đoán

Để đánh giá mô hình phân loại ảnh, các chỉ số phổ biến gồm Accuracy, Precision, Recall và F1-score. Accuracy biểu diễn tỷ lệ mẫu được dự đoán chính xác:

$$Accuracy = (TP + TN) / (TP + TN + FP + FN)$$

Precision biểu diễn mức độ chính xác của các mẫu được mô hình dự đoán là dương:

$$Precision = TP / (TP + FP)$$

Recall phản ánh khả năng phát hiện đúng các mẫu dương thực tế:

$$Recall = TP / (TP + FN)$$

F1-score là trung bình điều hòa giữa Precision và Recall:

$$F1 = 2 \times (Precision \times Recall) / (Precision + Recall)$$

Ngoài độ chính xác, đối với các hệ thống triển khai trên thiết bị nhúng hoặc Edge AI, cần xem xét thêm các yếu tố như số lượng tham số, kích thước mô hình, số phép tính MACs/FLOPs, dung lượng bộ nhớ và thời gian suy luận. Do đó, mục tiêu không chỉ là đạt độ chính xác cao mà còn phải cân bằng giữa hiệu năng phân loại và chi phí tính toán của mô hình.

2.4. Mô hình AI nhẹ cho Edge Device

Các mạng nơ-ron tích chập nhẹ được sử dụng rộng rãi cho bài toán phân loại ảnh trên thiết bị biên nhờ có số lượng tham số thấp, chi phí tính toán nhỏ và kích thước mô hình gọn. MobileNetV2 sử dụng inverted residual block và linear bottleneck, kết hợp với depthwise separable convolution nhằm giảm số lượng phép tính nhưng vẫn duy trì khả năng trích xuất đặc trưng. Dựa trên kiến trúc này, mô hình PetMobileNetV2-S3 được thiết kế với ảnh đầu vào 96x96, width multiplier 0.35, giảm số kênh đặc trưng cuối và thay lớp phân loại ImageNet bằng bốn lớp: Pet A, Pet B, Unknown và No Pet. MobileNetV3-Small tiếp tục cải tiến MobileNetV2 bằng cách kết hợp inverted residual, squeeze-and-excitation và tối ưu kiến trúc theo phần cứng. ShuffleNetV2 tập trung vào hiệu quả suy luận thực tế thông qua việc giảm chi phí truy cập bộ nhớ, kết hợp channel split và channel shuffle. Mô hình PetShuffleNetV2-S3 được xây dựng từ ShuffleNetV2-0.5×, sử dụng ảnh đầu vào 96×96 và thay lớp phân loại 1000 lớp bằng bốn lớp.

Trong những năm gần đây, nhiều kiến trúc thị giác nhẹ được phát triển nhằm cải thiện sự cân bằng giữa độ chính xác, độ trễ và chi phí tính toán trên thiết bị biên. MobileNetV4 giới thiệu Universal Inverted Bottleneck (UIB) và cơ chế Mobile MQA, hướng đến khả năng hoạt động hiệu quả trên nhiều loại phần cứng di động khác nhau. RepViT được đề xuất tại CVPR 2024 theo hướng thiết kế lại CNN nhẹ dựa trên các nguyên lý hiệu quả của Vision Transformer. Mô hình cho thấy CNN vẫn có khả năng đạt tỷ lệ accuracy–latency cạnh tranh trên thiết bị khi được tối ưu đúng kiến trúc.

Ở hướng Transformer nhẹ, SHViT sử dụng **Single-Head Attention** kết hợp thiết kế macro tối ưu bộ nhớ nhằm giảm dư thừa tính toán của Multi-Head Attention. Kiến trúc này đạt hiệu quả tốt về cả độ chính xác và latency trên CPU, GPU và thiết bị di động. Bên cạnh CNN và Transformer, Vision Mamba sử dụng Bidirectional State Space Model để học biểu diễn ảnh mà không phụ thuộc vào cơ chế self-attention truyền thống. Kiến trúc này cho thấy tiềm năng giảm chi phí bộ nhớ và tính toán, đặc biệt đối với ảnh có độ phân giải lớn. Gần đây, NuWa được giới thiệu tại CVPR 2026 nhằm tạo ra các Vision Transformer nhỏ, chuyên biệt theo từng nhóm lớp cho thiết bị biên. Phương pháp này tập trung loại bỏ các trọng số không cần thiết và tạo mô hình nhỏ hơn phù hợp với các yêu cầu tài nguyên khác nhau của Edge Device. Từ các nghiên cứu trên, MobileNetV2 có thể được sử dụng làm baseline truyền thống, trong khi MobileNetV4, RepViT, SHViT và các kiến trúc mới như Vision Mamba hoặc NuWa được sử dụng để đại diện cho các hướng phát triển hiện đại. Trong nghiên cứu này, các mô hình được so sánh dựa trên Accuracy, Macro-F1, số lượng tham số, MACs, kích thước mô hình, thời gian suy luận và mức sử dụng bộ nhớ nhằm lựa chọn kiến trúc phù hợp cho triển khai trên thiết bị Edge.

2.5. Tối ưu mô hình trên ESP32-S3

Việc triển khai mô hình học sâu trên ESP32-S3 gặp hạn chế về bộ nhớ, năng lực tính toán và mức tiêu thụ năng lượng. Do đó, các nghiên cứu về Edge AI thường tập trung vào các kỹ thuật giảm độ phức tạp của mô hình nhưng vẫn duy trì độ chính xác ở mức chấp nhận được. Một trong những kỹ thuật phổ biến nhất là **quantization**, trong đó trọng số và activation được chuyển từ FP32 sang các định dạng có độ chính xác thấp hơn như INT8. Phương pháp này giúp giảm đáng kể kích thước mô hình và lượng bộ nhớ sử dụng, đồng thời tăng tốc quá trình suy luận trên phần cứng có hỗ trợ phép toán

số nguyên. Đối với ESP32-S3, ESP-DL hỗ trợ các cấu hình lượng tử hóa như W8A8, W16A16 và mixed precision, trong đó INT8 thường được ưu tiên cho các ứng dụng yêu cầu mô hình nhỏ và thời gian suy luận thấp. Bên cạnh lượng tử hóa, việc giảm độ phân giải ảnh đầu vào cũng là một phương pháp hiệu quả để giảm chi phí tính toán. Với các mô hình CNN, số lượng phép toán phụ thuộc mạnh vào kích thước feature map, do đó việc giảm input từ 224x224 xuống 96x96 có thể làm giảm đáng kể số lượng MACs và bộ nhớ activation. Các kiến trúc sử dụng Depthwise Separable Convolution, inverted residual hoặc channel shuffle cũng được sử dụng nhằm giảm số lượng tham số và phép toán. Những kỹ thuật này là nền tảng của các mô hình nhẹ như MobileNetV2, MobileNetV3 và ShuffleNetV2.

Ngoài kích thước mô hình, mức sử dụng bộ nhớ trong quá trình suy luận cũng cần được xem xét. Activation và tensor trung gian có thể chiếm lượng RAM lớn hơn trọng số của mô hình, đặc biệt khi xử lý feature map có kích thước lớn. Vì vậy, khả năng triển khai thực tế cần được đánh giá trực tiếp dựa trên kích thước mô hình, latency, RAM/PSRAM và độ chính xác sau lượng tử hóa. Các kỹ thuật như giảm độ phân giải đầu vào, sử dụng kiến trúc CNN nhẹ và lượng tử hóa INT8 đóng vai trò quan trọng trong việc đưa mô hình phân loại ảnh xuống các thiết bị Edge có tài nguyên hạn chế.

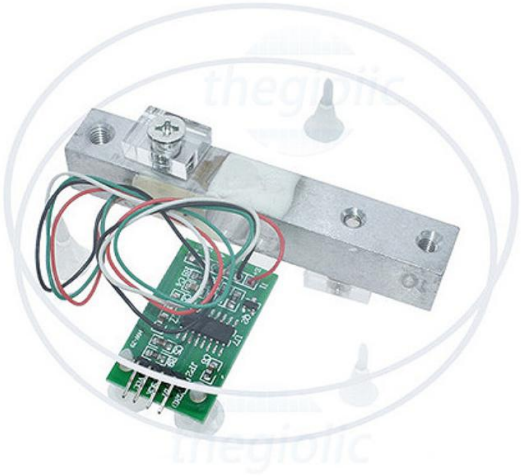
2.6. Servo Motor và cơ cấu cấp thức ăn

Động cơ Servo đóng vai trò là cơ cấu chấp hành chính trong hệ thống nhằm định lượng và nhả thức ăn một cách chính xác. Trong dự án này, mô-tơ Servo MG90S được lựa chọn sử dụng nhờ sự cân bằng hoàn hảo giữa thiết kế nhỏ gọn (trọng lượng chỉ khoảng 9g – 13.4g) và hiệu năng vận hành. Động cơ hoạt động ổn định ở dải điện áp 4.8V – 6V, cung cấp lực kéo khá khỏe lên tới 2.2 kg·cm (ở 4.8V). Đặc biệt, hệ thống bánh răng bên trong MG90S được làm bằng kim loại, mang lại độ bền cơ học và khả năng chịu tải vượt trội hơn hẳn so với loại bánh răng nhựa truyền thống như SG90, giúp hạn chế tối đa tình trạng tuôn hay mẻ nhông khi cơ cấu xả thức ăn vô tình bị kẹt hạt. Khác với động cơ DC thông thường, Servo MG90S cho phép vi điều khiển ESP32 kiểm soát chính xác vị trí góc quay từ 0 đến 180 độ thông qua tín hiệu xung PWM. Về mặt cơ khí, cơ cấu cấp thức ăn thường được thiết kế theo dạng cánh quạt chia ngăn hoặc cửa sập trượt. Mỗi khi nhận lệnh, Servo sẽ xoay một góc quy định để đẩy một lượng thức ăn chuẩn xác xuống phễu xả hoặc kéo mở tấm chắn trong một khoảng thời gian trễ nhất

định. Việc ứng dụng Servo MG90S kết hợp cùng các cơ cấu cơ khí này không chỉ giúp định lượng chuẩn khẩu phần ăn mà còn ngăn chặn tình trạng thú cưng tò mò tự cạy cửa xả nhờ lực hãm (holding torque) vững chắc từ hộp số kim loại của động cơ.

2.7. Load Cell và HX711

Load Cell là thiết bị cảm biến trọng lượng đảm nhiệm việc chuyển đổi lực cơ học từ khối lượng thức ăn trên đĩa cân thành tín hiệu điện tương tự. Thành phần cốt lõi của cảm biến này là các điện trở biến dạng (Strain Gauge) được mắc theo cấu trúc mạch cầu Wheatstone. Khi có lực tác động, thanh kim loại bên trong Load Cell bị uốn cong ở mức độ vi mô, làm thay đổi giá trị điện trở của Strain Gauge và sinh ra sự chênh lệch điện áp ở ngõ ra. Tuy nhiên, mức biến thiên điện áp này cực kỳ nhỏ (thường chỉ ở dải millivolt), khiến các chân Analog thông thường trên vi điều khiển không thể đọc được chính xác. Do đó, hệ thống bắt buộc phải sử dụng module HX711. Module này tích hợp bộ khuếch đại nhiễu thấp (PGA - Programmable Gain Amplifier) để phóng to tín hiệu vi sai, sau đó dùng bộ chuyển đổi tương tự sang số (ADC) độ phân giải 24-bit để số hóa tín hiệu.



Dữ liệu số cuối cùng được truyền về ESP32 thông qua giao thức nối tiếp đồng bộ hai dây (DT và SCK). Sự kết hợp giữa Load Cell và HX711 tạo ra một cơ chế phản hồi vòng kín (closed-loop feedback) hoàn chỉnh, giúp hệ thống liên tục giám sát khối lượng thức ăn thực tế theo thời gian thực. Nhờ đó, ESP32 có cơ sở dữ liệu để quyết định việc đóng mở Servo bù thêm thức ăn, đồng thời gửi cảnh báo qua ứng dụng nếu phát hiện sự cố như bát ăn bị lật hoặc thú cưng bỏ bữa.

2.8. Firebase

Firebase là nền tảng backend-as-a-service được sử dụng trong hệ thống AI Smart Pet Feeder để xác thực người dùng và tạo kênh đồng bộ dữ liệu giữa ứng dụng React Native với ESP32-S3. Thay vì để điện thoại kết nối trực tiếp đến địa chỉ IP cục bộ của bộ điều khiển, hai phía cùng đọc và ghi trên Firebase Realtime Database. Cách tổ chức này phù hợp với hệ thống IoT vì điện thoại và thiết bị có thể nằm ở hai mạng khác nhau, đồng thời giúp tách lớp giao diện khỏi lớp điều khiển phần cứng [1], [2].

Firebase Authentication được sử dụng theo phương thức Email/Password. Ứng dụng theo dõi trạng thái đăng nhập bằng onAuthStateChanged(); chỉ sau khi người dùng xác thực thành công, FeederContext mới khởi tạo lớp FirebaseFeederRepository và đăng ký listener tới dữ liệu thiết bị. Các thao tác đăng nhập, đăng ký và đăng xuất tương ứng sử dụng signInWithEmailAndPassword(), createUserWithEmailAndPassword() và signOut(). Trong phiên bản hiện tại, tài khoản dùng cho xác thực, còn cấu hình máy cho ăn được tổ chức theo thiết bị dùng chung feeder-01 chứ không lưu dưới users/{uid}.

Firebase Realtime Database là cơ sở dữ liệu NoSQL dạng cây JSON. Root dữ liệu mà ứng dụng và firmware sử dụng là devices/feeder-01. Ứng dụng đăng ký onValue() cho toàn bộ node thiết bị và theo dõi thêm .info/connected để biết client mobile còn kết nối tới Firebase hay không. Khi ESP32 cập nhật heartbeat, trạng thái cấp thức ăn, số gram đo được hoặc kết quả AI, listener nhận snapshot mới và giao diện được cập nhật mà không cần người dùng tải lại thủ công [2].

Bảng 1: Cấu trúc dữ liệu Firebase của hệ thống

Đường dẫn Firebase	Nội dung
devices/feeder-01/status	Trạng thái online, phiên bản firmware, địa chỉ IP, RSSI, heartbeat, chế độ hoạt động và mã lỗi.
devices/feeder-01/config/speciesProfiles/{dog|cat}	Cấu hình khẩu phần, giới hạn ngày, cooldown, auto-feed và thông báo cho từng loài.
devices/feeder-01/config/schedules/{dog|cat}/{scheduleId}	Lịch cho ăn theo từng ngăn; mỗi lịch gồm thời gian, khẩu phần và trạng thái enabled.

devices/feeder-01/command	Một command slot dùng cho lệnh Feed Now và trạng thái xử lý của lệnh.
devices/feeder-01/runtime/feeding	Trạng thái cấp thức ăn hiện tại, loài đang xử lý, targetGrams, actualGrams và LED.
devices/feeder-01/runtime/cooldown	Thời điểm hết cooldown riêng cho chó và mèo.
devices/feeder-01/runtime/latestDetection	Kết quả nhận diện gần nhất gồm speciesId, confidence và thời gian.
devices/feeder-01/feedingHistory/{pushId}	Lịch sử thực tế do firmware ghi sau mỗi lần cấp thức ăn thành công hoặc thất bại.

Đối với Feed Now, ứng dụng không tạo một danh sách feeding_commands như thiết kế cũ mà sử dụng một singleton command slot. Trước khi ghi, ứng dụng kiểm tra kết nối Firebase, heartbeat, errorCode, trạng thái runtime, cooldown, cấu hình profile và giới hạn ngày. Lệnh mới được giành quyền ghi bằng runTransaction(); nếu command hiện tại đang pending hoặc processing thì transaction bị hủy để tránh hai điện thoại ghi đè lên cùng một lệnh. Việc Firebase chấp nhận lệnh chỉ có nghĩa yêu cầu đã được lưu, không có nghĩa cơ cấu cấp thức ăn đã hoàn thành.

ESP32 gửi heartbeat theo chu kỳ 10 giây, trong khi ứng dụng sử dụng ngưỡng bảo vệ 120 giây để đánh giá heartbeat còn mới. .info/connected phản ánh kết nối của điện thoại tới Firebase, còn status.online và lastSeenEpoch phản ánh tình trạng của thiết bị. Việc tách hai khái niệm này giúp app không nhầm trường hợp điện thoại có Internet nhưng ESP32 đã mất kết nối.

Project vẫn cài module @react-native-firebase/messaging, tuy nhiên cờ FIREBASE_MESSAGING_ENABLED đang đặt false và contract hiện tại không định nghĩa đường dẫn token FCM. Vì vậy, phiên bản trong mã nguồn chỉ có thông báo trong ứng dụng được suy ra từ snapshot/lịch sử; chưa nên mô tả là đã có push notification nền. Firebase Cloud Messaging vẫn là hướng mở rộng phù hợp khi nhóm bổ sung token path và backend gửi thông báo [3].

Security Rules đi kèm source hiện cho phép tài khoản đã xác thực truy cập nhánh devices. Cấu hình này phù hợp cho demo môn học nhưng chưa đủ chặt cho sản phẩm nhiều người dùng. Khi triển khai thực tế cần bổ sung quan hệ user-device, giới hạn

quyền app đối với status/runtime/history và có cơ chế định danh riêng cho ESP32 để ngăn người dùng giả trạng thái completed hoặc tự tạo lịch sử.

2.9. React Native

React Native là framework mã nguồn mở dùng để xây dựng ứng dụng di động bằng React, JavaScript hoặc TypeScript. Các thành phần như View, Text, TextInput, Pressable và ScrollView được ánh xạ sang thành phần native của nền tảng, nhờ đó ứng dụng sử dụng một cơ sở mã chung nhưng vẫn có thể gọi các native module trên Android [4]. Ứng dụng AI Smart Pet Feeder được xây dựng bằng React Native CLI 0.86.3, React 19.2.3 và TypeScript. TypeScript giúp định nghĩa rõ hồ sơ loài, lịch, lệnh điều khiển, trạng thái runtime và lịch sử, giảm lỗi khi dữ liệu đi qua nhiều lớp của ứng dụng [5].

Bảng 2: Tổ chức mã nguồn ứng dụng React Native

Thành phần	Vai trò
screens	Chứa Dashboard, Species Profiles, Feed Now, Schedule, History, Statistics, Notifications, Device Settings, More và Login.
components	Chứa các thành phần giao diện tái sử dụng như Card, PrimaryButton, SpeciesSelector, StatusBadge và BottomNavigation.
context/FeederContext.tsx	Quản lý trạng thái đăng nhập, AppSnapshot, loading/error và cung cấp các thao tác nghiệp vụ cho toàn ứng dụng.
repositories	Tách giao diện khỏi quá trình đọc/ghi Firebase; FirebaseFeederRepository hiện thực các thao tác RTDB.
firebase	Chứa config.ts và deviceContract.ts: cấu hình RTDB, heartbeat, messaging và adapter chuẩn hóa dữ liệu Firebase.
types + utils	Định nghĩa model TypeScript và các hàm định dạng/thao tác hỗ trợ giao diện.

Ứng dụng không để từng screen truy cập Firebase trực tiếp. RootApp xử lý điều hướng; FeederContext quản lý state dùng chung; FirebaseFeederRepository thực hiện read/write/transaction; deviceContract.ts đóng vai trò adapter giữa schema Firebase và

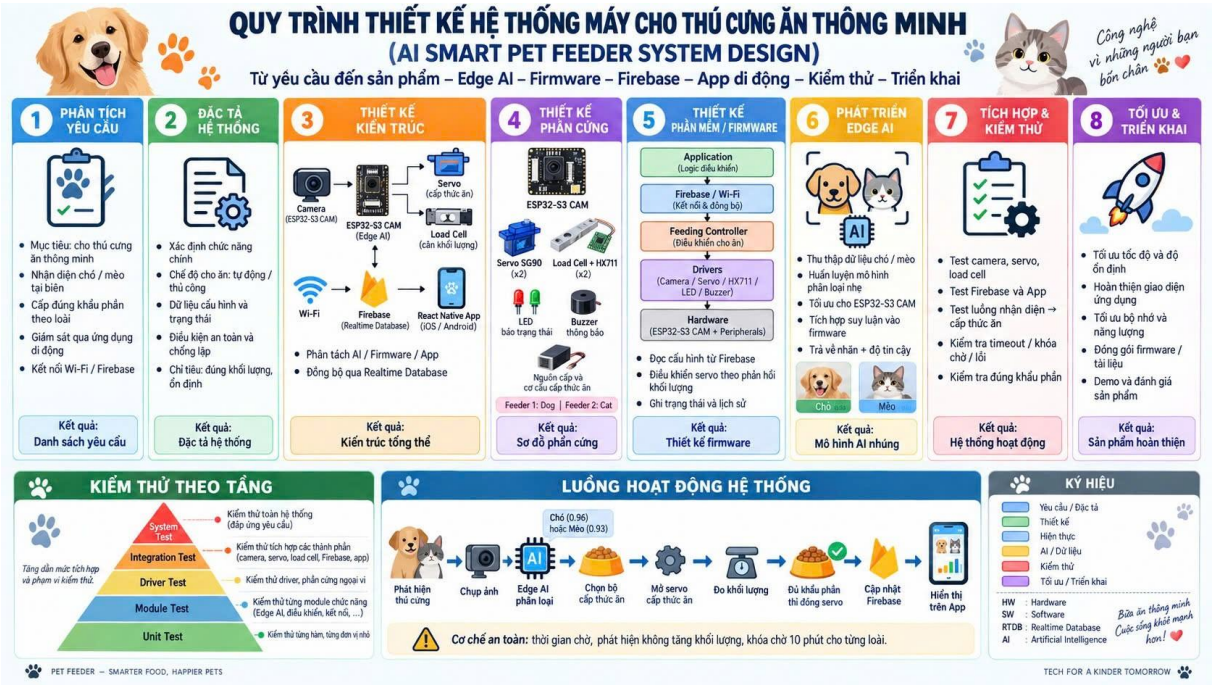
dữ liệu giao diện. Cách phân lớp này giúp thay đổi contract ở một nơi thay vì sửa toàn bộ các màn hình, đồng thời thuận lợi cho kiểm thử.

Các module @react-native-firebase/app, auth và database kết nối TypeScript với Firebase SDK native của Android. Project sử dụng Hermes và New Architecture của React Native [6], [7]. Ứng dụng không chạy AI; TensorFlow Lite Micro chạy trên ESP32-S3 và app chỉ nhận speciesId, confidence cùng trạng thái thiết bị qua Firebase để hiển thị, cấu hình, điều khiển và giám sát.

CHƯƠNG 3: PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

3.1. Yêu cầu hệ thống

Yêu cầu hệ thống được xác định dựa trên mục tiêu xây dựng một thiết bị cho ăn thông minh có khả năng nhận diện thú cưng tại biên, điều khiển lượng thức ăn theo phản hồi khối lượng và đồng bộ dữ liệu với ứng dụng di động. Các yêu cầu được chia thành hai nhóm: yêu cầu chức năng mô tả những chức năng hệ thống phải thực hiện và yêu cầu phi chức năng quy định các đặc tính về độ ổn định, an toàn, khả năng bảo trì.



3.1.1. Yêu cầu chức năng

Các yêu cầu chức năng được xác định dựa trên chuỗi hoạt động chính của hệ thống, từ thu nhận ảnh, nhận diện thú cưng, lựa chọn bộ cấp thức ăn, điều khiển cấp theo phản hồi khối lượng đến đồng bộ dữ liệu với Firebase và ứng dụng di động. Các yêu cầu này là cơ sở để thiết kế firmware, phần cứng và các chức năng phần mềm ở các mục tiếp theo.

Bảng 3.1. Yêu cầu chức năng của hệ thống

ID	Chức năng	Yêu cầu
01	Khởi tạo và thu nhận ảnh	ESP32-S3 CAM phải khởi tạo camera thành công và cung cấp khung ảnh đầu vào cho module Edge AI.

ID	Chức năng	Yêu cầu
02	Phân loại thú cưng	Mô hình Edge AI phải phân loại ảnh thành chó, mèo hoặc không xác định và trả về độ tin cậy để firmware đánh giá.
03	Chọn bộ cấp thức ăn	Firmware phải ánh xạ chó đến bộ cấp thức ăn số 1 (Servo 1 + Load Cell 1) và mèo đến bộ cấp thức ăn số 2 (Servo 2 + Load Cell 2).
04	Đọc cấu hình khẩu phần	Hệ thống phải đọc **khối lượng thức ăn mục tiêu** từ Firebase khi có giá trị cấu hình hợp lệ; nếu không có, firmware sử dụng **khẩu phần mặc định** đã được thiết lập sẵn.
05	Cấp thức ăn theo phản hồi khối lượng	Servo phải mở để xả thức ăn và Load Cell/HX711 phải đo khối lượng tăng thêm. Servo phải đóng khi lượng thức ăn đạt gần giá trị mục tiêu.
06	Cho ăn thủ công	Ứng dụng phải có khả năng gửi lệnh cho ăn thủ công qua Firebase để ESP32 thực hiện một chu kỳ cấp thức ăn có kiểm soát.
07	Cho ăn tự động từ AI	Khi kết quả nhận diện hợp lệ và đạt **ngưỡng độ tin cậy yêu cầu**, firmware phải sử dụng **kết quả phân loại loài** để lựa chọn cơ chế cấp thức ăn tương ứng và thực hiện quá trình cho ăn tự động.
08	Thời gian chờ giữa hai lần cho ăn	Sau khi hoàn tất một lần cấp thức ăn, hệ thống phải áp dụng **khoảng thời gian chờ riêng cho từng loài** trước khi cho phép thực hiện lần cấp tiếp theo.
09	Xử lý lỗi cấp thức ăn	Firmware phải đóng servo và kết thúc chu kỳ nếu vượt timeout, Load Cell không sẵn sàng hoặc khối lượng không tăng trong khoảng thời gian cho phép.

ID	Chức năng	Yêu cầu
10	Phản hồi trạng thái tại thiết bị	LED phải sáng trong thời gian cấp thức ăn; buzzer được sử dụng để báo hoàn tất hoặc báo lỗi theo quy ước của firmware.
11	Đồng bộ Firebase	ESP32-S3 CAM phải có khả năng **đọc cấu hình và lệnh điều khiển từ Firebase**, đồng thời **cập nhật trạng thái thiết bị, trạng thái cấp thức ăn và lịch sử hoạt động** lên Firebase.
12	Giám sát trên ứng dụng	Ứng dụng React Native phải hiển thị trạng thái thiết bị, kết quả nhận diện mới nhất, cấu hình khẩu phần và lịch sử cho ăn từ Firebase.

3.1.2. Yêu cầu phi chức năng

Bên cạnh các chức năng chính, hệ thống cần đáp ứng một số yêu cầu phi chức năng để đảm bảo khả năng vận hành ổn định và thuận lợi trong quá trình phát triển, kiểm thử và mở rộng.

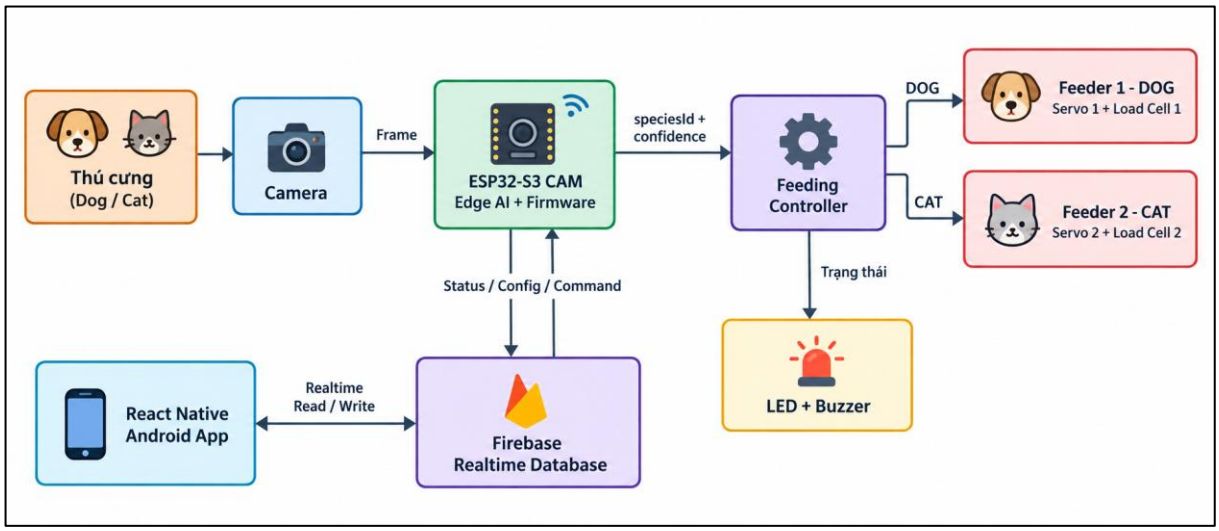
Bảng 3.2. Yêu cầu phi chức năng của hệ thống

ID	Thuộc tính	Yêu cầu
01	Độ ổn định	Các module camera, servo, HX711, Wi-Fi và Firebase phải được khởi tạo độc lập; lỗi ở một module phải được ghi nhận và không được làm servo duy trì trạng thái mở ngoài kiểm soát.
02	An toàn điều khiển	Trong mọi tình huống lỗi liên quan đến timeout, mất phản hồi khối lượng hoặc dữ liệu không hợp lệ, firmware phải ưu tiên đóng servo và tắt LED cấp thức ăn.
03	Khả năng phản hồi	Việc đọc Load Cell trong chu kỳ cấp thức ăn phải được thực hiện liên tục theo chu kỳ lấy mẫu để hệ thống có thể dừng xả khi khối lượng tiến gần mục tiêu.

ID	Thuộc tính	Yêu cầu
04	Khả năng bảo trì	Firmware phải được tổ chức theo các module độc lập như camera, AI, servo, load cell, logic cho ăn và Firebase; giao tiếp giữa AI và firmware.
05	Khả năng mở rộng	Thiết kế phải cho phép thay đổi model AI, khẩu phần hoặc ngưỡng điều khiển mà không cần viết lại toàn bộ phần điều khiển servo/load cell.
06	Khả năng hoạt động khi mạng không ổn định	Các chức năng điều khiển phần cứng cơ bản phải không phụ thuộc trực tiếp vào giao diện di động; firmware phải có cơ chế retry Wi-Fi/Firebase và dùng giá trị khẩu phần mặc định khi cấu hình từ Firebase không khả dụng.
07	Giới hạn tài nguyên Edge	Model AI và các buffer xử lý ảnh phải phù hợp bộ nhớ của ESP32-S3 CAM, tận dụng OPI PSRAM và tránh chiếm dụng tài nguyên ảnh hưởng đến điều khiển thời gian thực của cơ cấu cấp thức ăn.

3.2. Kiến trúc tổng thể hệ thống

Hệ thống được tổ chức theo kiến trúc phân lớp, trong đó ESP32-S3 CAM là nút xử lý trung tâm kết nối giữa khối nhận diện Edge AI, khối điều khiển phần cứng và nền tảng Firebase. Việc phân tách này giúp giới hạn rõ trách nhiệm của từng thành phần: AI chỉ thực hiện nhận diện; Feeding Controller chịu trách nhiệm ra quyết định cấp thức ăn; Servo và Load Cell thực thi và phản hồi quá trình cấp thức ăn; Firebase và ứng dụng di động đảm nhiệm cấu hình, đồng bộ và giám sát.



Hình 3.1. Kiến trúc tổng thể hệ thống AI Smart Pet Feeder

Theo Hình 3.1, hình ảnh từ camera được đưa vào ESP32-S3 CAM để thực hiện tiền xử lý và suy luận bằng mô hình AI **tại thiết bị biên**. Kết quả nhận diện bao gồm **loài thú cưng được dự đoán** (chó, mèo hoặc không xác định) và **độ tin cậy của kết quả nhận diện**. Mô hình AI không trực tiếp điều khiển cơ cấu cấp thức ăn mà chỉ cung cấp kết quả nhận diện cho **khối điều khiển cấp thức ăn**. Cách phân chia này giúp phần điều khiển của hệ thống vẫn có thể hoạt động ổn định ngay cả khi mô hình AI được thay đổi hoặc tối ưu trong quá trình phát triển.

Khối điều khiển cấp thức ăn đóng vai trò trung gian giữa kết quả nhận diện và các cơ cấu chấp hành. Khi hệ thống nhận diện chó, **bộ cấp thức ăn số 1**, gồm Servo 1 và Load Cell 1, được lựa chọn; khi nhận diện mèo, hệ thống sử dụng **bộ cấp thức ăn số 2**, gồm Servo 2 và Load Cell 2. Trước khi thực hiện cấp thức ăn, hệ thống kiểm tra **thời gian chờ giữa hai lần cho ăn** và xác định **khối lượng thức ăn mục tiêu**. Giá trị này được ưu tiên lấy từ cấu hình hoặc lệnh điều khiển được lưu trên Firebase; nếu không có giá trị hợp lệ, phần mềm nhúng sẽ sử dụng khẩu phần mặc định đã được thiết lập cho từng loài.

Quá trình cấp thức ăn được điều khiển dựa trên phản hồi khối lượng. Trước khi mở servo, phần mềm nhúng đo khối lượng ban đầu của khay thông qua **cảm biến Load Cell kết hợp với mạch chuyển đổi tín hiệu HX711**. Trong quá trình xả, lượng thức ăn mới được cấp được xác định bằng độ chênh lệch giữa khối lượng hiện tại và khối lượng

ban đầu. Khi lượng thức ăn tăng thêm đạt gần **khối lượng mục tiêu**, servo được đóng lại để kết thúc quá trình cấp. Cách điều khiển này giúp phần thức ăn đã có sẵn trong khay không bị tính vào lượng thức ăn mới cấp, đồng thời cho độ chính xác cao hơn so với phương pháp chỉ điều khiển servo theo một khoảng thời gian cố định.

Firebase đóng vai trò là cầu nối dữ liệu giữa ESP32-S3 CAM và ứng dụng di động phát triển bằng React Native. Dữ liệu của hệ thống được tổ chức thành các nhóm chính gồm **lệnh điều khiển, cấu hình thiết bị, trạng thái hoạt động, trạng thái thiết bị và lịch sử cho ăn**. Ứng dụng di động gửi lệnh và cập nhật cấu hình thông qua Firebase, trong khi ESP32-S3 CAM ghi lại trạng thái vận hành, kết quả cấp thức ăn và lịch sử hoạt động. Việc thống nhất cấu trúc dữ liệu giúp phần mềm nhúng và ứng dụng di động có thể được phát triển tương đối độc lập nhưng vẫn trao đổi dữ liệu theo một quy ước chung.

Ở phía thiết bị, **đèn LED** và **còi buzzer** được sử dụng để cung cấp phản hồi trực quan và âm thanh cho người dùng. Đèn LED sáng trong thời gian hệ thống đang cấp thức ăn, trong khi còi được sử dụng để báo hiệu khi quá trình cấp thức ăn hoàn tất hoặc khi xảy ra lỗi. Bên cạnh đó, phần mềm nhúng còn tích hợp các cơ chế bảo vệ như **giới hạn thời gian cấp thức ăn**, **phát hiện trường hợp khối lượng không tăng** và **thời gian chờ giữa hai lần cho ăn đối với từng loài**. Các cơ chế này giúp hạn chế tình trạng servo hoạt động quá lâu hoặc thú cưng liên tục kích hoạt quá trình cấp thức ăn trong một khoảng thời gian ngắn.

Bảng 3.3. Vai trò các khối trong kiến trúc tổng thể

Khối	**Vai trò chính**
Camera	Thu nhận hình ảnh thú cưng và cung cấp dữ liệu đầu vào cho Edge AI.
Khối Edge AI	Thực hiện tiền xử lý ảnh, suy luận mô hình và trả về **loài thú cưng được nhận diện** cùng **độ tin cậy của kết quả nhận diện**.

Khối	Vai trò chính
Khối điều khiển cấp thức ăn	Tiếp nhận kết quả nhận diện, kiểm tra điều kiện cho ăn, xác định khối lượng thức ăn mục tiêu và điều phối quá trình cấp thức ăn.
Servo + Load Cell/HX711	Servo thực hiện mở và đóng cơ cấu xả thức ăn; Load Cell kết hợp HX711 đo khối lượng để phản hồi cho hệ thống và xác định thời điểm kết thúc cấp thức ăn.
LED + Buzzer	Cung cấp tín hiệu trực quan và âm thanh để thông báo trạng thái đang cấp thức ăn, hoàn tất hoặc xảy ra lỗi.
Firebase Realtime Database	Đồng bộ lệnh điều khiển, cấu hình hệ thống, trạng thái hoạt động, trạng thái thiết bị và lịch sử cho ăn giữa ESP32-S3 CAM và ứng dụng di động.
Ứng dụng React Native	Cho phép người dùng cấu hình khẩu phần, gửi lệnh cho ăn thủ công, theo dõi trạng thái hệ thống và xem lịch sử hoạt động.

Như vậy, kiến trúc tổng thể được xây dựng theo nguyên tắc tách lớp rõ ràng giữa nhận diện, điều khiển và giám sát. Edge AI chịu trách nhiệm nhận biết loài; firmware quyết định và kiểm soát quá trình cấp thức ăn; phần cứng thực thi; Firebase và ứng dụng đảm nhiệm trao đổi dữ liệu với người dùng. Cấu trúc này phù hợp với mục tiêu triển khai Edge AI trên thiết bị nhúng và tạo nền tảng thuận lợi cho việc kiểm thử, bảo trì và mở rộng hệ thống trong các phiên bản tiếp theo.

3.3. Thiết kế phần cứng

3.3.1. ESP32-S3 CAM

Trong dự án này, vi điều khiển ESP32-S3 đóng vai trò là bộ điều khiển trung tâm điều phối toàn bộ các hoạt động của hệ thống, từ thu thập hình ảnh, xử lý AI (TensorFlow Lite), đến giao tiếp ngoại vi và truyền phát dữ liệu. ESP32-S3 được trang bị bộ vi xử lý lõi kép Xtensa LX7 mạnh mẽ, tích hợp sẵn kết nối Wi-Fi 2.4 GHz, cho phép phân luồng tác vụ (ví dụ: Core 0 xử lý Camera/Wi-Fi, Core 1 xử lý AI). Điểm cốt lõi khiến phần cứng này được lựa chọn là dung lượng PSRAM lên đến 8MB, cung cấp không gian lưu trữ rộng rãi để tạo bộ đệm khung hình cho Camera và cấp phát vùng

nhớ dung lượng lớn cho mạng nơ-ron tích chập (CNN) hoạt động mượt mà mà không gặp lỗi cạn kiệt bộ nhớ.



Trong đề tài, ESP32-S3 CAM đảm nhiệm các chức năng chính:

- Điều khiển và giao tiếp với camera để thu nhận hình ảnh.
- Đọc dữ liệu khối lượng từ module HX711.
- Xử lý dữ liệu thu được từ camera và Load Cell.
- Phát tín hiệu PWM để điều khiển servo motor.
- Thực hiện thuật toán điều khiển và đưa ra quyết định cho cơ cấu chấp hành.
- Sử dụng kết nối Wi-Fi để truyền dữ, hoặc hình ảnh đến thiết bị bên ngoài.

Việc lựa chọn ESP32-S3 CAM giúp giảm số lượng linh kiện trong hệ thống do vi điều khiển vừa đảm nhiệm xử lý dữ liệu vừa hỗ trợ kết nối camera và kết nối không dây.

3.3.2. Camera

Camera được sử dụng để thu nhận hình ảnh của đối tượng trong phạm vi hoạt động của hệ thống. Tín hiệu hình ảnh được truyền đến ESP32-S3 CAM để xử lý hoặc sử dụng làm dữ liệu đầu vào cho thuật toán nhận dạng.

Tùy thuộc vào module ESP32-S3 CAM được sử dụng, camera có thể sử dụng các cảm biến hình ảnh phổ biến như OV2640 hoặc các cảm biến tương đương. Camera có nhiệm vụ chuyển đổi hình ảnh quang học thành dữ liệu số để vi điều khiển có thể tiếp nhận và xử lý.

Trong hệ thống, camera thực hiện các chức năng:

1. Thu nhận hình ảnh từ môi trường.
2. Chuyển đổi hình ảnh thành dữ liệu số.
3. Truyền dữ liệu hình ảnh đến ESP32-S3 CAM.
4. Cung cấp dữ liệu đầu vào cho quá trình xử lý và nhận dạng đối tượng.

Khoảng cách từ camera đến đối tượng, góc quan sát, điều kiện ánh sáng và độ phân giải ảnh là những yếu tố có ảnh hưởng trực tiếp đến chất lượng dữ liệu đầu vào. Vì vậy, camera cần được cố định ở vị trí phù hợp để đảm bảo vùng quan sát bao phủ khu vực cần nhận dạng.

3.3.3. Servo Motor

Để phục vụ cơ cấu xả thức ăn kép, hệ thống sử dụng hai động cơ Servo MG90S hoạt động độc lập. Mỗi servo đảm nhiệm một cơ cấu xả riêng, tương ứng với một khay thức ăn. Nhờ đó, ESP32-S3 có thể điều khiển hai cửa xả một cách độc lập và thực hiện việc phân phối lượng thức ăn khác nhau cho từng khay.

Servo MG90S là loại micro servo có kích thước nhỏ gọn, sử dụng bánh răng kim loại và phù hợp với các cơ cấu có yêu cầu về không gian lắp đặt. Việc sử dụng bánh răng kim loại giúp servo có khả năng chịu tải và độ bền cơ học tốt hơn so với các loại servo sử dụng bánh răng nhựa trong cùng phân khúc.

Mỗi Servo MG90S có ba đường kết nối cơ bản:

- **VCC:** cấp nguồn cho servo.
- **GND:** nối đất.
- **Signal:** nhận tín hiệu PWM từ ESP32-S3.

Hai servo được điều khiển bằng hai kênh PWM độc lập. Ví dụ, Servo 1 có thể được điều khiển bởi GPIO 38 và Servo 2 bởi GPIO 39 nếu các GPIO này phù hợp với phiên bản ESP32-S3 CAM thực tế được sử dụng.

Việc điều khiển độc lập giúp hệ thống có thể thực hiện các thao tác như mở cửa xả thứ nhất trong khi cửa thứ hai đóng, hoặc thiết lập thời gian và góc quay khác nhau cho từng servo để đạt được lượng thức ăn mong muốn.

Do servo có dòng tiêu thụ thay đổi theo tải và có thể xuất hiện dòng đỉnh khi khởi động hoặc khi bị tải cơ học, hai servo nên được cấp nguồn từ đường 5V riêng có khả năng cung cấp đủ dòng. Không nên cấp nguồn trực tiếp cho servo từ chân 3.3V của ESP32-S3.

3.3.4. Load Cell và HX711

Hệ thống sử dụng hai cụm đo khối lượng độc lập, trong đó mỗi cụm bao gồm một Load Cell và một module HX711. Cụm đo thứ nhất được bố trí tại khay thức ăn thứ nhất và cụm thứ hai được bố trí tại khay thức ăn thứ hai.

Load Cell hoạt động dựa trên sự thay đổi điện trở của các strain gauge khi cảm biến chịu tác động của lực. Sự thay đổi này tạo ra một tín hiệu điện áp rất nhỏ, do đó cần được khuếch đại và chuyển đổi sang dữ liệu số trước khi đưa đến ESP32-S3.

Module HX711 đảm nhiệm hai chức năng chính là khuếch đại tín hiệu từ Load Cell và chuyển đổi tín hiệu tương tự thành dữ liệu số với độ phân giải cao. Dữ liệu sau khi được HX711 chuyển đổi sẽ được ESP32-S3 đọc và quy đổi thành giá trị khối lượng thông qua hệ số hiệu chuẩn.

Kiến trúc của hai cụm cân được biểu diễn như sau:

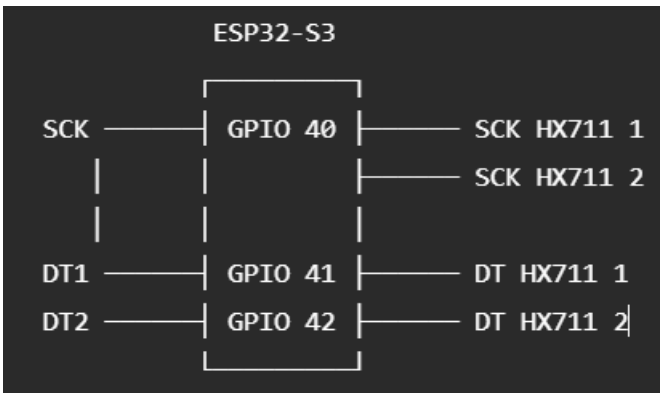
Load Cell 1 → HX711 1 → ESP32-S3

Load Cell 2 → HX711 2 → ESP32-S3

Mỗi HX711 sử dụng hai đường tín hiệu giao tiếp là **DT (Data)** và **SCK (Clock)**. Để tiết kiệm GPIO của ESP32-S3 do camera đã chiếm dụng nhiều chân, hai HX711 có thể sử dụng chung một đường SCK. Khi đó, hai chân DT vẫn được kết nối riêng biệt:

- HX711 1: SCK dùng chung, DT1 kết nối với GPIO riêng.
- HX711 2: SCK dùng chung, DT2 kết nối với GPIO riêng.

Ví dụ:



Việc dùng chung SCK giúp giảm một GPIO so với phương án sử dụng hai cặp chân hoàn toàn độc lập. Tuy nhiên, dữ liệu của hai HX711 vẫn được truyền trên hai đường DT riêng biệt. ESP32-S3 sẽ đọc từng HX711 thông qua đường DT tương ứng và chuyển đổi thành giá trị khối lượng của từng khay.

Trước khi đưa vào vận hành, mỗi Load Cell cần được thực hiện quá trình **tare và calibration** riêng để xác định điểm zero và hệ số chuyển đổi từ giá trị đọc của HX711 sang đơn vị khối lượng thực tế.

3.3.5. Nguồn cấp

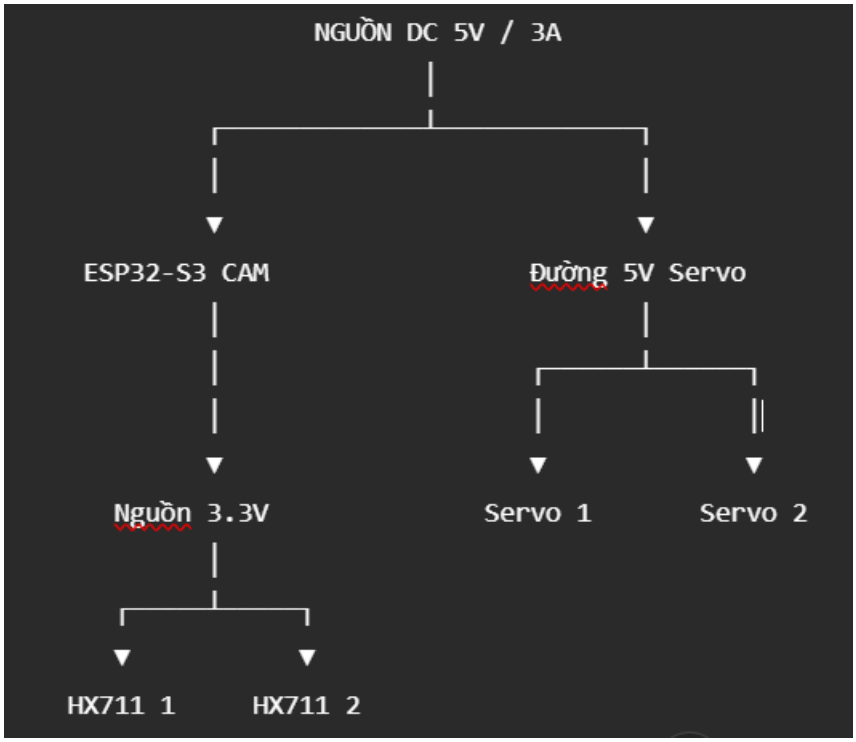
Do hệ thống sử dụng đồng thời hai Servo MG90S, hai module HX711, ESP32-S3 CAM và camera nên yêu cầu về nguồn cấp cao hơn so với cấu hình một kênh.

Nguồn 5V được phân phối thành các nhánh riêng cho cơ cấu chấp hành và mạch điều khiển.

Hai Servo MG90S được cấp nguồn từ đường 5V công suất chính. Do servo có thể tạo ra dòng tiêu thụ tăng cao trong thời điểm khởi động, đảo chiều hoặc chịu tải, cần bố trí tụ điện lọc có giá trị lớn gần khu vực cấp nguồn servo. Một tụ điện hóa khoảng **1000 µF** có thể được sử dụng song song giữa đường 5V và GND để hỗ trợ giảm sụt áp tức thời và hạn chế nhiễu do biến động dòng tải.

ESP32-S3 CAM được cấp nguồn thông qua đầu vào nguồn phù hợp của board. Board sẽ sử dụng mạch ổn áp tích hợp để tạo mức điện áp phù hợp cho vi điều khiển và các thiết bị ngoại vi. Hai module HX711 có thể được cấp nguồn ở mức 3.3V để đảm bảo mức logic tương thích với ESP32-S3.

Sơ đồ phân phối nguồn có thể được mô tả:



Tất cả các thiết bị phải sử dụng **GND chung** để tạo cùng mức tham chiếu điện áp cho các tín hiệu điều khiển và dữ liệu.

3.3.6. Sơ đồ kết nối phần cứng

Nguyên tắc thiết kế sơ đồ phần cứng cho hệ thống Smart Pet Feeder dựa trên cấu trúc hình sao (Star Topology), trong đó ESP32-S3 đóng vai trò là khối vi xử lý trung tâm, giao tiếp với các module ngoại vi và quản lý toàn bộ luồng tín hiệu. Hệ thống được chia thành 4 phân hệ kết nối chính: Phân hệ Nguồn, Phân hệ Thị giác (Camera), Phân hệ Định lượng (Load Cell) và Phân hệ Chấp hành (Servo).

a) Phân hệ Nguồn (Power Distribution) Để đảm bảo tính ổn định và tránh hiện tượng sụt áp (Brownout) gây reset vi điều khiển khi động cơ khởi động, hệ thống sử dụng nguồn tổ ong hoặc Adapter 5V - 2A (hoặc 3A). Nguồn 5V này được chia làm hai nhánh song song (nhánh công suất và nhánh điều khiển):

Nhánh 1 (Công suất): Cấp trực tiếp 5V và GND cho động cơ Servo MG90S. Tại điểm nối nguồn của Servo, một tụ hóa (Electrolytic Capacitor) có điện dung 1000µF/16V được mắc song song để lọc nhiễu và xả dòng bù áp tức thời khi động cơ bị ghì (stall).

Nhánh 2 (Điều khiển): Cấp 5V vào chân 5V_IN (hoặc VIN) của ESP32-S3. Mạch hạ áp LDO trên bo mạch ESP32 sẽ tự động chuyển đổi thành 3.3V để nuôi chip và cấp ra chân 3V3 cho các cảm biến.

Tất cả các chân GND của Nguồn, ESP32-S3, Servo và HX711 bắt buộc phải được nối chung (Common Ground) để đồng bộ mức điện áp tham chiếu (0V).

b) Chi tiết kết nối tín hiệu (Pin Mapping)

Kết nối Camera (OV2640): Cụm Camera thường được kết nối trực tiếp vào mạch ESP32-S3 thông qua cáp bẹ FPC (Flexible Printed Circuit) 24-pin. Giao diện này bao gồm bus dữ liệu 8-bit (D0-D7), các chân đồng bộ xung nhịp (XCLK, PCLK, VSYNC, HREF) và bus điều khiển SCCB (tương thích I2C qua

chân SDA, SCL). Việc kết nối vật lý này đã được tích hợp sẵn trên PCB nên không cần can thiệp dây nối ngoài.

Kết nối Cụm Cân điện tử (Load Cell & HX711):

Load Cell tới HX711: Kết nối theo 4 dây màu tiêu chuẩn của Strain Gauge:

Dây Đỏ (E+): Nguồn kích thích dương (Excitation+).

Dây Đen (E-): Nguồn kích thích âm (Excitation-).

Dây Trắng (A-): Tín hiệu vi sai âm (Signal-).

Dây Xanh lá (A+): Tín hiệu vi sai dương (Signal+).

HX711 tới ESP32-S3: Chân VCC nối với 3.3V (giúp mức logic tín hiệu đồng bộ an toàn với ESP32), chân GND nối GND. Chân dữ liệu DT và xung nhịp SCK nối với các chân GPIO tiêu chuẩn không bị trùng lặp chức năng Strapping.

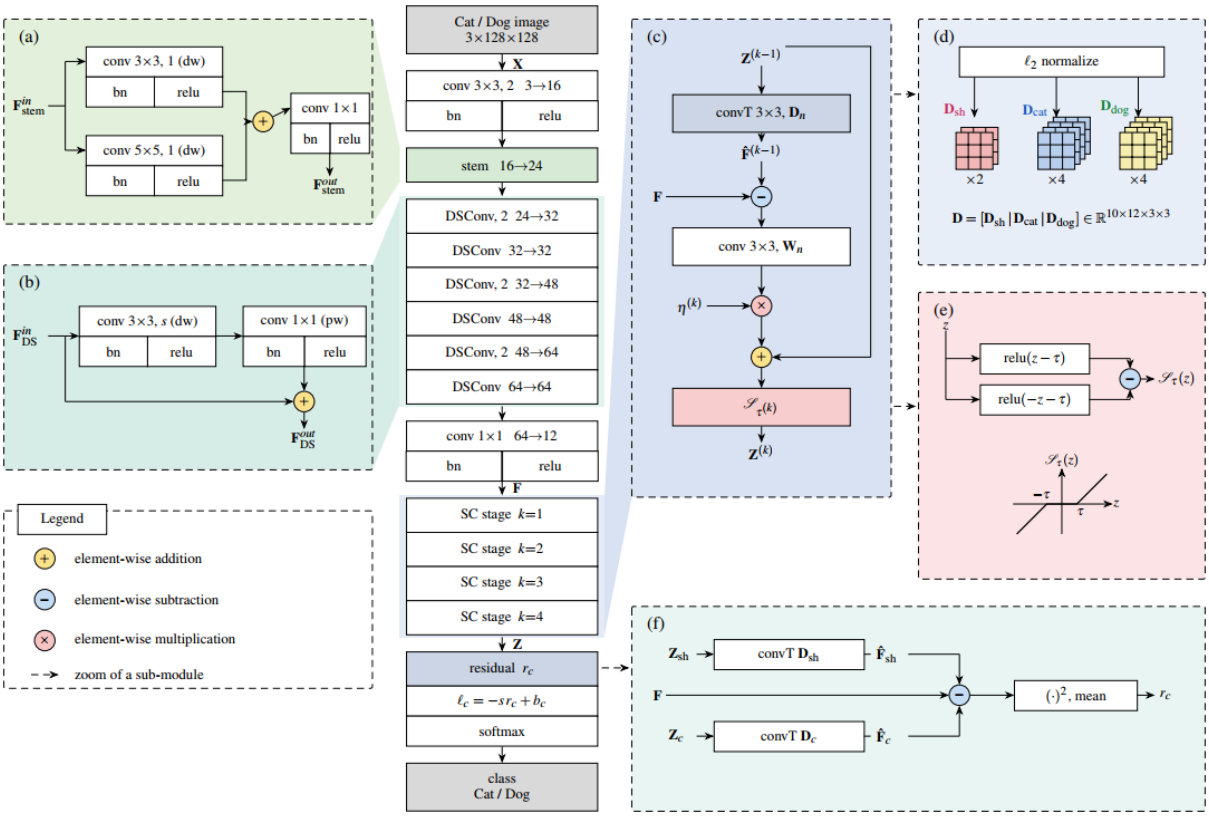
Kết nối Servo MG90S: Dây điều khiển Signal (thường có màu Cam hoặc Vàng) được nối trực tiếp vào một chân GPIO hỗ trợ phát xung PWM độc lập của ESP32-S3.

Khối linh kiện	**Thiết bị**	**Chân thiết bị**	**Kết nối trên ESP32-S3 / Nguồn**	**Ghi chú**
Nguồn cấp	Adapter 5V-3A	VCC (5V) & GND	5V_IN và GND toàn mạch	Nguồn chịu tải chính
Chấp hành 1	Servo MG90S (Trái)	VCC / GND / Signal	5V Nguồn / GND / GPIO 38	Chân PWM 1
Chấp hành 2	Servo MG90S (Phải)	VCC / GND / Signal	5V Nguồn / GND / GPIO 39	Chân PWM 2
Cảm biến 1	HX711 (Khay Trái)	VCC / GND	3V3 / GND	Nguồn logic 3.3V
		SCK (Clock)	GPIO 40	Chân Clock (Dùng chung)

		DT (Data 1)	GPIO 41	Đọc Data khay Trái
Cảm biến 2	HX711 (Khay Phải)	VCC / GND	3V3 / GND	Nguồn logic 3.3V
		SCK (Clock)	GPIO 40	Đấu chung Clock với HX711 (1)
		DT (Data 2)	GPIO 42	Đọc Data khay Phải

3.4. Thiết kế mô hình Edge AI

3.4.1. Tổng quan mô hình



3.4.1.1. Ý tưởng cốt lõi

Hầu hết các bộ phân loại ảnh kết thúc bằng *Global Average Pooling* rồi một lớp *Fully-Connected*: đặc trưng bị nén thành một vectơ, và quyết định được đưa ra bằng một siêu phẳng tuyến tính. SSR-UnrollNet-S3 thay thế phần đuôi đó bằng một cơ chế khác hẳn về bản chất:

1. Encoder nén ảnh RGB 128×128 thành một bản đồ đặc trưng rất nhỏ $F \in \mathbb{R}^{12\times8\times8}$.
2. Mô hình giả định F có thể được tái tạo tuyến tính từ một từ điển tích chập $\widehat{D}$ gồm $M = 10$ *atom*, với các hệ số Z thưa (phần lớn bằng 0).
3. Từ điển được chia thành ba nhóm: nhóm dùng chung (shared), nhóm đặc hiệu Cat, nhóm đặc hiệu Dog.
4. Việc tìm Z là một bài toán tối ưu; thay vì giải lặp đến hội tụ, mô hình trải phẳng (unroll) đúng $K = 4$ bước lặp thành 4 tầng mạng có tham số học được.
5. Quyết định phân lớp không đến từ một lớp tuyến tính mà từ sai số tái tạo: lớp nào tái tạo F tốt hơn (sau khi đã trừ đi phần nội dung dùng chung) thì lớp đó thắng.

Tên gọi phản ánh đúng ba thành phần: Shared–Specific · Residual · Unrolling · Net · S3 (ESP32-S3).

3.4.1.2. Luồng dữ liệu

```
X : 3×128×128   (ảnh RGB đã chuẩn hoá)
|
|-- Conv3x3 s2, 3->16              ==>  16x64x64
|-- Stem đa tỉ lệ: DW3 + DW5 (cộng),
|     rồi PW 16->24                ==>  24x64x64  <- đỉnh
|-- Stage 1: DSConv s2 24->32, 32->32  ==>  32x32x32
|-- Stage 2: DSConv s2 32->48, 48->48  ==>  48x16x16
|-- Stage 3: DSConv s2 48->64, 64->64  ==>  64x8x8
\-- Project: PW 64->12             ==>  F : 12x8x8
|
V   SHARED-SPECIFIC SPARSE UNROLLING HEAD
|
|-- 4 tầng LISTA:
|     R <- F - D*Z ;  Z <- S_tau(Z + eta*W.R)
|                              ==>  Z : 10x8x8 (thưa)
|-- Tái tạo dùng chung : F_s   (2 atom shared)
|-- Tái tạo riêng lớp  : F_cat, F_dog  (4 atom mỗi lớp)
|-- Sai số dư          : r_cat, r_dog
```

\-- Logits : l_c = -alpha * r_c + b_c

|

V

y_hat = argmax_c l_c thuộc {Cat, Dog}

3.4.1.3. Bảng ký hiệu chính

Ký hiệu	Kiểu / kích thước	Ý nghĩa
X	$\mathbb{R}^{3\times128\times128}$	Ảnh đầu vào đã chuẩn hoá
F	$\mathbb{R}^{C\times H\times W} = \mathbb{R}^{12\times8\times8}$	Bản đồ đặc trưng tiềm ẩn (latent feature map)
C, H, W	12, 8, 8	Số kênh và kích thước không gian của F
M	10	Tổng số atom của từ điển
S	2	Số atom dùng chung (shared_atoms)
A_{pc}	4	Số atom riêng cho mỗi lớp (apc)
N_{cls}	2	Số lớp (Cat, Dog)
k	3	Kích thước kernel của atom (ksize)
p	1	Padding, $p = \lfloor k/2 \rfloor$
K	4	Số tầng unrolling (layers)
D	$\mathbb{R}^{M\times C\times k\times k}$	Từ điển thô (tham số học được)
$\widehat{D}$	$\mathbb{R}^{M\times C\times k\times k}$	Từ điển sau chuẩn hoá theo từng atom
W	$\mathbb{R}^{M\times C\times k\times k}$	Từ điển phân tích (analysis) rời buộc, học riêng
$Z^{(k)}$	$\mathbb{R}^{M\times H\times W}$	Bản đồ hệ số thưa sau tầng k
$R^{(k)}$	$\mathbb{R}^{C\times H\times W}$	Sai số tái tạo tại tầng k
η_k	$(0, \eta_{\max})$	Bước cập nhật học được của tầng k
$\tau_{k,m}$	$\tau_{k,m} > 0$	Ngưỡng co (shrinkage) của atom m tại tầng k
$\mathcal{G}_s, \mathcal{G}_c$	tập chỉ số	Nhóm atom dùng chung / riêng lớp c
r_c	$\mathbb{R}$	Sai số dư của lớp c
α	$\alpha > 0$	Hệ số co giãn logit học được
b_c	$\mathbb{R}$	Bias của lớp c
ℓ_c	$\mathbb{R}$	Logit của lớp c
$x * \omega$	—	Tích chập tiêu chuẩn (cross-correlation như PyTorch)
$D \circledast Z$	—	Toán tử tổng hợp (transposed convolution)

3.4.1.4. Cấu hình mặc định

Tham số	Giá trị	Cờ dòng lệnh
Kích thước ảnh	128×128	--img_size 128
Bề rộng encoder	16,24,32,48,64	--widths
Kênh đặc trưng C	12	--feat_ch 12
Atom dùng chung S	2	--shared_atoms 2
Atom mỗi lớp A_{pc}	4	--apc 4
Kernel atom k	3	--ksize 3
Số tầng unrolling K	4	--layers 4
Bước tối đa $\eta_{\max}$	0.15	--step_max 0.15
Rời buộc $W \neq \widehat{D}$	Có	--untied_w
Tổng tham số	20 607	—
Tổng MACs	10 283 520	—

3.4.2. Nền tảng toán học

3.4.2.1. Bài toán biểu diễn thưa

Cho một tín hiệu $f \in \mathbb{R}^n$ và một từ điển $\mathrm{D} \in \mathbb{R}^{n \times M}$ có các cột $d_1, \ldots, d_M$ gọi là *atom*. Ta muốn viết f như một tổ hợp tuyến tính của rất ít atom:

$$f \approx \mathrm{D}z = \sum_{m=1}^{M} z_m \, d_m, \qquad \| \, z \, \|_0 \ \text{nhỏ} \qquad (2.1)$$

trong đó $\| \, z \, \|_0$ đếm số phần tử khác 0. Bài toán lý tưởng

$$\min_{z} \ 1/2 \, \| \, f - \mathrm{D}z \, \|_2^2 \quad \text{với ràng buộc} \quad \| \, z \, \|_0 \leq T \qquad (2.2)$$

là NP-khó vì $\|\cdot\|_0$ không lồi và không liên tục. Cách xử lý kinh điển là thay $\|\cdot\|_0$ bằng $\|\cdot\|_1$, thu được bài toán LASSO (còn gọi là Basis Pursuit Denoising):

$$\min_{z \in \mathbb{R}^M} J(z) \ = \ g(z) + h(z), \qquad g(z) = 1/2 \, \| \, f - \mathrm{D}z \, \|_2^2, \qquad h(z) = \lambda \, \| \, z \, \|_1 \qquad (2.3)$$

Ở đây g là phần trơn và lồi (khả vi mọi nơi), còn h là phần lồi nhưng không trơn (không khả vi tại 0). Sự phân tách này là then chốt: nó quyết định thuật toán giải ở Mục 2.3.

Vì sao ℓ_1 tạo nghiệm thưa. Quả cầu ℓ_1 $\{z: \| z \|_1 \leq t\}$ là một đa diện có các đỉnh nằm trên các trục toạ độ. Khi ta "thổi phồng" quả cầu này cho đến khi chạm vào tập nghiệm affine $\{z: Dz = f\}$, điểm chạm có xác suất cao rơi vào một đỉnh hoặc một cạnh, tức là có nhiều toạ độ bằng đúng 0. Quả cầu ℓ_2 trơn nên điểm chạm hầu như không bao giờ nằm trên trục, do đó ℓ_2 không tạo thưa. Về mặt giải tích, điều này thể hiện ở chỗ $\|\cdot\|_1$ có *điểm gãy* tại 0 với dưới vi phân là cả đoạn $[-1,1]$, đủ để triệt tiêu gradient của phần trơn trong một lân cận — đó chính là cơ chế tạo ra số 0 chính xác mà ta sẽ thấy ở (2.9).

3.4.2.2. Toán tử proximal và định lý soft-threshold

Với hàm lồi h và điểm v, toán tử proximal được định nghĩa:

$$\text{prox}_h(v) = \underset{x}{\text{argmin}} \; 1/2 \| x - v \|_2^2 + h(x) \qquad (2.4)$$

Nó trả lời câu hỏi: "điểm nào vừa gần v, vừa có h nhỏ?". Với $h = \lambda \|\cdot\|_1$ ta có kết quả then chốt sau.

Định lý 2.1 (Soft-threshold). Với mọi $\lambda > 0$,

$$\text{prox}_{\lambda\|\cdot\|_1}(v) = \mathcal{S}_\lambda(v), \qquad [\mathcal{S}_\lambda(v)]_m$$
$$= \text{sign}(v_m) \max\{|v_m| - \lambda, 0\} \qquad (2.5)$$

Chứng minh. Hàm mục tiêu trong (2.4) tách rời hoàn toàn theo từng toạ độ:

$$1/2 \| x - v \|_2^2 + \lambda \| x \|_1 = \sum_{m=1}^{M} [1/2 (x_m - v_m)^2 + \lambda |x_m|] \qquad (2.6)$$

nên ta chỉ cần giải bài toán một chiều $\min_{x \in \mathbb{R}} \phi(x) = 1/2 (x - v)^2 + \lambda|x|$. Hàm ϕ lồi chặt (vì thành phần bậc hai có hệ số $1 > 0$), nên nghiệm tồn tại duy nhất và được đặc trưng bởi điều kiện $0 \in \partial\phi(x^\star)$, tức là

$$0 \in x^\star - v + \lambda \, \partial|x^\star|, \qquad \partial|x| = \begin{cases} \{+1\}, & x > 0 \\ [-1,1], & x = 0 \\ \{-1\}, & x < 0 \end{cases} \qquad (2.7)$$

Xét ba trường hợp.

Trường hợp 1: $x^\star > 0$. Khi đó $\partial|x^\star| = \{1\}$ nên $x^\star = v - \lambda$. Điều kiện $x^\star > 0$ tương đương $v > \lambda$.

Trường hợp 2: $x^\star < 0$. Khi đó $\partial|x^\star| = \{-1\}$ nên $x^\star = v + \lambda$. Điều kiện $x^\star < 0$ tương đương $v < -\lambda$.

Trường hợp 3: $x^\star = 0$. Điều kiện (2.7) trở thành $0 \in -v + \lambda[-1,1]$, tức tồn tại $s \in [-1,1]$ sao cho $v = \lambda s$, tương đương $|v| \le \lambda$.

Ba trường hợp phủ kín và rời nhau trên $\mathbb{R}$, cho nghiệm

$$x^\star = \begin{cases} v - \lambda, & v > \lambda \\ 0, & |v| \le \lambda \\ v + \lambda, & v < -\lambda \end{cases} \qquad (2.8)$$

và biểu thức này chính là $\mathrm{sign}(v)\max\{|v| - \lambda, 0\}$. ■

Soft-threshold làm hai việc cùng lúc: (i) *giết* mọi hệ số có biên độ nhỏ hơn ngưỡng — đây là nguồn gốc duy nhất của tính thưa; (ii) *co* các hệ số còn lại về gốc đúng một lượng λ, tức là phạt đều mọi hệ số sống sót. Khác với ReLU (chỉ giữ phần dương), soft-threshold đối xứng: hệ số âm có biên độ lớn vẫn được giữ lại với dấu âm. Điều này quan trọng vì atom và hệ số âm biểu diễn "trừ đi một mẫu hình", một khả năng biểu đạt mà ReLU không có.

3.4.2.3. Thuật toán ISTA suy diễn từ nguyên lý majorization–minimization

Ta không thể giải (2.3) bằng gradient descent thuần tuý vì h không khả vi. Ý tưởng của ISTA (Iterative Shrinkage-Thresholding Algorithm) là tại mỗi bước, thay g bằng một *hàm chặn trên bậc hai đơn giản* rồi cực tiểu hoá hàm chặn đó.

Bước 1 — chặn trên phần trơn. Hàm $g(z) = 1/2 \parallel f - \mathrm{D}z \parallel^2$ có gradient

$$\nabla g(z) = -\mathrm{D}^\top(f - \mathrm{D}z) \qquad (2.9)$$

và gradient này Lipschitz với hằng số $L = \parallel \mathrm{D}^\top\mathrm{D} \parallel_2 = \sigma_{\max}(\mathrm{D})^2$. Với mọi $L' \ge L$ ta có bất đẳng thức chặn trên chuẩn:

$$g(z) \le g\left(z^{(k)}\right) + \langle \nabla g\left(z^{(k)}\right), z - z^{(k)} \rangle + \frac{L'}{2} \parallel z - z^{(k)} \parallel_2^2 \qquad (2.10)$$

với đẳng thức xảy ra tại $z = z^{(k)}$. Vế phải là hàm chặn trên (majorizer) và là một parabol đơn giản.

Bước 2 — cực tiểu hoá hàm chặn trên. Đặt $\eta = 1/L'$. Cực tiểu hoá vế phải của (2.10) cộng với $h(z)$:

$$z^{(k+1)} = \underset{z}{\mathrm{argmin}} \left\{ \langle \nabla g\left(z^{(k)}\right), z - z^{(k)} \rangle + \frac{1}{2\eta} \parallel z - z^{(k)} \parallel_2^2 + \lambda \parallel z \parallel_1 \right\} \qquad (2.11)$$

Gom bình phương (hoàn chỉnh bình phương) đối với z:

$$\langle \nabla g, z - z^{(k)} \rangle + \frac{1}{2\eta} \| z - z^{(k)} \|^2 = \frac{1}{2\eta}$$

$$\| z - \left(z^{(k)} - \eta \nabla g\left(z^{(k)} \right) \right) \|^2 + \text{const} \qquad (2.12)$$

Do đó (2.11) trở thành đúng định nghĩa proximal (2.4) với điểm $v = z^{(k)} - \eta \nabla g\left(z^{(k)} \right)$ và tham số $\eta\lambda$. Áp dụng Định lý 2.1:

$$\boxed{z^{(k+1)} = \mathcal{S}_{\eta\lambda}\left(z^{(k)} + \eta\, \mathrm{D}^{\top}\left(f - \mathrm{D}z^{(k)} \right) \right)} \qquad (2.13)$$

Đây là một bước ISTA, gồm đúng hai thao tác: một bước gradient trên phần trơn, rồi một phép soft-threshold. Cấu trúc ba nhịp này — *tính sai số → phản chiếu ngược qua* $\mathrm{D}^{\top}$ *→ co ngưỡng* — sẽ được lặp lại nguyên vẹn trong Mục 4.

Ghi chú về hội tụ. Nếu $0 < \eta \leq 1/L$ thì dãy $\{J\left(z^{(k)} \right)\}$ giảm đơn điệu và $J\left(z^{(k)} \right) - J(z^{\star}) = O(1/k)$. Điều kiện $\eta \leq 1/L$ giải thích vì sao trong mô hình, bước η_k bị chặn trên bởi $\eta_{\max} = 0.15$ thay vì để tự do (xem (4.12)): đó là một ràng buộc ổn định số kế thừa trực tiếp từ lý thuyết.

3.4.2.4. Từ ISTA sang LISTA

ISTA cần hàng trăm đến hàng nghìn vòng lặp để hội tụ — không thể chấp nhận trên vi điều khiển. LISTA (Gregor & LeCun, 2010) đưa ra ba thay đổi:

1. Cắt cụt cố định. Chỉ chạy đúng K vòng lặp ($K = 4$ ở đây). Mỗi vòng lặp trở thành một *tầng* của mạng nơ-ron.
2. Rời buộc tham số (untying). Ma trận phân tích $\mathrm{D}^{\top}$ trong (2.13) được thay bằng một ma trận học được W độc lập với D. Khi đó tầng k có dạng

$$z^{(k+1)} = \mathcal{S}_{\tau_k}\left(z^{(k)} + \eta_k\, \mathrm{W}^{\top}\left(f - \mathrm{D}z^{(k)} \right) \right) \qquad (2.14)$$

3. Học các siêu tham số. η_k và τ_k không còn được suy ra từ L và λ mà được học bằng lan truyền ngược, và được phép khác nhau giữa các tầng.

Lý do lý thuyết cho việc rời buộc: với số vòng lặp hữu hạn, mục tiêu không còn là "giải đúng LASSO" mà là "đưa ra một Z đủ tốt cho tác vụ phía sau trong đúng K bước". Có thể chứng minh rằng với W được chọn khéo, sai số $\| z^{(K)} - z^{\star} \|$ giảm theo hàm mũ theo K thay vì $O(1/K)$. Trong thực hành, LISTA đạt chất lượng của hàng trăm vòng ISTA chỉ với vài tầng.

Điểm mấu chốt cho tài liệu này: một mạng LISTA *không phải* là một mạng nơ-ron được đặt tên mỹ miều. Nó là một thuật toán tối ưu đã biết, được viết lại dưới dạng đồ thị tính toán, trong đó các hằng số thuật toán được thay bằng tham số học được. Mọi thành phần đều có ý nghĩa toán học tiền định, và đây chính là cơ sở cho tính giải thích được (interpretability) của mô hình.

3.4.2.5. Mở rộng sang miền tích chập

Ở trên, f là một vectơ và D là ma trận. Với ảnh, ta không muốn làm phẳng $F \in \mathbb{R}^{C\times H\times W}$ thành vectơ CHW chiều vì (i) ma trận D sẽ khổng lồ và (ii) ta mất tính bất biến tịnh tiến. Thay vào đó ta dùng mô hình biểu diễn thưa tích chập (convolutional sparse coding):

$$F \approx \sum_{m=1}^{M} \widehat{D}_m \circledast Z_m \qquad (2.15)$$

trong đó $\widehat{D}_m \in \mathbb{R}^{C\times k\times k}$ là atom thứ m (một bộ lọc nhỏ), $Z_m \in \mathbb{R}^{H\times W}$ là *bản đồ hệ số* của atom đó, và $\circledast$ là phép tổng hợp (đặt atom vào mọi vị trí không gian, nhân với hệ số, rồi cộng chồng lên nhau).

Điều này tương đương với mô hình ma trận (2.1) trong đó D là ma trận *Toeplitz khối* chứa mọi phép tịnh tiến của mọi atom, nhưng không bao giờ được xây dựng tường minh. Mọi kết quả ở Mục 2.2–2.3 giữ nguyên vì chúng chỉ dựa trên tính lồi và trên quan hệ liên hợp (adjoint) giữa toán tử phân tích và tổng hợp — quan hệ này sẽ được chứng minh tường minh ở Bổ đề 4.1.

3.4.3. Encoder hướng phần cứng: từng lớp, từng công thức

Vai trò của encoder là *nén ảnh* $3 \times 128 \times 128$ *(49 152 số) xuống* $12 \times 8 \times 8$ *(768 số)*, tức tỉ lệ nén 64:1, trong khi vẫn giữ đủ thông tin để phân biệt Cat/Dog. Mọi lớp đều chỉ dùng năm phép toán: Conv2d, Conv2d depthwise, Conv2d pointwise, cộng phần tử, và ReLU — đây là tập toán tử mà ESP-DL hỗ trợ lượng tử hoá tốt nhất.

3.4.3.1. Tích chập hai chiều

Với đầu vào $x \in \mathbb{R}^{C_{\text{in}}\times H_{\text{in}}\times W_{\text{in}}}$, kernel $\omega \in \mathbb{R}^{C_{\text{out}}\times (C_{\text{in}}/g)\times k\times k}$, stride s, padding p, số nhóm g, đầu ra tại kênh o và vị trí (i, j) là

$$y_{o,i,j} = \sum_{c=1}^{C_{\text{in}}/g} \sum_{u=0}^{k-1} \sum_{v=0}^{k-1} \omega_{o,c,u,v} \cdot x_{\gamma(o)+c,\ si+u-p,\ sj+v-p} \qquad (3.1)$$

trong đó $\gamma(o) = \lfloor o\, g/C_{\text{out}} \rfloor \cdot (C_{\text{in}}/g)$ là chỉ số kênh đầu vào bắt đầu của nhóm chứa kênh o, và các phần tử có chỉ số nằm ngoài biên được gán 0 (zero padding). Lưu ý PyTorch thực hiện *cross-correlation* chứ không lật kernel; điều này không ảnh hưởng đến khả năng biểu đạt vì kernel là tham số học được, nhưng có ảnh hưởng đến phép biến đổi ở Mục 8.2 nên cần ghi nhớ.

Kích thước đầu ra:

$$H_{\text{out}} = \lfloor (H_{\text{in}} + 2p - k)/s \rfloor + 1 \qquad (3.2)$$

Trong toàn bộ encoder ta luôn dùng $p = \lfloor k/2 \rfloor$ với k lẻ, nên (3.2) rút gọn thành $H_{\text{out}} = \lceil H_{\text{in}}/s \rceil$, tức là stride 1 giữ nguyên kích thước và stride 2 giảm đúng một nửa.

Hai trường hợp đặc biệt được dùng dày đặc:

- Depthwise ($g = C_{\text{in}} = C_{\text{out}}$): mỗi kênh có bộ lọc riêng, không trộn kênh.

$$y_{c,i,j} = \sum_{u,v} \omega_{c,0,u,v}\ x_{c,\, si+u-p,\, sj+v-p} \qquad (3.3)$$

- Pointwise ($k = 1, g = 1$): trộn kênh tuyến tính tại từng vị trí, không nhìn lân cận.

$$y_{o,i,j} = \sum_{c=1}^{C_{\text{in}}} \omega_{o,c}\ x_{c,i,j} \qquad (3.4)$$

3.4.3.2. Batch Normalization

Mọi lớp tích chập trong encoder đều không có bias (bias=False) và được theo sau bởi BatchNorm, vì bias sẽ bị BN hấp thụ hoàn toàn. Trong pha huấn luyện, với một mini-batch $\mathcal{B}$ và kênh c:

$$\mu_c = \frac{1}{|\mathcal{B}|HW} \sum_{n\in\mathcal{B}} \sum_{i,j} x_{n,c,i,j}\,, \qquad \sigma_c^2 = \frac{1}{|\mathcal{B}|HW} \sum_{n\in\mathcal{B}} \sum_{i,j} (x_{n,c,i,j} - \mu_c)^2 \qquad (3.5)$$

$$\text{BN}(x)_{n,c,i,j} = \gamma_c \cdot \frac{x_{n,c,i,j} - \mu_c}{\sqrt{\sigma_c^2 + \varepsilon}} + \beta_c, \qquad \varepsilon = 10^{-5} \qquad (3.6)$$

Song song đó, BN duy trì trung bình động $\bar{\mu}_c, \bar{\sigma}_c^2$ bằng cập nhật $\bar{\mu}_c \leftarrow (1-\rho)\bar{\mu}_c + \rho\mu_c$ với $\rho = 0.1$. Ở pha suy luận, μ_c, σ_c^2 trong (3.6) được thay bằng $\bar{\mu}_c, \bar{\sigma}_c^2$, khiến BN trở thành một phép affine tất định theo kênh — chính tính chất này cho phép gộp BN vào tích chập khi triển khai (Mục 8.1).

Chỉ có γ_c, β_c là tham số học được, tức $2C_{\text{out}}$ tham số mỗi lớp BN. $\bar{\mu}, \bar{\sigma}^2$ là *buffer*, không nằm trong parameters() nhưng vẫn chiếm Flash khi triển khai.

3.4.3.3. Hàm kích hoạt

Encoder chỉ dùng ReLU:

$$\text{ReLU}(z) = \max(z, 0) \qquad (3.7)$$

Mô hình giáo viên (teacher) dùng SiLU $z\,\sigma(z)$ và khối Squeeze-and-Excitation; cả hai đều bị loại bỏ khỏi student. Lý do không phải là thẩm mỹ:

- SiLU cần hàm mũ, tốn chu kỳ và khó lượng tử hoá INT8 chính xác trên MCU không có FPU chuyên dụng cho hàm siêu việt.
- SE yêu cầu global pooling giữa mạng, tạo điểm đồng bộ hoá toàn cục làm gãy luồng xử lý theo khối (tiling) mà runtime nhúng dựa vào để giữ RAM thấp.
- ReLU là ánh xạ tuyến tính từng khúc và thuần nhất dương ($\text{ReLU}(ax) = a\,\text{ReLU}(x)$ với $a > 0$), nên nó giao hoán tốt với phép co giãn lượng tử hoá.

3.4.3.4. Tích chập tách chiều sâu và lý do tiết kiệm

Một tích chập thường $C_{\text{in}} \to C_{\text{out}}$ với kernel $k \times k$ được thay bằng chuỗi *depthwise rồi pointwise*.

Mệnh đề 3.1. Tỉ lệ chi phí giữa DSConv và tích chập thường (cả về tham số lẫn MACs, bỏ qua BN) là

$$\rho = \frac{C_{\text{in}}k^2 + C_{\text{in}}C_{\text{out}}}{C_{\text{in}}C_{\text{out}}k^2} = \frac{1}{C_{\text{out}}} + \frac{1}{k^2} \qquad (3.8)$$

Chứng minh. Tích chập thường có $C_{\text{out}}C_{\text{in}}k^2$ trọng số và, với đầu ra $H \times W$, tốn $C_{\text{out}}HW\,C_{\text{in}}k^2$ MACs. Nhánh depthwise có $C_{\text{in}}k^2$ trọng số và $C_{\text{in}}HWk^2$ MACs (theo (3.3), mỗi kênh độc lập). Nhánh pointwise có $C_{\text{out}}C_{\text{in}}$ trọng số và $C_{\text{out}}HWC_{\text{in}}$ MACs (theo (3.4)). Lấy tỉ số và rút gọn HW cho kết quả (3.8).

Với $k = 3$, $C_{\text{out}} = 64$: $\rho = 1/64 + 1/9 \approx 0.127$, tức tiết kiệm khoảng 7.9 lần. Đây là lý do toàn bộ thân encoder dùng DSConv.

Khối S3DSConvBlock được định nghĩa:

$$\text{DS}(x) = \text{PW}\big(\text{DW}(x)\big), \qquad \text{DW} = \text{ReLU} \circ \text{BN} \circ \text{Conv}^{3\times3}_{\text{depthwise}}, \quad \text{PW} = \text{ReLU} \circ \text{BN} \circ \text{Conv}^{1\times1} \qquad (3.9)$$

và có nhánh tắt (residual) chỉ khi kích thước khớp:

$$y = \begin{cases} x + \mathrm{DS}(x), & s = 1 \text{ và } C_{\text{in}} = C_{\text{out}} \\ \mathrm{DS}(x), & \text{ngược lại} \end{cases} \qquad (3.10)$$

Nhánh tắt giúp gradient chảy ngược không suy giảm: $\partial y / \partial x = I + \partial \mathrm{DS}(x) / \partial x$, nên ngay cả khi Jacobian của khối tiến về 0 thì gradient vẫn truyền được qua thành phần đơn vị.

3.4.3.5. Stem đa tỉ lệ dạng cộng

Lớp thứ hai của encoder nhìn cùng một đầu vào bằng hai cửa sổ khác nhau rồi cộng kết quả:

$$\mathrm{Stem}(x) = \mathrm{PW}_{16\to 24}\left(\mathrm{DW}^{3\times 3}(x) \; + \; \mathrm{DW}^{5\times 5}(x)\right) \qquad (3.11)$$

trong đó nhánh $\mathrm{DW}^{3\times 3}$ cho trường nhìn hẹp (bắt chi tiết cục bộ) và nhánh $\mathrm{DW}^{5\times 5}$ cho trường nhìn rộng (bắt ngữ cảnh xung quanh).

So sánh với phương án nối kênh (concat) mà mô hình teacher dùng: concat tạo tensor trung gian $32 \times 64 \times 64 = 131\,072$ phần tử, trong khi cộng giữ nguyên $16 \times 64 \times 64 = 65\,536$. Trên ESP32-S3 nơi đỉnh activation quyết định khả năng nạp mô hình, giảm một nửa tensor trung gian quan trọng hơn khả năng biểu đạt bị mất. Phép cộng vẫn giữ được thông tin đa tỉ lệ vì hai nhánh có kernel khác nhau nên học các mẫu hình bổ sung cho nhau; điều mất đi là khả năng cho lớp PW phía sau *chọn lọc* giữa hai tỉ lệ bằng trọng số riêng.

Trường nhìn hiệu dụng của tổng hai nhánh là hợp của hai trường nhìn, tức 5×5 trên lưới sau khi đã giảm mẫu một lần, tương đương 10×10 pixel ảnh gốc.

3.4.3.6. Lớp chiếu và đặc trưng tiềm ẩn

Lớp cuối là pointwise $64 \to 12$, ép số kênh xuống rất thấp:

$$F_{c,i,j} = \mathrm{ReLU}\left(\mathrm{BN}\left(\sum_{c\prime=1}^{64} \omega_{c,c\prime}\, x_{c\prime,i,j}\right)\right), \qquad F \in \mathbb{R}^{12\times 8\times 8} \qquad (3.12)$$

Con số $C = 12$ không tuỳ tiện. Nó bị ràng buộc từ hai phía:

- Chặn dưới: từ điển cần $M = 10$ atom trực giao trong không gian phẳng $\mathbb{R}^{Ck^2}$, nên cần $Ck^2 = 12 \cdot 9 = 108 \geq M = 10$ — thoả mãn rất thoải mái.
- Chặn trên: chi phí của head tỉ lệ thuận với C (xem (9.4)), và F là tensor phải giữ nguyên trong RAM suốt K tầng unrolling.

Đáng lưu ý là ReLU ở lớp chiếu khiến $F \geq 0$. Trong khi đó hệ số Z được phép mang dấu âm (soft-threshold đối xứng). Vì vậy các atom phải học cách tổ hợp cả cộng lẫn trừ để tái tạo một tín hiệu không âm — một ràng buộc mà mô hình xử lý được nhưng cần ý thức khi phân tích atom đã học.

3.4.3.7. Bảng chi tiết từng lớp của encoder

Ký hiệu: k = kernel, s = stride, g = số nhóm. Cột *Params* gồm cả trọng số tích chập và $2C_{\text{out}}$ tham số BN. Cột *MACs* tính theo $C_{\text{out}} \cdot H_{\text{out}} W_{\text{out}} \cdot (C_{\text{in}}/g) \cdot k^2$.

#	Lớp	$k/s/g$	Vào → Ra	Params	MACs
1	pre Conv	3/2/1	$3 \times 128 \times 128 \rightarrow 16 \times 64 \times 64$	464	1 769 472
2	stem.b3 DW	3/1/16	$16 \times 64 \times 64 \rightarrow 16 \times 64 \times 64$	176	589 824
3	stem.b5 DW	5/1/16	$16 \times 64 \times 64 \rightarrow 16 \times 64 \times 64$	432	1 638 400
4	stem.fuse PW	1/1/1	$16 \times 64 \times 64 \rightarrow 24 \times 64 \times 64$	432	1 572 864
5	stage1.0 DW	3/2/24	$24 \times 64 \times 64 \rightarrow 24 \times 32 \times 32$	264	221 184
6	stage1.0 PW	1/1/1	$24 \times 32 \times 32 \rightarrow 32 \times 32 \times 32$	832	786 432
7	stage1.1 DW	3/1/32	$32 \times 32 \times 32 \rightarrow 32 \times 32 \times 32$	352	294 912
8	stage1.1 PW	1/1/1	$32 \times 32 \times 32 \rightarrow 32 \times 32 \times 32$	1 088	1 048 576
9	stage2.0 DW	3/2/32	$32 \times 32 \times 32 \rightarrow 32 \times 16 \times 16$	352	73 728
10	stage2.0 PW	1/1/1	$32 \times 16 \times 16 \rightarrow 48 \times 16 \times 16$	1 632	393 216
11	stage2.1 DW	3/1/48	$48 \times 16 \times 16 \rightarrow 48 \times 16 \times 16$	528	110 592
12	stage2.1 PW	1/1/1	$48 \times 16 \times 16 \rightarrow 48 \times 16 \times 16$	2 400	589 824
13	stage3.0 DW	3/2/48	$48 \times 16 \times 16 \rightarrow 48 \times 8 \times 8$	528	27 648
14	stage3.0 PW	1/1/1	$48 \times 8 \times 8 \rightarrow 64 \times 8 \times 8$	3 200	196 608
15	stage3.1 DW	3/1/64	$64 \times 8 \times 8 \rightarrow 64 \times 8 \times 8$	704	36 864
16	stage3.1 PW	1/1/1	$64 \times 8 \times 8 \rightarrow 64 \times 8 \times 8$	4 224	262 144
17	project PW	1/1/1	$64 \times 8 \times 8 \rightarrow 12 \times 8 \times 8$	792	49 152
	Tổng encoder			18 400	9 661 440

Hai khối có nhánh tắt theo (3.10): khối stage1.1 ($32 \rightarrow 32$, $s = 1$) và stage2.1 ($48 \rightarrow 48$), stage3.1 ($64 \rightarrow 64$). Các khối có $s = 2$ hoặc đổi số kênh không có nhánh tắt.

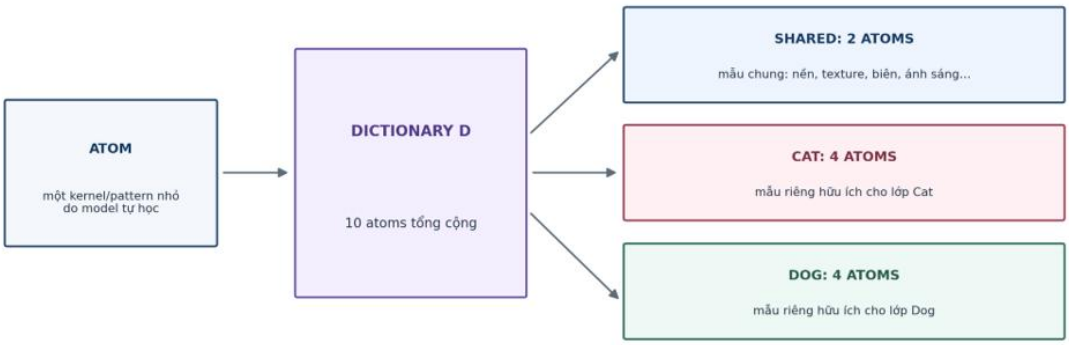
Đỉnh activation. Tensor lớn nhất là đầu ra lớp 4 với $24 \times 64 \times 64 = 98\,304$ phần tử. Ở INT8 riêng tensor này chiếm 98 304 byte, tức 96 KiB. Đây là ràng buộc

cứng nhất của toàn bộ thiết kế: mọi ý tưởng làm rộng lớp đầu đều va vào con số này trước khi va vào giới hạn Flash.

Vị trí	Hình dạng	Số phần tử	INT8 (KiB)
Ảnh vào	$3 \times 128 \times 128$	49 152	48.0
Sau pre	$16 \times 64 \times 64$	65 536	64.0
Sau stem.fuse	$24 \times 64 \times 64$	98 304	96.0
Sau stage1	$32 \times 32 \times 32$	32 768	32.0
Sau stage2	$48 \times 16 \times 16$	12 288	12.0
Sau stage3	$64 \times 8 \times 8$	4 096	4.0
F	$12 \times 8 \times 8$	768	0.75
Z	$10 \times 8 \times 8$	640	0.63

Bảng trên cho thấy một đặc điểm quan trọng: 93.9 % MACs và 89.3 % tham số nằm ở encoder, trong khi toàn bộ phần "thú vị" về mặt phương pháp (head SSR) chỉ tốn 6.1 % MACs. Mô hình vì vậy rẻ đúng ở chỗ cần rẻ.

3.4.4. Shared–Specific Sparse Unrolling Head



3.4.4.1. Tổ chức từ điển thành ba nhóm

Từ điển gồm M atom, mỗi atom là một kernel $\mathbb{R}^{C\times k\times k}$:

$$M = S + N_{\text{cls}} \cdot A_{\text{pc}} = 2 + 2 \cdot 4 = 10 \qquad (4.1)$$

Các atom được xếp liên tiếp trong một tensor duy nhất $D \in \mathbb{R}^{M\times C\times k\times k}$ và được chia nhóm theo chỉ số:

$$\mathcal{G}_s = \{1, \dots, S\} = \{1,2\}, \qquad \mathcal{G}_c = \{S + (c-1)A_{\text{pc}} + 1, \ \dots, \ S + c\,A_{\text{pc}}\} \qquad (4.2)$$

nên $\mathcal{G}_{\text{cat}} = \{3,4,5,6\}$ và $\mathcal{G}_{\text{dog}} = \{7,8,9,10\}$. Viết theo khối:

$$D = [D_s \quad D_{\text{cat}} \quad D_{\text{dog}}] \qquad (4.3)$$

Giả thuyết mô hình. Đặc trưng của một con mèo và một con chó chia sẻ rất nhiều cấu trúc: nền, ánh sáng, kết cấu lông, đường biên, hình học phổ quát của động vật bốn chân. Chỉ một phần nhỏ của F mới thực sự mang thông tin phân biệt. Nếu để một từ điển duy nhất cho mỗi lớp, cả hai từ điển sẽ phải học lại phần chung, làm lãng phí atom và làm hai sai số $r_{\text{cat}}, r_{\text{dog}}$ gần như bằng nhau. Việc tách D_s ra buộc phần chung được biểu diễn *một lần*, để phần riêng cạnh tranh trên đúng thông tin còn lại.

3.4.4.2. Chuẩn hoá atom

Mô hình (2.15) có bất định về thang đo: thay $\widehat{D}_m \to a\widehat{D}_m$ và $Z_m \to Z_m/a$ cho cùng một tái tạo nhưng làm thay đổi giá trị $\| Z \|_1$, khiến ngưỡng τ mất ý nghĩa. Để khử bất định, mọi atom được chuẩn hoá về chuẩn Frobenius đơn vị trước mỗi lần dùng:

$$\widehat{D}_m = \frac{D_m}{\max(\| D_m \|_F,\ 10^{-8})}, \qquad \| D_m \|_F = \sqrt{\sum_{c=1}^{C} \sum_{u,v} D_{m,c,u,v}^2} \tag{4.4}$$

Tương tự cho từ điển phân tích: $\widehat{W}_m = W_m / \| W_m \|_F$. Ngưỡng dưới 10^{-8} tránh chia cho 0 nếu một atom bị thoái hoá.

Lưu ý quan trọng: phép chuẩn hoá này nằm trong đồ thị tính toán, nên gradient chảy qua nó. Cụ thể, với $n = \| D_m \|_F$,

$$\frac{\partial \widehat{D}_m}{\partial D_m} = \frac{1}{n}\left(I - \widehat{D}_m \widehat{D}_m^{\top}\right) \tag{4.5}$$

(viết ở dạng phẳng), tức gradient bị chiếu lên không gian trực giao với chính atom — thành phần gradient làm thay đổi *độ dài* atom bị triệt tiêu, chỉ còn thành phần làm xoay hướng atom. Đây là hành vi đúng: hướng của atom là thứ mang thông tin, độ dài thì không.

3.4.4.3. Khởi tạo trực giao

Tại thời điểm khởi tạo, ta muốn các atom càng ít giống nhau càng tốt. Thuật toán:

1. Lấy ma trận ngẫu nhiên $A \in \mathbb{R}^{d \times M}$ với $d = Ck^2 = 12 \cdot 9 = 108$, các phần tử i.i.d. $\mathcal{N}(0,1)$, seed cố định 12345.
2. Phân rã QR rút gọn: $A = QR$ với $Q \in \mathbb{R}^{d \times M}$ có các cột trực chuẩn.
3. Đặt $D^{(0)} = Q^{\top}$ rồi định dạng lại thành $M \times C \times k \times k$.

Kết quả là $D^{(0)}\left(D^{(0)}\right)^{\top} = I_M$, tức hệ số tương quan giữa mọi cặp atom bằng 0. Điều kiện khả thi là $M \leq d$, ở đây $10 \leq 108$, thoả mãn rất rộng rãi.

Vì sao điều này quan trọng: đại lượng quyết định chất lượng khôi phục thưa là độ kết dính (mutual coherence)

$$\mu\left(\widehat{D}\right) = \max_{m \neq m'} |\langle \widehat{D}_m, \widehat{D}_{m'} \rangle| \qquad (4.6)$$

Lý thuyết cho biết ISTA/LASSO khôi phục đúng giá đỡ (support) khi $\| z \|_0 < 1/2\,(1 + 1/\mu)$. Khởi tạo trực giao cho $\mu = 0$, tức điều kiện ban đầu tốt nhất có thể; hàm mất mát $\mathcal{L}_{\text{sep}}$ ở (5.14) có nhiệm vụ giữ μ nhỏ trong suốt quá trình huấn luyện.

3.4.4.4. Hai toán tử: phân tích và tổng hợp

Toán tử phân tích $\mathcal{A}: \mathbb{R}^{C \times H \times W} \to \mathbb{R}^{M \times H \times W}$, cài đặt bằng F.conv2d(x, W, padding=p):

$$[\mathcal{A}(x)]_{m,i,j} = \sum_{c=1}^{C} \sum_{u=0}^{k-1} \sum_{v=0}^{k-1} \widehat{W}_{m,c,u,v} \; x_{c,\, i+u-p,\, j+v-p} \qquad (4.7)$$

Nó trả lời: *"tại vị trí (i, j), tín hiệu khớp với atom m đến mức nào?"*

Toán tử tổng hợp $\mathcal{A}^{\top}: \mathbb{R}^{M \times H \times W} \to \mathbb{R}^{C \times H \times W}$, cài đặt bằng F.conv_transpose2d(Z, D, padding=p):

$$[\mathcal{A}^{\top}(Z)]_{c,i,j} = \sum_{m=1}^{M} \sum_{u=0}^{k-1} \sum_{v=0}^{k-1} \widehat{D}_{m,c,u,v} \; Z_{m,\, i-u+p,\, j-v+p} \qquad (4.8)$$

Nó trả lời: *"nếu đặt atom m với biên độ $Z_{m,i,j}$ tại mọi vị trí rồi cộng chồng, ta được tín hiệu gì?"* Chú ý dấu: (4.7) có $+u$, (4.8) có $-u$. Đó chính là điều khiến hai toán tử liên hợp với nhau.

Bổ đề 4.1 (Quan hệ liên hợp). Với cùng một tensor trọng số $\omega \in \mathbb{R}^{M \times C \times k \times k}$ và cùng padding p, hai toán tử conv2d và conv_transpose2d là liên hợp của nhau theo tích vô hướng Euclid:

$$\langle \text{conv2d}(x, \omega),\, Z \rangle = \langle x,\, \text{conv_transpose2d}(Z, \omega) \rangle \quad \forall x, Z \qquad (4.9)$$

Chứng minh. Khai triển vế trái theo (4.7):

$$\text{VT} = \sum_{m,i,j} Z_{m,i,j} \sum_{c,u,v} \omega_{m,c,u,v} \; x_{c,\, i+u-p,\, j+v-p} \qquad (4.10)$$

Đổi biến $i' = i + u - p$, $j' = j + v - p$, tức $i = i' - u + p$, $j = j' - v + p$. Vì tổng lấy trên toàn bộ miền (với quy ước 0 ngoài biên), phép đổi biến là song ánh và ta được

$$\mathrm{VT} = \sum_{c,i',j'} x_{c,i',j'} \ \Psi_{c,i',j'}, \qquad \Psi_{c,i',j'} := \sum_{m,u,v} \omega_{m,c,u,v} \ Z_{m,\, i'-u+p,\, j'-v+p} \tag{4.11}$$

So sánh định nghĩa của Ψ với (4.8), ta thấy Ψ đúng bằng conv_transpose2d(Z, ω, padding=p). Do đó $\mathrm{VT} = \langle x,\ \mathrm{conv_transpose2d}(Z, \omega)\rangle = \mathrm{VP}$. ■

Hệ quả. Khi $\widehat{W} = \widehat{D}$, bước cập nhật trong (2.13) đúng là bước gradient của hàm $1/2 \parallel F - \mathcal{A}^{\top}(Z) \parallel^2$ theo Z, vì $\nabla_Z = -\mathcal{A}\big(F - \mathcal{A}^{\top}(Z)\big)$. Khi $\widehat{W} \neq \widehat{D}$ (chế độ --untied_w mặc định), đây không còn là gradient chính xác mà là một *hướng đi xuống đã được học*, đúng tinh thần LISTA ở (2.14).

3.4.4.5. Một tầng unrolling: bốn bước

Khởi tạo $Z^{(0)} = 0$ (đây là lựa chọn quan trọng: nó đảm bảo tầng 1 chỉ nhìn thấy F, không mang thiên kiến nào). Với $k = 0,1, \dots, K-1$:

Bước 1 — tái tạo hiện tại.

$$\hat{F}^{(k)} = \mathcal{A}^{\top}\big(Z^{(k)}\big), \qquad \hat{F}^{(k)}_{c,i,j} = \sum_{m=1}^{M} \sum_{u,v} \widehat{D}_{m,c,u,v} \ Z^{(k)}_{m,\, i-u+p,\, j-v+p} \tag{4.12}$$

Bước 2 — sai số tái tạo.

$$R^{(k)} = F - \hat{F}^{(k)} \ \in \mathbb{R}^{C \times H \times W} \tag{4.13}$$

Bước 3 — bước gradient (phản chiếu sai số qua từ điển phân tích).

$$\tilde{Z}^{(k+1)}_{m,i,j} = Z^{(k)}_{m,i,j} + \eta_k \sum_{c=1}^{C} \sum_{u,v} \widehat{W}_{m,c,u,v} \ R^{(k)}_{c,\, i+u-p,\, j+v-p} \tag{4.14}$$

Bước 4 — co ngưỡng theo từng atom.

$$Z^{(k+1)}_{m,i,j} = \mathcal{S}_{\tau_{k,m}}\big(\tilde{Z}^{(k+1)}_{m,i,j}\big) \tag{4.15}$$

Sau $K = 4$ tầng ta thu được $Z := Z^{(K)} \in \mathbb{R}^{10 \times 8 \times 8}$.

Mỗi tầng hỏi: "phần nào của F chưa được giải thích?" (bước 2), "atom nào chịu trách nhiệm cho phần chưa giải thích đó?" (bước 3), "chỉ giữ lại những atom thực sự đáng kể" (bước 4). Lặp lại bốn lần, các atom lần lượt chiếm lấy các thành phần của F

theo thứ tự đóng góp giảm dần — rất giống Matching Pursuit nhưng thực hiện song song và khả vi.

3.4.4.6. Tham số học được của head và giá trị khởi tạo

Ba đại lượng của thuật toán được tham số hoá sao cho luôn nằm trong miền hợp lệ mà không cần ràng buộc cứng:

$$\eta_k = \eta_{\max} \cdot \sigma(\gamma_k) = 0.15 \cdot \frac{1}{1 + e^{-\gamma_k}} \in (0,\, 0.15) \qquad (4.16)$$

$$\tau_{k,m} = \mathrm{softplus}\big(\theta_{k,m}\big) = \log\big(1 + e^{\theta_{k,m}}\big) > 0 \qquad (4.17)$$

$$\alpha = \mathrm{softplus}(\alpha_{\mathrm{raw}}) > 0 \qquad (4.18)$$

Tham số thực sự được học là $\gamma \in \mathbb{R}^K$, $\theta \in \mathbb{R}^{K \times M}$, $\alpha_{\mathrm{raw}} \in \mathbb{R}$, $b \in \mathbb{R}^{N_{\mathrm{cls}}}$. Ba phép biến đổi trên đều trơn và đơn điệu, nên lan truyền ngược hoạt động bình thường, đồng thời η_k không bao giờ vượt $\eta_{\max}$ (điều kiện ổn định từ Mục 2.3) và τ, α không bao giờ âm (điều kiện để soft-threshold và logit có nghĩa).

Giá trị khởi tạo được chọn rất có chủ ý:

Tham số thô	Giá trị khởi tạo	Đại lượng hiệu dụng	Ý nghĩa
γ_k	0	$\eta_k = 0.15 \cdot 0.5 = 0.075$	Bước đi bằng đúng nửa mức trần
$\theta_{k,m}$	-3	$\tau = \log(1 + e^{-3}) \approx 0.0486$	Ngưỡng nhỏ: tầng đầu gần như tuyến tính, thưa hoá tăng dần khi học
α_{raw}	0.5413	$\alpha = \log(1 + e^{0.5413}) = 1.0000$	Khởi tạo đúng bằng 1, vì $0.5413 = \log(e - 1)$
b_c	0	—	Không thiên vị lớp nào

Chi tiết $\alpha_{\mathrm{raw}} = \log(e - 1)$ là một lựa chọn tinh tế: nó khiến hệ số co giãn logit bắt đầu chính xác tại 1, để mô hình không bị bão hoà softmax ngay từ epoch đầu.

3.4.4.7. Soft-threshold

Dạng chuẩn $\mathrm{sign}(z)\,\mathrm{ReLU}(|z| - \tau)$ cần hai toán tử Sign và Abs. Mô hình viết lại bằng một đẳng thức chỉ dùng ReLU:

$$\boxed{\mathcal{S}_\tau(z) = \mathrm{ReLU}(z - \tau) - \mathrm{ReLU}(-z - \tau)} \qquad (4.19)$$

Định lý 4.2. Với mọi $\tau \geq 0$ và mọi $z \in \mathbb{R}$, đẳng thức (4.19) đúng, và vế phải bằng $\mathrm{sign}(z)\max(|z| - \tau, 0)$.

Chứng minh. Xét ba trường hợp vét cạn.

Trường hợp $z > \tau$. Khi đó $z - \tau > 0$ nên $\mathrm{ReLU}(z-\tau) = z - \tau$. Đồng thời $-z - \tau < -2\tau \le 0$ nên $\mathrm{ReLU}(-z-\tau) = 0$. Vế phải bằng $z - \tau$. Mặt khác $\mathrm{sign}(z) = 1$, $|z| - \tau = z - \tau > 0$, nên dạng chuẩn cũng bằng $z - \tau$.

Trường hợp $|z| \le \tau$. Khi đó $z - \tau \le 0$ và $-z - \tau \le 0$, cả hai ReLU đều bằng 0, vế phải bằng 0. Dạng chuẩn có $|z| - \tau \le 0$ nên cũng bằng 0.

Trường hợp $z < -\tau$. Khi đó $z - \tau < -2\tau \le 0$ nên số hạng đầu bằng 0. Còn $-z - \tau > 0$ nên $\mathrm{ReLU}(-z-\tau) = -z - \tau$, vế phải bằng $-(-z-\tau) = z + \tau$. Dạng chuẩn: $\mathrm{sign}(z) = -1$, $|z| - \tau = -z - \tau > 0$, tích bằng $-(-z-\tau) = z + \tau$.

Ba trường hợp phủ kín $\mathbb{R}$, vậy đẳng thức đúng với mọi z.

Nhấn mạnh: đây là phép biến đổi tương đương chính xác, không phải xấp xỉ. Giá trị số giống nhau đến từng bit (trong số học chính xác), nên đồ thị triển khai cho cùng kết quả với đồ thị huấn luyện. Lợi ích: Sign và Abs không nằm trong tập toán tử được tối ưu/lượng tử hoá tốt của ESP-DL, còn ReLU và phép trừ thì có.

3.4.4.8. Khử nội dung dùng chung và sai số dư

Sau khi có Z, ta tính một lần phần tái tạo từ nhóm atom dùng chung:

$$\hat{F}_s = \sum_{m \in \mathcal{G}_s} \widehat{D}_m \circledast Z_m, \qquad \left[\hat{F}_s\right]_{c,i,j} = \sum_{m \in \mathcal{G}_s} \sum_{u,v} \widehat{D}_{m,c,u,v}\, Z_{m,\, i-u+p,\, j-v+p} \tag{4.20}$$

Rồi với mỗi lớp c, tái tạo từ nhóm riêng của lớp đó:

$$\hat{F}_c = \sum_{m \in \mathcal{G}_c} \widehat{D}_m \circledast Z_m \tag{4.21}$$

Sai số dư của lớp c là sai số bình phương trung bình còn lại sau khi trừ cả hai:

$$\boxed{r_c = \frac{1}{C\,H\,W} \parallel F - \hat{F}_s - \hat{F}_c \parallel_F^2 = \frac{1}{768} \sum_{c\prime=1}^{12} \sum_{i=1}^{8} \sum_{j=1}^{8} \left(F_{c\prime,i,j} - \left[\hat{F}_s\right]_{c\prime,i,j} - \left[\hat{F}_c\right]_{c\prime,i,j} \right)^2} \tag{4.22}$$

Điểm cốt lõi: $\hat{F}_s$ giống hệt nhau trong biểu thức của r_{cat} và r_{dog}. Do đó hiệu $r_{\mathrm{cat}} - r_{\mathrm{dog}}$ chỉ phụ thuộc vào phần *sau khi đã khử nội dung chung*. Nếu bỏ nhóm shared đi, mọi cấu trúc phổ quát (nền, ánh sáng, kết cấu lông) sẽ phải được hai nhóm riêng biểu diễn trùng lặp, làm cả hai sai số cùng nhỏ đi một lượng gần bằng nhau và nén biên quyết định lại. Đây chính là lập luận cho đóng góp "shared-content cancellation".

3.4.4.9. Từ sai số dư đến logit

$$\ell_c = -\alpha\, r_c + b_c, \qquad \hat{y} = \operatorname*{argmax}_c \ell_c \qquad (4.23)$$

$$p(c \mid X) = \frac{\exp(\ell_c)}{\sum_{c'} \exp(\ell_{c'})} \qquad (4.24)$$

Vì $\alpha > 0$ theo (4.18), phép ánh xạ $r \mapsto -\alpha r$ nghịch biến, nên khi $b = 0$:

$$\operatorname*{argmax}_c \ell_c \;=\; \operatorname*{argmin}_c r_c \qquad (4.25)$$

tức "lớp nào tái tạo tốt hơn thì lớp đó thắng". Khi $b \neq 0$, bias có thể đảo thứ tự ở các trường hợp sát nhau — đó là cơ chế hiệu chỉnh mất cân bằng lớp mà mô hình được phép học.

Mệnh đề 4.3 (Diễn giải xác suất). Giả sử mô hình sinh: cho lớp c, đặc trưng F được sinh từ tái tạo $\hat{F}_s + \hat{F}_c$ cộng nhiễu Gauss trắng phương sai σ^2 trên CHW chiều, và tiên nghiệm lớp là π_c. Khi đó hậu nghiệm log có dạng đúng (4.23) với

$$\alpha = \frac{CHW}{2\sigma^2}, \qquad b_c = \log \pi_c \qquad (4.26)$$

Chứng minh. Mật độ Gauss cho

$$\log p(F \mid c) = -\frac{1}{2\sigma^2} \parallel F - \hat{F}_s - \hat{F}_c \parallel^2 - \frac{CHW}{2} \log(2\pi\sigma^2) \qquad (4.27)$$

Thay $\|\cdot\|^2 = CHW \cdot r_c$ theo (4.22) và áp dụng Bayes $\log p(c \mid F) = \log p(F \mid c) + \log \pi_c + \text{const}$, số hạng hằng (không phụ thuộc c) bị softmax triệt tiêu, ta được $\log p(c \mid F) \propto -\frac{CHW}{2\sigma^2} r_c + \log \pi_c$. ■

Hệ quả thực tiễn: α không phải một hệ số tuỳ tiện mà là nghịch đảo (đã co giãn) của phương sai nhiễu mà mô hình tự ước lượng. α lớn nghĩa là mô hình tin rằng chênh lệch sai số nhỏ cũng đủ để kết luận; α nhỏ nghĩa là mô hình thận trọng. Việc để α học được tương đương với việc học độ tin cậy hiệu chuẩn (calibration).

3.4.4.10. Khác biệt so với Global Average Pooling + Linear

Tiêu chí	GAP + FC	SSR head
Thông tin không gian	Bị xoá hoàn toàn bởi trung bình	Giữ nguyên lưới 8×8 suốt quá trình

Tiêu chí	GAP + FC	SSR head
Dạng quyết định	Siêu phẳng tuyến tính trên vectơ đặc trưng	So sánh chất lượng tái tạo, phi tuyến theo F
Khả năng giải thích	Trọng số FC khó diễn giải	Mỗi atom là một mẫu hình 3×3 có thể trực quan hoá; Z cho biết mẫu hình nào kích hoạt ở đâu
Xử lý nội dung chung	Ngầm định, trộn lẫn	Tường minh, tách thành nhóm riêng
Tham số	$C_{\text{feat}} \cdot N_{\text{cls}}$	$2MCk^2 + K(M+1) + N_{\text{cls}} + 1$
Tính chất	Một phép chiếu	K bước của một thuật toán tối ưu có bảo chứng

Lưu ý thẳng thắn: SSR head đắt hơn GAP+FC (2 207 tham số và 622 080 MACs so với vài chục tham số). Cái giá đó chỉ đáng nếu ablation chứng minh được lợi ích về độ chính xác hoặc tính giải thích. Đây là lý do thí nghiệm A0 (cùng encoder + GAP/FC) là baseline bắt buộc.

3.4.5. Hàm mất mát

Hàm mục tiêu có giám sát là tổ hợp tuyến tính:

$$\mathcal{L}_{\text{sup}} = \mathcal{L}_{\text{CE}} + \lambda_r \mathcal{L}_{\text{recon}} + \lambda_x \mathcal{L}_{\text{cross}} + \lambda_m \mathcal{L}_{\text{margin}} + \lambda_s \mathcal{L}_{\text{sep}} \qquad (5.1)$$

với trọng số mặc định $\lambda_r = 0.05, \lambda_x = 0.05, \lambda_m = 0.20, \lambda_s = 0.01$. Bốn số hạng sau chỉ được tính khi có nhãn, tức chỉ tồn tại lúc huấn luyện; ở suy luận chúng biến mất hoàn toàn khỏi đồ thị.

3.4.5.1. Cross-entropy với làm mượt nhãn

Nhãn cứng $y \in \{0,1\}$ được thay bằng phân phối mục tiêu làm mượt với $\epsilon = 0.05$:

$$q_c = (1-\epsilon)\, 1\{c = y\} + \frac{\epsilon}{N_{\text{cls}}} = \begin{cases} 0.975, & c = y \\ 0.025, & c \neq y \end{cases} \qquad (5.2)$$

$$\mathcal{L}_{\text{CE}} = -\sum_{c=1}^{N_{\text{cls}}} q_c \log p_c, \qquad p_c = \frac{e^{\ell_c}}{\sum_{c'} e^{\ell_{c'}}} \qquad (5.3)$$

Gradient theo logit có dạng rất gọn và cho thấy ngay tác dụng:

$$\frac{\partial \mathcal{L}_{\text{CE}}}{\partial \ell_c} = p_c - q_c \qquad (5.4)$$

Với nhãn cứng ($\epsilon = 0$), gradient chỉ triệt tiêu khi $p_y \to 1$, nên mô hình bị đẩy tới logit vô cùng lớn — tức $\alpha \left| r_{\text{cat}} - r_{\text{dog}} \right| \to \infty$. Với $\epsilon = 0.05$, gradient triệt tiêu tại $p_y = 0.975$, nên mô hình dừng lại ở mức tự tin hữu hạn. Điều này đặc biệt quan trọng ở đây vì đẩy α lên vô hạn sẽ làm hỏng cả hiệu chuẩn lẫn dải động khi lượng tử hoá INT8.

3.4.5.2. Mất mát tái tạo

$$\mathcal{L}_{\text{recon}} = \frac{1}{B} \sum_{n=1}^{B} r_{y_n}^{(n)} \qquad (5.5)$$

Chỉ sai số của lớp đúng được phạt. Nếu phạt cả hai lớp, ta sẽ vô tình yêu cầu từ điển sai cũng phải tái tạo tốt, triệt tiêu đúng tín hiệu phân biệt mà ta cần. Số hạng này đảm bảo bài toán nền (biểu diễn thưa) vẫn được giải có nghĩa, chứ không để mạng thoái hoá thành "một bộ phân loại tuỳ ý ngẫu nhiên trông giống sparse coding".

Trọng số nhỏ ($\lambda_r = 0.05$) phản ánh thứ tự ưu tiên: phân loại đúng quan trọng hơn tái tạo đẹp.

3.4.5.3. Mất mát định tuyến năng lượng giữa các nhóm riêng

Gọi năng lượng của nhóm atom riêng lớp c là tổng năng lượng trung bình không gian của các bản đồ hệ số thuộc nhóm đó:

$$e_c = \sum_{m \in \mathcal{G}_c} \frac{1}{HW} \sum_{i,j} Z_{m,i,j}^2 \qquad (5.6)$$

$$\mathcal{L}_{\text{cross}} = \frac{1}{B} \sum_{n=1}^{B} \frac{\left(\sum_c e_c^{(n)} \right) - e_{y_n}^{(n)}}{\sum_c e_c^{(n)}} \qquad (5.7)$$

(mẫu số được kẹp dưới bởi 10^{-8}; tử số được kẹp dưới bởi 0 để chống sai số dấu phẩy động.)

Đại lượng này là tỉ lệ năng lượng riêng bị rò sang các lớp sai, nằm trong $[0,1]$:

- $\mathcal{L}_{\text{cross}} = 0$: toàn bộ năng lượng riêng tập trung vào nhóm đúng — trạng thái lý tưởng.
- $\mathcal{L}_{\text{cross}} = 1$: nhóm đúng hoàn toàn im lặng — trạng thái tệ nhất.
- Với hai lớp và không có tín hiệu gì, giá trị kỳ vọng là 0.5.

Lưu ý nhóm shared không tham gia vào (5.6): các atom dùng chung được phép có năng lượng lớn tuỳ ý, vì đó đúng là vai trò của chúng.

3.4.5.4. Mất mát biên trên sai số đã chuẩn hoá

Sai số thô r_c có thang đo phụ thuộc mạnh vào độ lớn của F, vốn thay đổi giữa các ảnh. Do đó ta chuẩn hoá theo trung bình nội mẫu:

$$\tilde{r}_c = \frac{r_c}{\max(\bar{r},\, 10^{-8})}, \qquad \bar{r}$$

$$= \frac{1}{N_{\text{cls}}} \sum_{c'} r_{c'} \quad \text{(coi như hằng số khi lấy gradient)} \qquad (5.8)$$

Mẫu số được detach, tức gradient không chảy qua $\bar{r}$. Đây là chi tiết cài đặt quan trọng: nếu không detach, mô hình có thể giảm loss bằng cách *phóng to mẫu số* thay vì cải thiện thứ hạng — một đường tắt vô nghĩa.

Biên thô và mất mát hinge:

$$\Delta = \tilde{r}_y - \min_{c \neq y} \tilde{r}_c \qquad (5.9)$$

$$\mathcal{L}_{\text{margin}} = \frac{1}{B} \sum_{n=1}^{B} \text{ReLU}\left(\Delta^{(n)} + \delta\right), \qquad \delta = 0.10 \qquad (5.10)$$

Mất mát bằng 0 khi và chỉ khi $\Delta \leq -\delta$, tức sai số của lớp đúng phải nhỏ hơn sai số của đối thủ gần nhất ít nhất δ đơn vị chuẩn hoá.

Mệnh đề 5.1 (Diễn giải trong trường hợp nhị phân). Với $N_{\text{cls}} = 2$, điều kiện $\mathcal{L}_{\text{margin}} = 0$ tương đương với

$$\frac{r_y}{r_{\text{sai}}} \leq \frac{2-\delta}{2+\delta} = \frac{1.9}{2.1} \approx 0.9048 \qquad (5.11)$$

Chứng minh. Với hai lớp, $\bar{r} = (r_1 + r_2)/2$, nên $\tilde{r}_1 + \tilde{r}_2 = \frac{r_1+r_2}{\bar{r}} = 2$. Đặt $t = \tilde{r}_y$, khi đó $\tilde{r}_{\text{sai}} = 2 - t$ và $\Delta = t - (2-t) = 2t - 2$. Điều kiện $\Delta \leq -\delta$ cho $t \leq 1 - \delta/2$. Thay ngược $t = 2r_y/(r_y + r_{\text{sai}})$:

$$\frac{2r_y}{r_y + r_{\text{sai}}} \leq 1 - \frac{\delta}{2} = \frac{2-\delta}{2} \;\Leftrightarrow\; 4r_y \leq (2-\delta)(r_y + r_{\text{sai}}) \;\Leftrightarrow\; (2+\delta)r_y \leq (2-\delta)r_{\text{sai}}$$

chia hai vế cho $(2+\delta)r_{\text{sai}} > 0$ ta được (5.11).

Đây là một phát biểu rất dễ kiểm tra và rất dễ báo cáo: mô hình được yêu cầu tái tạo lớp đúng tốt hơn ít nhất khoảng 9.5 % so với lớp sai. Không đạt thì bị phạt tuyến tính theo mức thiếu hụt.

3.4.5.5. Mất mát tách biệt từ điển

Làm phẳng các atom đã chuẩn hoá thành $V \in \mathbb{R}^{M \times Ck^2}$ (mỗi hàng chuẩn đơn vị). Ma trận Gram $VV^\top$ có phần tử (m, m') đúng bằng cosin góc giữa hai atom. Ta phạt bình phương tương quan giữa các nhóm khác nhau:

$$\mathcal{L}_{\text{sep}} = \text{mean}\left(\left\{ \text{mean}\left[\left(V_{\mathcal{G}_a} V_{\mathcal{G}_b}^\top\right)^{\odot 2}\right] : (a,b) \in \mathcal{P} \right\}\right) \qquad (5.12)$$

trong đó $\mathcal{P}$ là tập các cặp nhóm cần tách:

$$\mathcal{P} = \{(s,\text{cat}),\ (s,\text{dog}),\ (\text{cat},\text{dog})\} \qquad (5.13)$$

Viết tường minh cho cấu hình hiện tại (2 shared, 4 cat, 4 dog):

$$\mathcal{L}_{\text{sep}} = \frac{1}{3}\left[\frac{1}{8}\sum_{m\in\mathcal{G}_s}\sum_{m'\in\mathcal{G}_{\text{cat}}} \langle \widehat{D}_m, \widehat{D}_{m'}\rangle^2 + \frac{1}{8}\sum_{m\in\mathcal{G}_s}\sum_{m'\in\mathcal{G}_{\text{dog}}} \langle \widehat{D}_m, \widehat{D}_{m'}\rangle^2 \right.$$

$$\left. + \frac{1}{16}\sum_{m\in\mathcal{G}_{\text{cat}}}\sum_{m'\in\mathcal{G}_{\text{dog}}} \langle \widehat{D}_m, \widehat{D}_{m'}\rangle^2 \right] \qquad (5.14)$$

Chú ý ba điều:

1. Tương quan trong cùng một nhóm không bị phạt. Bốn atom của Cat được phép giống nhau — thậm chí nên vậy nếu lớp Cat cần một không gian con nhỏ.
2. Đại lượng này là một chặn trên mềm cho độ kết dính μ ở (4.6), nối trực tiếp với điều kiện khôi phục thưa.
3. $\mathcal{L}_{\text{sep}}$ chỉ phụ thuộc vào $\widehat{D}$, không phụ thuộc dữ liệu. Nó là một chính quy hoá thuần tuý về cấu trúc, và vì thế trọng số của nó nhỏ nhất ($\lambda_s = 0.01$).

3.4.5.6. Tổng kết các thành phần

3.4.6. Chưng cất tri thức

3.4.6.1. Vì sao cần

Mô hình student có 20 607 tham số, nhỏ hơn teacher 5.92 lần và rẻ hơn 10.55 lần về MACs. Nén mạnh như vậy luôn mất chất lượng. Chưng cất tri thức (knowledge distillation) bù lại bằng cách cho student học từ phân phối mềm của teacher thay vì chỉ từ nhãn cứng.

3.4.6.2. Công thức

Với nhiệt độ $T = 3$, định nghĩa phân phối làm mềm cho teacher và student:

$$p_c^t = \frac{\exp(\ell_c^t/T)}{\sum_{c'} \exp(\ell_{c'}^t/T)}, \qquad p_c^s = \frac{\exp(\ell_c^s/T)}{\sum_{c'} \exp(\ell_{c'}^s/T)} \tag{6.1}$$

$$\mathcal{L}_{\mathrm{KD}} = T^2 \cdot \mathrm{KL}(p^t \,\|\, p^s) = T^2 \sum_c p_c^t \log \frac{p_c^t}{p_c^s} \tag{6.2}$$

Vì sao nhân T^2. Gradient của KL theo logit student là

$$\frac{\partial}{\partial \ell_c^s} \mathrm{KL}(p^t \| p^s) = \frac{1}{T}(p_c^s - p_c^t) \tag{6.3}$$

Khi T lớn, p^s và p^t đều tiến về phân phối đều nên hiệu $p^s - p^t$ co lại xấp xỉ theo $1/T$; cộng thêm hệ số $1/T$ tường minh, gradient suy giảm theo $1/T^2$. Nhân T^2 đúng bằng lượng đó giữ cho độ lớn gradient không đổi khi ta đổi T, nhờ vậy λ_{KD} có thể được chỉnh độc lập với T.

3.4.6.3. Tăng dần trọng số và mục tiêu tổng

$$\lambda_{\mathrm{KD}}(e) = \lambda_{\mathrm{KD}}^{\max} \cdot \min\left(1, \frac{e}{E_{\mathrm{ramp}}}\right), \qquad \lambda_{\mathrm{KD}}^{\max} = 0.40, \quad E_{\mathrm{ramp}} = 5 \tag{6.4}$$

$$\mathcal{L}_{\mathrm{total}} = \mathcal{L}_{\mathrm{sup}} + \lambda_{\mathrm{KD}}(e) \cdot \mathcal{L}_{\mathrm{KD}} \tag{6.5}$$

Lý do tăng dần: ở những epoch đầu, logit của student gần như ngẫu nhiên, nên KL rất lớn và gradient của nó lấn át hoàn toàn $\mathcal{L}_{\mathrm{sup}}$. Cho λ_{KD} đi từ 0.08 lên 0.40 trong 5 epoch giúp student trước hết học cấu trúc cơ bản từ nhãn thật, rồi mới học sự tinh tế từ teacher.

3.4.6.4. Ghép cặp góc nhìn giữa teacher và student

Một lỗi phổ biến trong KD là để teacher và student nhìn hai phép tăng cường ngẫu nhiên khác nhau; khi đó KL so sánh hai nội dung khác nhau và trở thành nhiễu. Mô hình xử lý bằng lớp PairedTrainTransform:

1. Sinh một phép biến đổi hình học/màu ngẫu nhiên (RandomResizedCrop với tỉ lệ diện tích $[0.70,1.0]$, tỉ lệ cạnh $[0.80,1.25]$, lật ngang $p = 0.5$, ColorJitter nhẹ, xoay $\pm 8°$), áp lên ảnh gốc, thu được ảnh 160×160.
2. Teacher nhận ảnh đó, chuẩn hoá theo thống kê ImageNet $\mu = (0.485,0.456,0.406)$, $\sigma = (0.229,0.224,0.225)$.
3. Student nhận chính ảnh đó hạ mẫu song tuyến về 128×128, chuẩn hoá theo $\mu = \sigma = (0.5,0.5,0.5)$, tức ánh xạ $[0,1] \to [-1,1]$.

Nhờ vậy, hiệu $p^t - p^s$ phản ánh đúng khác biệt giữa hai mô hình chứ không phải khác biệt giữa hai ảnh. Việc student thấy phiên bản độ phân giải thấp hơn là có chủ ý: đó đúng là điều kiện triển khai thực tế.

3.4.7. Quy trình tối ưu hoá

3.4.7.1. AdamW

Với mỗi tham số ϕ, ký hiệu $g_t = \nabla_\phi \mathcal{L}_{\text{total}}$ tại bước t:

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t, \qquad v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2 \tag{7.1}$$

$$\hat{m}_t = \frac{m_t}{1-\beta_1^t}, \qquad \hat{v}_t = \frac{v_t}{1-\beta_2^t} \tag{7.2}$$

$$\phi_t = \phi_{t-1} - \rho_t \left(\frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon} + \lambda_{\text{wd}}\, \phi_{t-1} \right) \tag{7.3}$$

với $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\varepsilon = 10^{-8}$, $\lambda_{\text{wd}} = 5 \times 10^{-4}$.

Điểm khác biệt giữa AdamW và Adam+L2 nằm ở vị trí của $\lambda_{\text{wd}}\phi$: trong AdamW nó được cộng sau khi chia cho $\sqrt{\hat{v}}$, nên mức suy giảm trọng số không bị bóp méo bởi thang đo gradient của từng tham số. Với mô hình này, điều đó đặc biệt có ý nghĩa vì các nhóm tham số có thang gradient rất khác nhau (D, W, θ, γ, α_{raw}).

3.4.7.2. Lịch trình tốc độ học

Giai đoạn khởi động tuyến tính trong $E_w = 4$ epoch:

$$\rho(e) = \rho_0 \left(0.1 + 0.9 \cdot \frac{e}{E_w} \right), \qquad e = 0,1,\dots,E_w \tag{7.4}$$

Sau đó cosine annealing về $\rho_{\min} = 0.01\rho_0$:

$$\rho(e) = \rho_{\min} + \frac{\rho_0 - \rho_{\min}}{2} \left(1 + \cos \frac{\pi\,(e - E_w)}{E_{\max} - E_w} \right) \tag{7.5}$$

với $\rho_0 = 1.5 \times 10^{-3}$, $E_{\max} = 70$. Khởi động là cần thiết vì ở bước đầu, $Z^{(0)} = 0$ khiến gradient theo D có cấu trúc rất khác so với trạng thái ổn định, dễ gây bước nhảy lớn.

3.4.7.3. Trung bình động hàm mũ của trọng số

Một bản sao ϕ^{EMA} được cập nhật sau mỗi bước:

$$d_t = \min\left(d_{\max}, \frac{1+t}{10+t} \right), \qquad \phi_t^{\text{EMA}} = d_t\, \phi_{t-1}^{\text{EMA}} + (1-d_t)\, \phi_t \tag{7.6}$$

với $d_{\max} = 0.999$. Hệ số $\frac{1+t}{10+t}$ là cơ chế *warm-up cho chính EMA*: ở $t = 1$ nó bằng 0.18 (bám sát mô hình), tiến tới 0.999 khi t lớn. Không có nó, bản EMA sẽ bị kẹt rất lâu ở giá trị khởi tạo.

Toàn bộ đánh giá validation và checkpoint tốt nhất đều dùng ϕ^{EMA}, không dùng ϕ. EMA xấp xỉ trung bình Polyak, thường nằm trong vùng cực tiểu phẳng hơn nên khái quát hoá tốt hơn.

3.4.7.4. Cắt gradient

Trước mỗi bước cập nhật:

$$g \leftarrow g \cdot \min\left(1, \frac{\kappa}{\| g \|_2}\right), \qquad \kappa = 1.0 \qquad (7.7)$$

trong đó $\| g \|_2$ là chuẩn của gradient gộp toàn bộ tham số. Đây không phải trang trí: vòng lặp unrolling nhân gradient qua K tầng, tạo cấu trúc tương tự BPTT của mạng hồi quy và có nguy cơ bùng nổ gradient tương ứng.

3.4.9. Phân tích độ phức tạp

3.4.9.1. Công thức đếm

Tham số một lớp tích chập kèm BN:

$$P_{\mathrm{conv}} = C_{\mathrm{out}} \cdot \frac{C_{\mathrm{in}}}{g} \cdot k^2 \; + \; 2C_{\mathrm{out}} \qquad (9.1)$$

MACs một lớp tích chập:

$$\mathrm{MAC}_{\mathrm{conv}} = C_{\mathrm{out}} \cdot H_{\mathrm{out}} W_{\mathrm{out}} \cdot \frac{C_{\mathrm{in}}}{g} \cdot k^2 \qquad (9.2)$$

Tham số của head:

$$P_{\mathrm{head}} = MCk^2 + MCk^2 + K + KM + 1 + N_{\mathrm{cls}} \qquad (9.3)$$

Từng số hạng tương ứng với: MCk^2 cho từ điển tổng hợp D; MCk^2 cho từ điển phân tích W; K cho các γ_k; KM cho các ngưỡng thô $\theta_{k,m}$; 1 cho α_{raw}; và N_{cls} cho bias b. Thay số:

$$P_{\mathrm{head}} = 2 \cdot 10 \cdot 12 \cdot 9 + 4 + 40 + 1 + 2 = 2160 + 47 = 2207$$

MACs của head. Đặt đơn vị cơ sở

$$\beta = HW \cdot M \cdot C \cdot k^2 = 64 \cdot 10 \cdot 12 \cdot 9 = 69\,120 \qquad (9.4)$$

Mỗi tầng unrolling tốn đúng 2β: một lần tổng hợp (4.12) và một lần phân tích (4.14). Phần tính sai số dư cuối tốn:

tính $\hat{F}_s$ tốn $HW \cdot S \cdot Ck^2$, và mỗi $\hat{F}_c$ tốn $HW \cdot A_{\text{pc}} \cdot Ck^2$. Cộng lại:

$$HW \cdot S \cdot Ck^2 + N_{\text{cls}} \cdot HW \cdot A_{\text{pc}} \cdot Ck^2 = HW\,Ck^2\left(S + N_{\text{cls}}A_{\text{pc}}\right) = HW\,Ck^2 M = \beta \qquad (9.5)$$

Vậy

$$\text{MAC}_{\text{head}} = (2K+1)\,\beta = 9 \cdot 69\,120 = 622\,080 \qquad (9.6)$$

Một đẳng thức đẹp: tổng chi phí của phần tái tạo cuối bằng đúng chi phí của một lần tổng hợp đầy đủ, vì $S + N_{\text{cls}}A_{\text{pc}} = M$ theo (4.1).

3.4.9.2. Bảng tổng hợp

Thành phần	Tham số	Tỉ lệ	MACs	Tỉ lệ
Encoder $S3FeatureEncoder$	18 400	89.29 %	9 661 440	93.95 %
Head SSR	2 207	10.71 %	622 080	6.05 %
— trong đó D	1 080	5.24 %	—	—
— trong đó W	1 080	5.24 %	—	—
— trong đó $\gamma, \theta, \alpha, b$	47	0.23 %	—	—
Toàn mô hình	20 607	100 %	10 283 520	100 %

Ước lượng FLOPs $\approx 2 \times$ MACs, tức 20.57 M.

3.4.9.3. So sánh với mô hình teacher

Đại lượng	Teacher	Student S3	Tỉ lệ nén
Ảnh vào	160×160	128×128	1.56 × ít pixel
Bề rộng encoder	32–160	16–64	—
Kênh đặc trưng C	16	12	—
Lưới đặc trưng	12×12	8×8	2.25 × nhỏ hơn
Atom dùng chung	4	2	—
Atom mỗi lớp	6	4	—
Kernel atom	5×5	3×3	2.78 × rẻ hơn
Tầng unrolling	5	4	—
Kích hoạt	SiLU + SE	ReLU	—
Tham số	122 016	20 607	5.92 ×

Đại lượng	Teacher	Student S3	Tỉ lệ nén
MACs	≈ 108.49 M	10.28 M	10.55 ×

3.4.10. Chỉ số đánh giá

Ma trận nhầm lẫn $\mathrm{CM} \in \mathbb{N}^{2\times 2}$ với hàng là nhãn thật, cột là dự đoán. Với lớp c:

$$\mathrm{TP}_c = \mathrm{CM}_{cc}, \quad \mathrm{FP}_c = \sum_{c' \neq c} \mathrm{CM}_{c'c}, \quad \mathrm{FN}_c = \sum_{c' \neq c} \mathrm{CM}_{cc'} \qquad (10.1)$$

$$P_c = \frac{\mathrm{TP}_c}{\mathrm{TP}_c + \mathrm{FP}_c}, \qquad R_c = \frac{\mathrm{TP}_c}{\mathrm{TP}_c + \mathrm{FN}_c}, \qquad F1_c = \frac{2P_cR_c}{P_c + R_c} \qquad (10.2)$$

$$\mathrm{Acc} = \frac{\sum_c \mathrm{CM}_{cc}}{\sum_{c,c'} \mathrm{CM}_{cc'}}, \qquad \mathrm{F1}_{\mathrm{macro}} = \frac{1}{N_{\mathrm{cls}}} \sum_c F\,1_c \qquad (10.3)$$

Checkpoint tốt nhất được chọn theo $\mathrm{F1}_{\mathrm{macro}}$ trên validation, không theo accuracy, vì macro-F1 phạt đều cả hai lớp kể cả khi có lệch phân bố.

Kết quả validation sơ bộ (EMA, checkpoint tốt nhất, $n = 2500$):

Chỉ số	Giá trị
Accuracy	93.08 %
Macro-F1	93.08 %
Cat P / R / F1	92.17 % / 94.16 % / 93.15 %
Dog P / R / F1	94.03 % / 92.00 % / 93.00 %
Ma trận nhầm lẫn	$\begin{bmatrix} 1177 & 73 \\ 100 & 1150 \end{bmatrix}$
Teacher (tham chiếu)	Macro-F1 95.72 % tại epoch 38

Chênh lệch teacher → student là 2.64 điểm phần trăm macro-F1, đổi lại 5.92 × ít tham số và 10.55 × ít MACs. Tập test vẫn đang khoá.

Cảnh báo về độ tin cậy. Như đã nêu ở Mục 7.5, lần chạy này có NaN ở epoch 3 và 9. Các con số trên nên được coi là *sơ bộ* cho đến khi chạy lại với nhánh margin đã được ổn định hoá về mặt số học.

33.5. Thiết kế chức năng cho ăn theo loài

Chức năng cho ăn theo loài được thiết kế nhằm liên kết trực tiếp kết quả phân loại từ mô hình Edge AI với cơ cấu cấp thức ăn vật lý. Trong phạm vi sản phẩm hoàn thiện, hệ thống hỗ trợ hai lớp chính là chó và mèo. Mỗi lớp được ánh xạ đến một bộ cấp

thức ăn độc lập, nhờ đó hệ thống có thể sử dụng loại thức ăn và khẩu phần riêng cho từng loài mà không cần nhận diện từng cá thể thú cưng.

Cấu trúc ánh xạ được thiết kế cố định ở mức phần cứng: Bộ cấp thức ăn 1 dành cho chó, sử dụng Servo 1 tại GPIO 42 và Load Cell 1 thông qua HX711 với chân DT GPIO 1, SCK GPIO 2. Bộ cấp thức ăn 2 dành cho mèo, sử dụng Servo 2 tại GPIO 41 và Load Cell 2 với DT GPIO 39, SCK GPIO 40. LED tại GPIO 48 được bật trong thời gian cấp thức ăn và buzzer tại GPIO 47 dùng để phản hồi trạng thái hoàn tất hoặc lỗi.

Bảng 3.x. Ánh xạ loài và bộ cấp thức ăn trong hệ thống

Loài	Bộ cấp	Cơ cấu xả	Phản hồi khối lượng	Khẩu phần mặc định
Chó	Bộ cấp thức ăn số 1	Servo 1 - GPIO 42	Load Cell 1 + HX711 (DT 1, SCK 2)	80 g
Mèo	Bộ cấp thức ăn số 2	Servo 2 - GPIO 41	Load Cell 2 + HX711 (DT 39, SCK 40)	45 g

Khẩu phần ưu tiên được lấy từ Firebase theo cấu hình của từng loài. Khi giá trị trên Firebase không tồn tại hoặc không hợp lệ, firmware sử dụng giá trị mặc định để bảo đảm hệ thống vẫn có thể hoạt động ở chế độ dự phòng. Trong firmware, giá trị mặc định được đặt là 80 g cho chó và 45 g cho mèo. Các giá trị này là tham số cấu hình và có thể hiệu chỉnh theo điều kiện thực tế trên ứng dụng.

Hệ thống hỗ trợ hai nguồn kích hoạt chính. Ở chế độ thủ công, ứng dụng gửi lệnh cho ăn ngay lập tức thông qua Firebase, kèm theo thông tin về loài thú cưng **và** khối lượng thức ăn mục tiêu. Ở chế độ tự động, sau khi mô hình AI nhận diện được loài với độ tin cậy đạt ngưỡng yêu cầu, firmware sử dụng kết quả nhận diện để lựa chọn bộ cấp thức ăn tương ứng. Cả hai nguồn kích hoạt đều được xử lý thông qua cùng một khối điều khiển cấp thức ăn, nhờ đó logic điều khiển servo, đo khối lượng, giới hạn thời gian xử lý và ghi lịch sử hoạt động được sử dụng thống nhất.

Để tránh việc camera nhận diện cùng một thú cưng liên tục và làm hệ thống cấp thức ăn nhiều lần, mỗi loài được thiết lập một thời gian chờ độc lập giữa hai lần cho ăn. Sau khi quá trình cấp thức ăn hoàn tất thành công, loài tương ứng sẽ tạm thời không được phép kích hoạt thêm một chu kỳ cấp thức ăn trong khoảng thời gian đã cấu hình.

Cơ chế này được áp dụng riêng cho từng loài, vì vậy việc chó đang trong thời gian chờ không ảnh hưởng đến khả năng cấp thức ăn cho mèo.

Thiết kế này giúp phân tách rõ trách nhiệm giữa mô hình AI và firmware. Mô hình AI chỉ thực hiện nhiệm vụ xác định loài thú cưng và độ tin cậy của kết quả nhận diện, trong khi firmware chịu trách nhiệm quyết định có thực hiện cấp thức ăn hay không, lựa chọn bộ cấp tương ứng, xác định khẩu phần, kiểm tra thời gian chờ và điều khiển cơ cấu chấp hành. Cách phân tách này giúp giảm sự phụ thuộc giữa mô hình AI và phần điều khiển phần cứng, đồng thời thuận lợi khi cần thay đổi hoặc cập nhật mô hình trong tương lai.

3.6. Thiết kế thuật toán cấp thức ăn

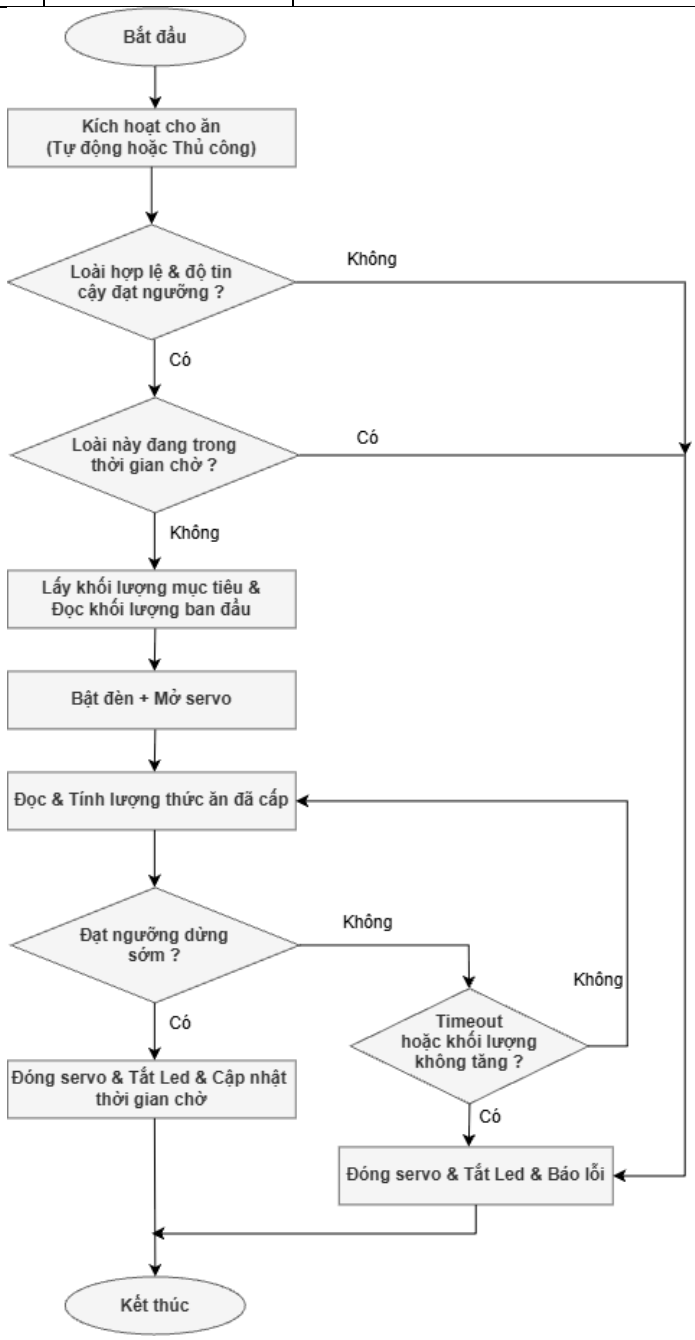
Thuật toán cấp thức ăn đóng vai trò trung tâm, tiếp nhận yêu cầu từ AI hoặc Firebase và chuyển yêu cầu đó thành một chuỗi điều khiển phần cứng có kiểm soát. Mục tiêu của thuật toán là chỉ kích hoạt đúng bộ cấp, cấp gần đúng khối lượng mục tiêu, dừng an toàn khi có lỗi và cập nhật đầy đủ trạng thái của chu kỳ cho ăn.

Đầu vào của thuật toán gồm loài thú cưng được nhận diện, khối lượng thức ăn mục tiêu, mã định danh của yêu cầu và nguồn kích hoạt quá trình cho ăn. Loài thú cưng được sử dụng để xác định bộ cấp thức ăn tương ứng, khối lượng thức ăn mục tiêu cho biết lượng thức ăn cần cấp, mã định danh giúp phân biệt từng yêu cầu để theo dõi trạng thái và lưu lịch sử, nguồn kích hoạt cho biết quá trình cho ăn được thực hiện từ thao tác thủ công trên ứng dụng hay từ cơ chế tự động dựa trên kết quả nhận diện. Nếu khối lượng mục tiêu không được cung cấp hoặc không hợp lệ, firmware sẽ đọc khẩu phần đã cấu hình trên Firebase, nếu vẫn không có dữ liệu hợp lệ, hệ thống sử dụng giá trị mặc định được thiết lập trong firmware.

Bảng 3.x. Các đầu vào chính của thuật toán cấp thức ăn

Thông tin	Giá trị	Vai trò
Loài thú cưng được nhận diện	Chó / Mèo	Xác định bộ cấp thức ăn tương ứng
Khối lượng thức ăn mục tiêu	Số thực (g)	Xác định lượng thức ăn cần cấp; nếu không có giá trị hợp lệ thì lấy từ cấu hình

Mã định danh	Chuỗi ký tự	Phân biệt từng yêu cầu để cập nhật trạng thái và lưu lịch sử
Nguồn kích hoạt	Thủ công / Tự động	Cho biết quá trình cho ăn được kích hoạt từ ứng dụng hay từ kết quả nhận diện AI
Độ tin cậy của kết quả nhận diện	0.0 - 1.0	Được sử dụng trong chế độ tự động để kiểm tra mức độ tin cậy của kết quả AI



Hình 1: . Lưu đồ thuật toán cấp thức ăn

Thuật toán được thiết kế theo nguyên **tắc** an toàn khi xảy ra lỗi. Servo luôn được đưa về trạng thái đóng khi quá trình cấp thức ăn kết thúc, kể cả trong các trường hợp

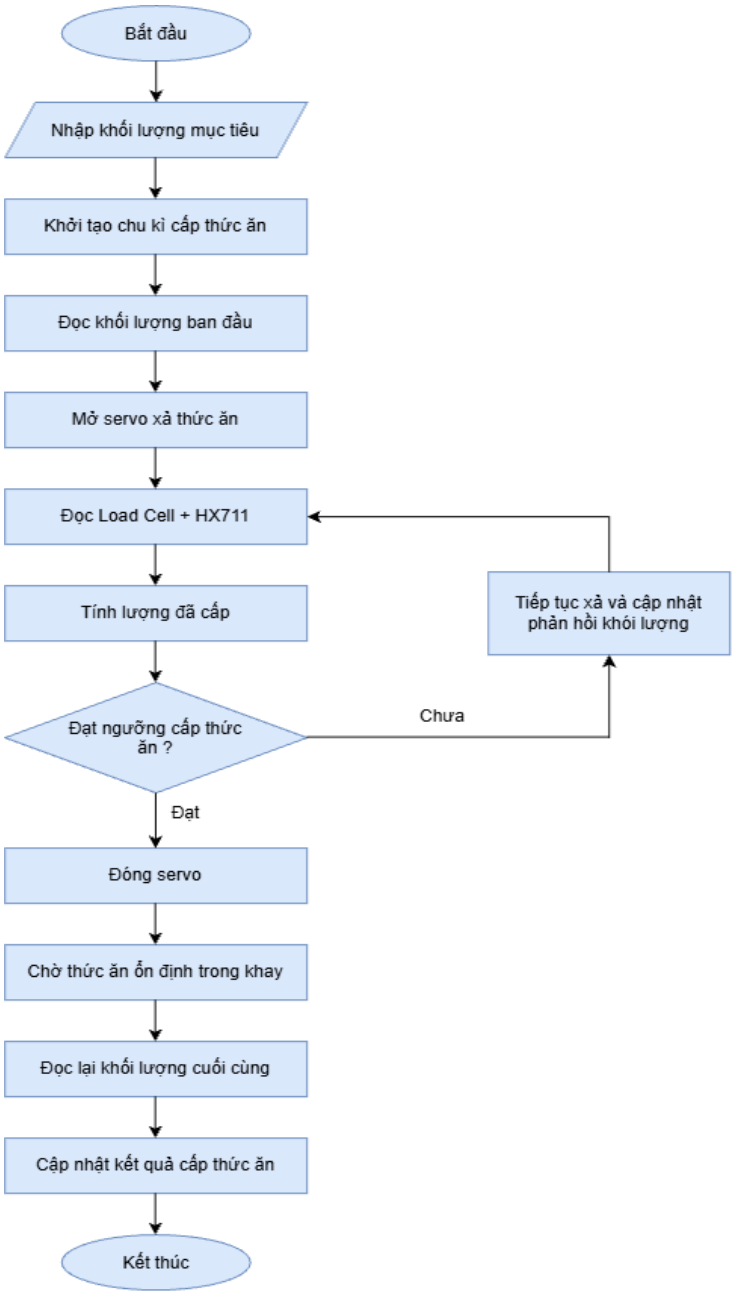
cảm biến khối lượng đọc lỗi, khối lượng không thay đổi hoặc thời gian cấp thức ăn vượt quá giới hạn cho phép. Trong phiên bản firmware hiện tại, thời gian tối đa cho một chu kỳ cấp thức ăn được thiết lập là 20 giây. Nếu hệ thống không ghi nhận khối lượng tăng tối thiểu khoảng 0,8 g trong vòng 5 giây, quá trình được xem là không có tiến triển và firmware sẽ dừng cấp thức ăn, đóng servo và ghi nhận trạng thái lỗi. Các giá trị này là tham số triển khai ban đầu và có thể tiếp tục được hiệu chỉnh dựa trên kết quả thử nghiệm thực tế của cơ cấu cấp thức ăn.

Trong quá trình giao tiếp với Firebase, mỗi lệnh cấp thức ăn được quản lý theo các trạng thái tương ứng với chờ xử lý, đang thực hiện, hoàn thành hoặc thất bại. Đồng thời, firmware cập nhật các thông tin hoạt động của thiết bị như loài đang được xử lý, khối lượng thức ăn mục tiêu, khối lượng thực tế đã cấp và trạng thái đèn báo, cho phép ứng dụng di động theo dõi quá trình vận hành. Sau mỗi chu kỳ, hệ thống lưu một bản ghi lịch sử gồm mã định danh yêu cầu, loài thú cưng, chế độ cấp thức ăn, khối lượng mục tiêu, khối lượng thực tế, kết quả thực hiện và thông tin lỗi nếu có**.**

Việc sử dụng chung một thuật toán điều khiển cho cả chế độ thủ công và tự động giúp hệ thống tránh phải xây dựng hai luồng xử lý riêng biệt. Trong chế độ tự động, mô hình AI chỉ cung cấp kết quả nhận diện loài, còn các bước kiểm tra điều kiện an toàn, xác định khẩu phần và điều khiển phần cứng vẫn được firmware thực hiện thông qua khối điều khiển cấp thức ăn. Cách tổ chức này giúp firmware có cấu trúc thống nhất, thuận lợi cho quá trình kiểm thử, bảo trì và mở rộng hệ thống.

3.7. Thiết kế điều khiển dựa trên phản hồi khối lượng

Để kiểm soát lượng thức ăn đã cấp, hệ thống sử dụng cơ chế điều khiển vòng kín dựa trên phản hồi khối lượng từ Load Cell và HX711. Servo không được điều khiển theo một khoảng thời gian mở cố định, vì lưu lượng thức ăn có thể thay đổi theo kích thước hạt, mức thức ăn còn lại trong hộp, ma sát cơ khí và góc mở của cửa xả. Thay vào đó, firmware liên tục quan sát khối lượng thực tế và đóng servo khi lượng thức ăn tăng thêm đạt ngưỡng yêu cầu.



Hình 2: Lưu đồ điều khiển cấp thức ăn dựa trên phản hồi khối lượng

Trước khi mở servo, firmware đọc và lưu lại khối lượng ban đầu của khay**.** Trong quá trình cấp thức ăn, khối lượng hiện tại được cập nhật liên tục từ cảm biến Load Cell. Lượng thức ăn thực tế đã cấp được xác định bằng độ chênh lệch giữa khối lượng hiện tại và khối lượng ban đầu**:**

Lượng thức ăn đã cấp = Khối lượng hiện tại − Khối lượng ban đầu

Cách tính theo độ tăng khối lượng giúp hệ thống không bị ảnh hưởng bởi lượng thức ăn đã có sẵn trong khay trước khi bắt đầu chu kỳ. Ví dụ, nếu khay đang có 15 g và hệ thống cần cấp thêm 45 g, servo sẽ không dừng khi cân hiển thị 45 g mà chỉ dừng khi

tổng khối lượng trong khay đạt xấp xỉ 60 g. Để hạn chế lượng thức ăn tiếp tục rơi xuống do quán tính sau khi servo bắt đầu đóng, firmware sử dụng **ngưỡng dừng sớm**. Trong phiên bản hiện tại, sai số dừng sớm được đặt là 2 g. Vì vậy, servo bắt đầu đóng khi lượng thức ăn đã cấp đạt gần giá trị mục tiêu theo điều kiện:

Lượng thức ăn đã cấp ≥ Khối lượng mục tiêu − Sai số dừng sớm

Sau khi servo đóng, firmware chờ một khoảng thời gian ngắn để phần thức ăn còn lại trong máng rơi hết xuống khay, sau đó đọc lại cảm biến Load Cell để xác định khối lượng thực tế cuối cùng đã cấp. Giá trị này được dùng để cập nhật trạng thái của lần cho ăn và lưu vào lịch sử trên Firebase. Nhờ đó, hệ thống phân biệt rõ giữa ngưỡng điều khiển dùng để quyết định thời điểm đóng servo **và** khối lượng thực tế cuối cùng dùng để ghi nhận kết quả**.**

Tham số	Giá trị	Ý nghĩa
Chu kỳ đọc khối lượng	120 ms	Khoảng thời gian giữa hai lần đọc giá trị khối lượng trong quá trình cấp thức ăn
Sai số dừng sớm	2 g	Bù cho lượng thức ăn còn tiếp tục rơi xuống sau khi servo bắt đầu đóng
Mức tăng khối lượng tối thiểu	0,8 g	Ngưỡng dùng để xác định quá trình cấp thức ăn vẫn đang có tiến triển
Thời gian tối đa không ghi nhận tiến triển	5 s	Dừng quá trình cấp nếu khối lượng không tăng đủ trong khoảng thời gian quy định
Thời gian cấp thức ăn tối đa	20 s	Giới hạn thời gian tối đa cho một chu kỳ cấp thức ăn
Thời gian ổn định sau khi mở servo	500 ms	Khoảng thời gian chờ để cơ cấu ổn định sau khi servo được mở
Thời gian chờ thức ăn rơi hoàn tất	800 ms	Khoảng thời gian chờ sau khi servo đóng để phần thức ăn còn lại trong máng rơi xuống khay

Bảng 1: Các tham số điều khiển khối lượng trong firmware

Hai cảm biến Load Cell cần được hiệu chuẩn trước khi hệ thống cho phép thực hiện quá trình cấp thức ăn tự động. Firmware hỗ trợ chức năng hiệu chuẩn thông qua cổng Serial. Trước tiên, cảm biến được đưa về trạng thái không tải để xác lập giá trị gốc, sau đó đặt một vật có khối lượng đã biết lên Load Cell để xác định hệ số chuyển đổi từ tín hiệu đo sang đơn vị khối lượng. Hệ số hiệu chuẩn được lưu trong bộ nhớ không mất dữ liệu của ESP32-S3, nhờ đó vẫn được giữ lại sau khi thiết bị khởi động lại. Nếu một

Load Cell chưa được hiệu chuẩn, firmware sẽ từ chối thực hiện chu kỳ cấp thức ăn tương ứng thay vì sử dụng trực tiếp giá trị thô từ mạch HX711. Bên cạnh điều kiện đạt khối lượng mục tiêu, quá trình điều khiển còn liên tục kiểm tra tính hợp lệ của tín hiệu phản hồi. Nếu mạch HX711 không cung cấp được dữ liệu, khối lượng không tăng đủ trong khoảng thời gian cho phép hoặc thời gian cấp thức ăn vượt quá giới hạn tối đa, firmware sẽ đóng servo ngay và kết thúc chu kỳ với trạng thái lỗi. Khi đó, buzzer phát tín hiệu cảnh báo và thông tin lỗi được cập nhật lên Firebase. Các cơ chế này giúp hạn chế nguy cơ servo duy trì trạng thái mở trong các trường hợp như hết thức ăn trong ngăn chứa, thức ăn bị kẹt hoặc cảm biến khối lượng gặp sự cố. Nhờ sử dụng phản hồi trực tiếp từ khối lượng đo được, hệ thống không phụ thuộc hoàn toàn vào thời gian quay của servo mà điều khiển dựa trên lượng thức ăn thực tế đã cấp. Cách tiếp cận này giúp cải thiện tính ổn định và khả năng lặp lại của quá trình cấp thức ăn, đồng thời cung cấp khối lượng thực tế đã cấp để ứng dụng di động hiển thị và giám sát. Các giá trị như sai số dừng sớm và giới hạn thời gian hiện tại được thiết lập theo cấu hình của firmware phiên bản đầu tiên; việc hiệu chỉnh tối ưu các tham số này cần dựa trên kết quả thử nghiệm thực tế của sản phẩm hoàn thiện.

3.8. Thiết kế Firebase

Thiết kế Firebase của hệ thống được xây dựng theo nguyên tắc phân tách quyền sở hữu dữ liệu giữa ứng dụng và firmware. Ứng dụng chủ yếu ghi cấu hình và tạo yêu cầu điều khiển; ESP32-S3 chịu trách nhiệm xác nhận hành động vật lý bằng cách cập nhật trạng thái, số đo cảm biến, cooldown, kết quả AI và lịch sử. Vì vậy giao diện không tự đánh dấu một lệnh là completed và không tự tạo feedingHistory để tránh làm dữ liệu trên cloud khác với trạng thái thật của phần cứng.

3.8.1. Phân chia quyền đọc và ghi dữ liệu

Bảng 4: Phân chia trách nhiệm giữa ứng dụng và ESP32-S3

Nhóm dữ liệu	Ứng dụng React Native	ESP32-S3	Mục đích
config/speciesProfiles	Đọc và sửa 5 trường cấu hình	Đọc portionGrams và cooldownMinutes	Cấu hình riêng cho Dog/Cat.

config/schedules	CRUD lịch theo từng loài	Firmware hiện tại chưa đọc node này	Lưu lịch cấu hình từ mobile.
command	Tạo lệnh mới khi slot không bận	Đọc pending, cập nhật processing/completed/failed	Điều khiển Feed Now.
status	Chỉ đọc	Ghi heartbeat và trạng thái	Giám sát khả dụng của thiết bị.
runtime/feeding	Chỉ đọc	Ghi live status và actualGrams	Theo dõi quá trình cấp thức ăn.
runtime/cooldown	Chỉ đọc	Ghi thời điểm hết cooldown	Khóa cho ăn lặp theo loài.
runtime/latestDetection	Chỉ đọc	AI/firmware ghi	Hiển thị kết quả nhận diện.
feedingHistory	Chỉ đọc	Push bản ghi sau feed	Lịch sử và thống kê.

Cấu hình của hệ thống hiện được gắn với mã thiết bị feeder-01 thay vì gắn với uid. Hai tài khoản được Firebase Rules cho phép truy cập cùng feeder sẽ nhìn thấy cùng profile, schedule và trạng thái. Đây là mô hình phù hợp cho prototype dùng chung một máy; nếu mở rộng nhiều hộ gia đình cần bổ sung ánh xạ uid-device.

3.8.2. Cấu trúc cây dữ liệu Realtime Database

Cây dữ liệu Firebase Realtime Database đang được app sử dụng

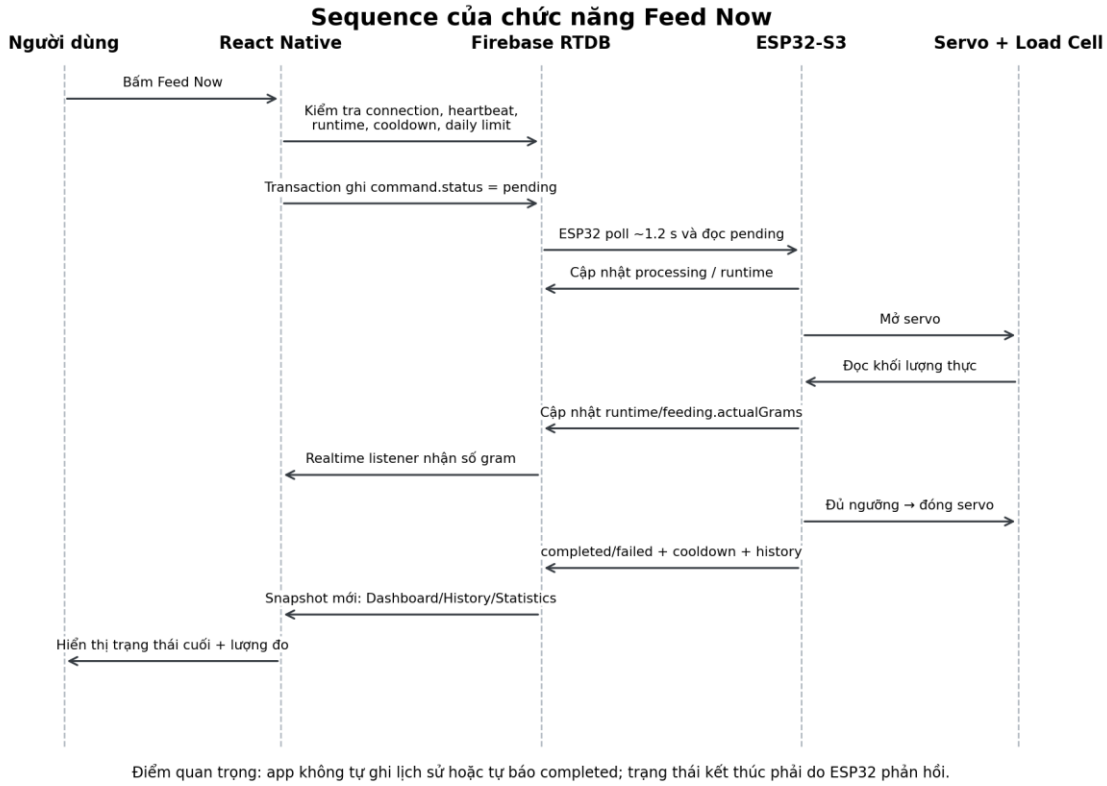
- **devices**
  - **feeder-01**
    - **status**
      - online · firmwareVersion · ipAddress · wifiRssi
      - lastSeenEpoch · operatingMode · errorCode
    - **config**
      - speciesProfiles
        - dog / cat → portionGrams · dailyLimitGrams · cooldownMinutes
        - autoFeedEnabled · notificationsEnabled
      - schedules
        - dog/{scheduleId} · cat/{scheduleId}
        - time · portionGrams · enabled
    - **runtime**
      - feeding → status · activeSpecies · targetGrams · actualGrams · ledOn
      - cooldown → dogUntilEpoch · catUntilEpoch
      - latestDetection → speciesId · confidence · capturedAtEpoch
    - **command**
      - requestId · type · speciesId · targetGrams · status · requestedAtEpoch
      - actualGrams · errorCode · completedAtEpoch (do ESP32 bổ sung)
    - **feedingHistory/{pushId}**
      - requestId · speciesId · mode · targetGrams · actualGrams
      - result · errorCode · timestampEpoch

Hình 3: Cấu trúc dữ liệu Firebase Realtime Database của thiết bị feeder-01

Nhánh status chứa online, firmwareVersion, ipAddress, wifiRssi, lastSeenEpoch, operatingMode và errorCode. Nhánh config gồm speciesProfiles và schedules. runtime được tách thành feeding, cooldown và latestDetection. command là một slot duy nhất thay vì một hàng đợi; còn feedingHistory sử dụng pushId để giữ nhiều bản ghi theo thời gian. Cấu trúc này giúp app có thể subscribe một node devices/feeder-01 và dựng toàn bộ AppSnapshot từ một snapshot thống nhất.

Profile của mỗi loài gồm portionGrams, dailyLimitGrams, cooldownMinutes, autoFeedEnabled và notificationsEnabled. Firmware hiện chỉ có hàm đọc portionGrams và cooldownMinutes; dailyLimitGrams được ứng dụng dùng để kiểm tra trước khi gửi lệnh, notificationsEnabled phục vụ logic thông báo trong app, còn autoFeedEnabled chưa được firmware AI kiểm tra trong source hiện tại.

3.8.3. Luồng đồng bộ Feed Now



Hình 4: Trình tự đồng bộ của chức năng Feed Now

Khi người dùng chọn Dog hoặc Cat và nhập số gram, app kiểm tra dữ liệu đầu vào, đọc snapshot mới nhất và lấy .info/serverTimeOffset để giảm ảnh hưởng của sai lệch đồng hồ điện thoại. isDeviceReady() yêu cầu Firebase còn kết nối, status.online bằng true, errorCode hợp lệ và heartbeat không quá cũ. Sau đó app tiếp tục kiểm tra runtime/feeding, profile, cooldown và giới hạn ngày dựa trên history đã nhận.

Nếu tất cả điều kiện hợp lệ, app tạo requestId bằng push-key nhưng không tạo command/{requestId}. requestId chỉ là mã duy nhất; payload sáu trường được ghi trực tiếp vào singleton command gồm requestId, type=feed_now, speciesId, targetGrams, status=pending và requestedAtEpoch. runTransaction() chỉ commit khi command hiện tại rỗng hoặc ở idle, completed hoặc failed; nếu pending/processing thì client mới bị từ chối.

Firmware poll command theo chu kỳ khoảng 1,2 giây. feedingControllerLoop() lấy pending command mới và gọi feedSpecies() ở mode manual. Khi bắt đầu, firmware cập nhật command sang processing và runtime/feeding sang feeding, mở servo, đọc Load Cell theo chu kỳ 120 ms và liên tục ghi actualGrams. Khi đạt ngưỡng mục tiêu trừ

sai số dừng sớm, servo đóng; firmware đo lại khối lượng sau khi thức ăn ổn định rồi ghi completed, cooldown và một record feedingHistory. Nếu timeout, Load Cell lỗi hoặc không có tiến triển khối lượng, luồng chuyển failed với errorCode tương ứng.

3.8.4. Cấu hình theo loài, lịch và cooldown

Bảng 5: Trạng thái đồng bộ các trường cấu hình

Trường	App ghi	Firmware sử dụng	Ghi chú
portionGrams	Có	Có	Khẩu phần mặc định khi lệnh/AI không truyền target hợp lệ.
dailyLimitGrams	Có	Chưa thấy trong firmware	App cộng actualGrams của history trong ngày để chặn trước lệnh.
cooldownMinutes	Có	Có	Firmware áp dụng riêng Dog/Cat và ghi cooldownUntilEpoch.
autoFeedEnabled	Có	Chưa thấy trong firmware	AI loop hiện gọi feedSpecies() khi confidence đạt ngưỡng mà không đọc cờ này.
notificationsEnabled	Có	Không	Dùng để điều khiển thông báo trong ứng dụng.
schedules	CRUD đầy đủ	Chưa có scheduler đọc config/schedules	Mới hoàn thành App ↔ Firebase, chưa end-to-end.

Lịch được tách thành config/schedules/dog và config/schedules/cat. App sử dụng ID nội bộ có dạng dog/{key} hoặc cat/{key} để tránh sửa nhầm khi hai nhánh có push-key giống nhau. Lịch mới mặc định disabled; update/toggle dùng transaction và không tạo lại lịch nếu một client khác đã xóa node. Tuy nhiên source firmware hiện tại chưa chứa vòng lặp đọc config/schedules, vì vậy báo cáo chỉ nên khẳng định chức năng CRUD và đồng bộ lịch lên Firebase, chưa khẳng định thiết bị đã tự cấp thức ăn đúng giờ.

3.8.5. Heartbeat và đánh giá thiết bị sẵn sàng

ESP32 gửi heartbeat mỗi 10 giây. App chấp nhận heartbeat nếu lastSeenEpoch không quá 120 giây và không nằm quá xa trong tương lai. Đây là ngưỡng an toàn ở phía

mobile, không phải timeout của firmware. Kết hợp .info/connected và status giúp phân biệt ba trạng thái: điện thoại mất Firebase, ESP32 mất Firebase và cả hai cùng online. Khi điều kiện không đạt, Feed Now bị khóa thay vì ghi lệnh rồi chờ vô hạn.

3.8.6. Tính toàn vẹn dữ liệu và xử lý tranh chấp

Các thao tác ghi được thiết kế để tránh optimistic success. Feed Now tắt applyLocally trong transaction; ứng dụng chỉ báo yêu cầu đã được chấp nhận sau khi server commit. Trạng thái cuối vẫn phải quay về qua realtime listener từ ESP32. Với schedule, app kiểm tra node còn tồn tại trước update/toggle và dùng transaction để không tái tạo một lịch đã bị client khác xóa. History thiếu species, actualGrams hoặc result hợp lệ bị adapter bỏ qua thay vì suy diễn dữ liệu giả.

3.8.7. Bảo mật và giới hạn hiện tại

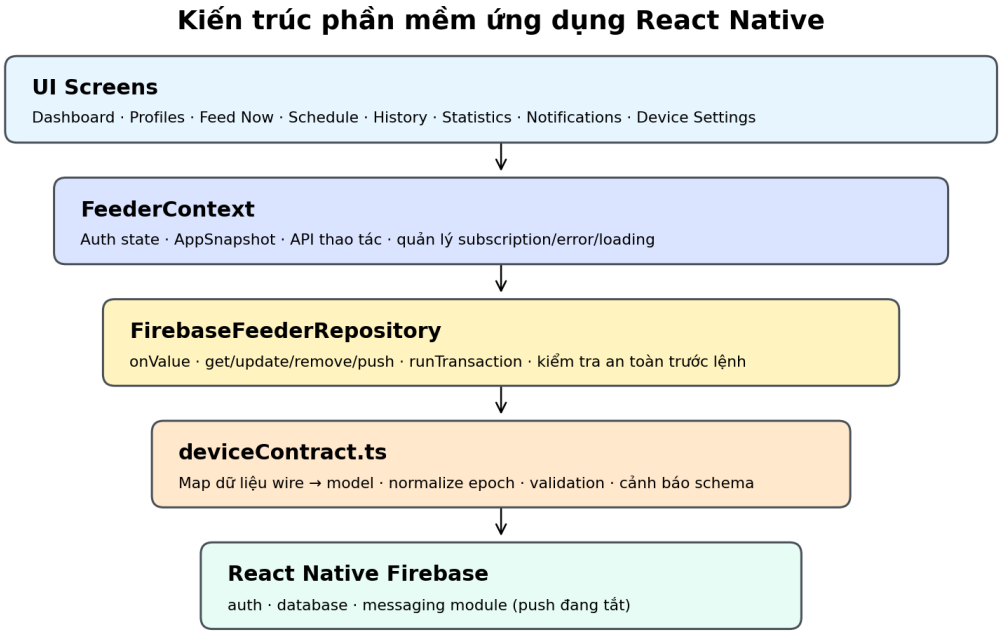
database.rules.json trong gói source là cấu hình demo yêu cầu auth != null tại nhánh devices. Cách này chưa phân biệt quyền của app và firmware. Đối với triển khai thực tế cần giới hạn app chỉ được ghi config/command, thiết bị được ghi status/runtime/history, kiểm tra loại dữ liệu và khoảng giá trị, đồng thời ràng buộc thiết bị với chủ sở hữu.

Các giới hạn cần ghi rõ trong báo cáo gồm: schedule chưa được firmware thực thi; autoFeedEnabled chưa được AI loop kiểm tra; dailyLimitGrams mới được bảo vệ ở phía app đối với Feed Now; Firebase Messaging đang tắt; và chưa có command expiry/ACK để firmware bỏ qua lệnh pending quá cũ sau mất mạng hoặc khởi động lại. Những điểm này không làm mất giá trị của kiến trúc đồng bộ hiện tại nhưng là các hạng mục cần hoàn thiện trước khi triển khai thực tế.

3.9. Thiết kế ứng dụng React Native

Ứng dụng React Native được tổ chức theo hướng nhiều lớp để tách giao diện khỏi backend. Các màn hình chỉ làm việc với AppSnapshot và các hàm nghiệp vụ do FeederContext cung cấp; Context gọi FeederRepository; FirebaseFeederRepository hiện thực repository bằng Realtime Database; cuối cùng deviceContract.ts chịu trách nhiệm chuyển dữ liệu JSON động thành model TypeScript. Thiết kế này hạn chế việc logic Firebase bị lặp ở nhiều màn hình và làm rõ nơi cần sửa khi contract thay đổi.

3.9.1. Kiến trúc phần mềm ứng dụng



Hình 5: Kiến trúc các lớp của ứng dụng React Native

RootApp theo dõi trạng thái đăng nhập và route hiện tại. Khi người dùng chưa đăng nhập, ứng dụng hiển thị LoginScreen; sau khi xác thực, các tab chính gồm Dashboard, Profiles, Feed Now, Schedule và More. Các màn History, Statistics, Notifications và Device Settings được mở từ More hoặc shortcut trên Dashboard. Điều hướng hiện dùng state nội bộ thay vì React Navigation, phù hợp với quy mô prototype hiện tại.

3.9.2. FeederContext và vòng đời dữ liệu

FeederContext đăng ký onAuthStateChanged() để nhận user Firebase. Khi có user, Context tạo FirebaseFeederRepository và gọi subscribe(). Repository theo dõi đồng thời .info/connected và devices/feeder-01; mỗi snapshot được map thành AppSnapshot rồi đưa vào state React, làm các màn hình render lại. Khi logout hoặc component bị hủy, listener được unsubscribe để tránh rò rỉ kết nối.

Context cung cấp các thao tác cấp cao gồm login(), register(), logout(), refresh(), updateProfile(), addSchedule(), updateSchedule(), toggleSchedule(), deleteSchedule() và feedNow(). Hai thao tác updateDeviceName và updateOperatingMode bị repository chủ động từ chối trong contract hiện tại vì status/operatingMode được coi là dữ liệu do firmware sở hữu.

3.9.3. FirebaseFeederRepository

FirebaseFeederRepository là lớp duy nhất chứa thao tác RTDB của ứng dụng. deviceRef() luôn trỏ về devices/feeder-01. requireConnection() kiểm tra .info/connected trước các thay đổi cấu hình; updateProfile() serialize dữ liệu profile rồi update đúng nhánh; addSchedule() dùng push() để tạo key; update/toggle dùng transaction; delete dùng remove(); còn feedNow() thực hiện toàn bộ chuỗi validation và transaction command slot.

Tách repository khỏi screen giúp phần UI không cần biết đường dẫn Firebase hoặc cách xử lý transaction. Khi cần thay đổi schema, có thể chỉnh repository/adapter mà không thay toàn bộ giao diện. Cấu trúc FeederRepository interface cũng cho phép thay backend hoặc mock trong kiểm thử.

3.9.4. Device contract và chuẩn hóa dữ liệu

deviceContract.ts kiểm tra dữ liệu Firebase trước khi đưa vào giao diện. Adapter chỉ chấp nhận dog hoặc cat cho speciesId, kiểm tra số hữu hạn cho gram/confidence và chuẩn hóa timestamp. Epoch dưới 10^12 được hiểu là giây, số lớn hơn được hiểu là mili giây. Cách xử lý này giúp app tương thích với cả hai đơn vị khi đọc, trong khi command gửi đi hiện cố định theo giây Unix.

Đối với history, adapter yêu cầu loài, target/actual và result hợp lệ; thiếu thời gian thì record vẫn có thể hiển thị nhưng không được cộng vào biểu đồ theo ngày. Các cảnh báo schema được đưa vào AppSnapshot để Dashboard có thể thông báo rằng số liệu thống kê chưa đầy đủ thay vì silently thay thế bằng dữ liệu giả.

3.9.5. Thiết kế các màn hình chức năng

Bảng 6: Các màn hình chính của ứng dụng

Màn hình	Chức năng	Nguồn dữ liệu chính
Login	Đăng nhập/đăng ký bằng email và mật khẩu.	Firebase Authentication
Dashboard	Connected/Unavailable, RSSI, live grams, AI detection, lịch gần nhất và hoạt động gần đây.	status, runtime, schedules, feedingHistory
Species Profiles	Sửa khẩu phần, daily limit, cooldown, auto-feed và notifications cho Dog/Cat.	config/speciesProfiles

Feed Now	Chọn loài/số gram, kiểm tra readiness, gửi lệnh và theo dõi kết quả.	command, runtime/feeding, history
Schedule	Thêm, sửa, bật/tắt và xóa lịch theo từng loài.	config/schedules
History	Hiển thị các lần cấp thức ăn với target, actual, mode, result và thời gian.	feedingHistory
Statistics	Tổng gram, số feed, success rate, phân bố theo loài/thời gian và AI confidence.	feedingHistory, latestDetection
Notifications	Thông báo trong app từ trạng thái và history; đánh dấu đã đọc trong phiên.	AppSnapshot
Device Settings	Hiển thị firmware, IP, RSSI, heartbeat và operatingMode.	status
More	Điều hướng đến các màn phụ.	UI state

Dashboard không chỉ nhìn status.online để quyết định Connected mà kết hợp isDeviceReady(). Live measurement lấy từ runtime/feeding.actualGrams. Latest AI Detection hiển thị speciesId, confidence và thời gian do firmware ghi. Khi dữ liệu chưa có, UI hiển thị trạng thái Unknown hoặc thông báo chưa có dữ liệu thay vì tạo giá trị mẫu.

Species Profiles duy trì draft cục bộ để heartbeat đến liên tục không ghi đè thay đổi chưa Save. Mỗi profile có portionGrams, dailyLimitGrams, cooldownMinutes, autoFeedEnabled và notificationsEnabled. Khi Save, ứng dụng chỉ ghi năm trường contract; tên/emoji của loài là dữ liệu trình bày trong UI.

Feed Now là màn hình có nhiều điều kiện bảo vệ nhất. Nút gửi bị khóa khi Firebase mất kết nối, heartbeat cũ, thiết bị báo lỗi, runtime đang bận, cooldown còn hiệu lực, profile chưa hợp lệ hoặc lượng mới làm vượt daily limit. Sau transaction thành công, màn hình tiếp tục theo dõi command và runtime; chỉ khi firmware ghi completed/failed thì trạng thái cuối mới xuất hiện.

Schedule tách danh sách Dog và Cat, dùng thời gian HH:mm theo đồng hồ ESP32 và tạo lịch mới ở trạng thái disabled. History chỉ hiển thị record thực tế từ firmware. Statistics tái sử dụng history để tổng hợp số lần cho ăn và khối lượng. Notifications hiện

là in-app notification; Firebase Cloud Messaging chưa bật nên ứng dụng chưa nhận push khi chạy nền hoặc bị đóng.

3.9.6. Kiểm soát thao tác và an toàn phía ứng dụng

Bảng 7: Các tình huống lỗi và biện pháp kiểm soát trên ứng dụng

Tình huống	Cơ chế xử lý
Người dùng bấm Feed Now liên tục	Cờ sending ở repository kết hợp transaction singleton command.
Hai điện thoại gửi gần đồng thời	Chỉ transaction đầu tiên commit khi slot còn idle/completed/failed.
Điện thoại còn mạng nhưng ESP32 mất kết nối	Heartbeat và status.online khiến isDeviceReady() trả false.
Thiết bị đang cho ăn	runtime/feeding.status khác idle/completed/failed sẽ chặn lệnh mới.
Cooldown chưa hết	runtime/cooldownUntilMs theo loài được so sánh với thời gian server.
Vượt giới hạn ngày	App cộng actualGrams của history completed trong ngày trước khi gửi.
Schedule đã bị client khác xóa	update/toggle kiểm tra tồn tại và transaction không tạo lại node.
Dữ liệu Firebase sai hoặc thiếu	deviceContract bỏ record không hợp lệ và tạo warning cho UI.

3.9.7. Luồng dữ liệu từ giao diện đến thiết bị

Luồng chung được thực hiện theo thứ tự: người dùng thao tác trên screen; screen gọi API của FeederContext; Context chuyển yêu cầu cho FirebaseFeederRepository; repository kiểm tra điều kiện và ghi RTDB; Firebase truyền dữ liệu tới ESP32; firmware thực hiện phần cứng và ghi phản hồi; onValue() nhận snapshot mới; deviceContract map dữ liệu sang AppSnapshot; cuối cùng Context cập nhật state và giao diện render lại. Luồng này bảo đảm mọi kết quả hiển thị sau thao tác điều khiển đều quay về qua backend chung thay vì chỉ thay đổi cục bộ trên điện thoại.

CHƯƠNG 4: PHÂN TÍCH KẾT QUẢ

4.1. Môi trường thực nghiệm

Bộ điều khiển chính của hệ thống sử dụng **ESP32-S3 CAM**, đảm nhiệm việc thu nhận ảnh từ camera, thực hiện suy luận mô hình AI và điều phối các thiết bị ngoại vi. Việc thực hiện xử lý trực tiếp trên vi điều khiển giúp giảm sự phụ thuộc vào máy chủ bên ngoài và phù hợp với định hướng Edge AI của đề tài.

Hệ thống cấp thức ăn được xây dựng với hai cơ cấu cấp độc lập nhằm phục vụ các nhóm thú cưng khác nhau. Mỗi cơ cấu sử dụng **động cơ servo** để đóng hoặc mở cửa xả thức ăn. Sau khi hệ thống xác định được loài thú cưng, bộ điều khiển lựa chọn cơ cấu cấp thức ăn và khẩu phần tương ứng với cấu hình đã lưu.

Để kiểm soát lượng thức ăn thực tế, mỗi khối cấp thức ăn được kết hợp với **Load Cell và module HX711**. Load Cell có nhiệm vụ đo khối lượng thức ăn tại máng, trong khi HX711 khuếch đại và chuyển đổi tín hiệu từ cảm biến sang dữ liệu số để ESP32-S3 xử lý. Giá trị khối lượng được theo dõi liên tục trong quá trình cấp thức ăn nhằm xác định thời điểm đóng servo khi lượng thức ăn đạt mức yêu cầu.

Ngoài ra, hệ thống sử dụng **LED và buzzer** để phản hồi trạng thái hoạt động, hỗ trợ người dùng nhận biết các trạng thái như đang nhận diện, đang cấp thức ăn, hoàn tất hoặc xảy ra lỗi.

Firmware của hệ thống được triển khai trên ESP32-S3 CAM và được tổ chức thành các khối chức năng độc lập nhằm thuận tiện cho quá trình phát triển và kiểm thử. Các khối chính bao gồm điều khiển camera, suy luận mô hình AI, điều khiển servo, đọc dữ liệu HX711, hiệu chuẩn Load Cell, kết nối Wi-Fi, giao tiếp với Firebase và quản lý trạng thái của quá trình cho ăn.

Quá trình thử nghiệm được xây dựng nhằm đánh giá các chức năng chính của hệ thống, bao gồm:

- Khả năng thu nhận và xử lý hình ảnh từ camera.
- Khả năng phân loại chó và mèo của mô hình AI.
- Thời gian thực hiện suy luận trên ESP32-S3 CAM.
- Khả năng lựa chọn đúng cấu hình cấp thức ăn theo kết quả nhận diện.
- Khả năng điều khiển servo trong quá trình cấp thức ăn.
- Khả năng đo và kiểm soát lượng thức ăn bằng Load Cell và HX711.

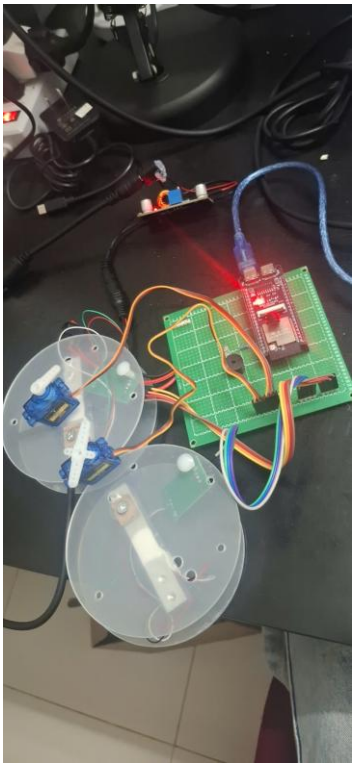
- Sai số giữa lượng thức ăn cài đặt và lượng thức ăn thực tế.
- Khả năng gửi lệnh cho ăn từ ứng dụng đến thiết bị.
- Khả năng đồng bộ trạng thái và lịch sử hoạt động thông qua Firebase.
- Khả năng xử lý các trường hợp lỗi như không phát hiện thú cưng, nhận diện có độ tin cậy thấp, thức ăn không tăng sau khi mở servo hoặc quá trình cấp thức ăn vượt quá thời gian cho phép.

Các kết quả thu được từ những thử nghiệm trên được sử dụng làm cơ sở để đánh giá **độ chính xác, độ ổn định, thời gian phản hồi và khả năng ứng dụng thực tế** của hệ thống trong các phần tiếp theo.

4.2. Kết quả mô hình AI

Method	Year	Method Description	Accuracy (%) ↑	F1 ↑	Params (M) ↓
Light CNN + PCNN/DWT	2023	PCNN feature extractor + DWT compression	69.11	0.9955	**0.011**
EfficientNetV2-S	2025	EfficientNetV2 + pretrained fine-tuning	97.57	0.9757	20.434
Multi-Path CNN	2025	Multi-scale convolution + attention fusion	97.30	0.9730	0.927
PAM + ResNet-20	2024	Probabilistic attention integrated into ResNet	91.62	0.9146	0.57
ResNet-Lite	2024	Lightweight residual CNN + knowledge distillation	89.56	0.8930	1.425
SNN-SqueezeNet-P	2026	LIF-based SNN + structured pruning	81.00	0.8100	0.599
EfficientNet-B0*	2026	Lightweight EfficientNet in FPGA-oriented AL pipeline	88.25	0.8825	5.3
Ours			**94.04**	**0.938**	**0.23**

4.3. Kết quả triển khai Edge AI



Các thành phần đều khởi tạo thành công: camera, hai servo, hai HX711, AI, Wi-Fi và Firebase. Load Cell cũng có hệ số hiệu chuẩn riêng cho chó và mèo. Mô hình đang chạy là CSUNet-SSR-S3 dạng float32, cần workspace 1,114,112 byte, tương đương khoảng 1,06 MB. Wi-Fi kết nối với RSSI -22 dBm và Firebase chuyển sang trạng thái Ready.
Pasted text

Như vậy, ở mức startup, không có bằng chứng cho thấy camera, AI, Wi-Fi, Firebase hay HX711 bị lỗi khởi tạo.

2. Hiệu năng AI trên ESP32-S3

Mình trích được 316 lần suy luận AI. Thống kê thời gian:

Thành phần	Trung bình	Trung vị	Khoảng quan sát
Capture	136 ms	7 ms	0 – 4394 ms
Decode	238 ms	235 ms	226 – 268 ms
Preprocess	1251 ms	1261 ms	1225 – 1273 ms
Inference	1250 ms	1269 ms	1183 – 1318 ms
Tổng	2875 ms	2803 ms	2635 – 7197 ms

Có thể thấy phần tốn thời gian nhất là preprocess + inference, mỗi phần khoảng 1,25 giây. Khi camera không bị chậm, một chu kỳ thường mất khoảng 2,6–2,9 giây. Log thực tế cũng cho thấy nhiều trường hợp quanh 2635–2900 ms. Pasted text

Tuy nhiên, có các outlier do thời gian capture tăng mạnh. Ví dụ capture lên tới 2303 ms khiến tổng thời gian thành 5067 ms; trường hợp khác tổng thời gian đạt tới 7196,9 ms. Pasted text Pasted text

Điểm đáng tối ưu: hiện tại preprocessing gần bằng chính thời gian inference. Nếu mục tiêu là phản hồi nhanh, không nên chỉ tối ưu model mà còn phải tối ưu pipeline resize/normalize/decode ảnh.

3. Cơ chế nhận diện khá thận trọng

Ngưỡng quyết định được đặt ở:

confidence ≥ 85%

và không cho ăn ngay sau một frame. Hệ thống yêu cầu 2 lần nhận diện liên tiếp cùng lớp:

Confirmation: 1/2 → Confirmation: 2/2 → CONFIRMED

Ví dụ mèo đạt 85,6% rồi 94,5% mới được xác nhận và kích hoạt cơ chế cho ăn. Pasted text Với chó, một trường hợp đạt 94,4% và 95,8% ở hai frame liên tiếp mới được xác nhận. Pasted text

Thống kê toàn bộ log:

Chỉ số	Kết quả
Tổng inference	316
none	239
Frame đạt ngưỡng cat	56
Frame đạt ngưỡng dog	21
Lần xác nhận hoàn chỉnh 2/2	20
Cat được xác nhận	14
Dog được xác nhận	6
Confirmation bị reset	35 lần
Confidence trung bình tại frame 2/2	89,8%

Việc xuất hiện 35 lần reset cho thấy confidence khá thường xuyên dao động quanh ngưỡng. Ví dụ một frame mèo đạt 88,3%, nhưng frame sau giảm xuống 84,5%, khiến xác nhận bị hủy. Pasted text

Đây vừa là ưu điểm vừa là hạn chế. Cơ chế 2-frame giúp tránh cho ăn do một dự đoán nhất thời, nhưng làm thời gian phản ứng thực tế dài hơn vì mỗi inference đã mất khoảng 2,8 giây.

Quan trọng: từ log này không thể nói accuracy của AI là 90% hay 95%. Log chỉ chứa prediction và confidence, không có nhãn ground truth của từng frame. Muốn tính Accuracy, Precision, Recall, F1 hoặc confusion matrix thì phải có nhãn thật tương ứng.

4. Cooldown hoạt động đúng chức năng an toàn

Sau khi nhận diện thành công, hệ thống có cooldown để tránh thú cưng đứng trước camera làm máy cấp thức ăn liên tục. Có 5 lần AI tiếp tục xác nhận được đối tượng nhưng lệnh cấp thức ăn bị chặn do cooldown.

Ví dụ:

CONFIRMED cat → cooldown active for 9 more seconds

Pasted text

Đây là một điểm tốt trong logic firmware, vì AI chạy liên tục nên nếu không có cooldown, cùng một con vật có thể kích hoạt nhiều lần cấp thức ăn liên tiếp.

4.4. Kết quả Firebase và ứng dụng

Kết quả phần Firebase và ứng dụng được đánh giá dựa trên mã nguồn, tài liệu validation đi kèm project và khả năng tích hợp với firmware hiện tại. Do tài liệu bàn giao không chứa log đầy đủ của một chu kỳ chạy phần cứng thật trong cùng môi trường kiểm thử, các kết luận dưới đây phân biệt rõ phần đã kiểm tra ở mức phần mềm với phần cần ảnh/log thực nghiệm để xác nhận end-to-end.

4.4.1. Kết quả kiểm tra ứng dụng

Bảng 8: Kết quả validation được ghi trong mã nguồn ứng dụng

Hạng mục	Kết quả trong tài liệu VALIDATION.md	Ý nghĩa

TypeScript	npm run typecheck: đạt	Không ghi nhận lỗi kiểu TypeScript trong đợt validation.
ESLint	npm run lint: đạt	Không ghi nhận lỗi/cảnh báo lint trong đợt validation.
Unit test	39 tests, 3 test suites: đạt	Các test logic/adapter/repository trong source chạy thành công.
Metro bundle Android	Bundle dev=false: đạt	Mã JavaScript có thể được Metro đóng gói.
Firebase Android config	google-services.json khớp project/package	Ứng dụng trỏ đúng project ai-smart-pet-feeder và package com.aismartpetfeeder.
Version	App 1.1.0	Phiên bản package hiện tại.

Các kết quả trên là kiểm thử phần mềm được ghi lại trong bản bàn giao. Tài liệu validation cũng lưu ý rằng môi trường kiểm thử đó không thực hiện cài APK lên thiết bị Android thật và không thay thế cho kiểm thử servo/load cell/Firebase bằng phần cứng. Vì vậy, ảnh chụp app và Firebase Console khi nhóm demo trên thiết bị thật nên được bổ sung vào bản báo cáo cuối.

4.4.2. Kết quả các chức năng Firebase và mobile

Bảng 9: Mức hoàn thiện của các chức năng mobile

Chức năng	Trạng thái theo source hiện tại	Nhận xét
Authentication	Đã triển khai	Login, register, logout bằng Firebase Email/Password.
Realtime Dashboard	Đã triển khai	Subscribe status/runtime/history và .info/connected.
Profile Dog/Cat	Đã triển khai App ↔ Firebase	Lưu 5 trường cấu hình theo loài.
Schedule CRUD	Đã triển khai App ↔ Firebase	Thêm/sửa/toggle/xóa đúng nhánh Dog/Cat.
Feed Now	Đã triển khai luồng app + firmware ở mức mã nguồn	Validation, transaction, command polling, runtime và history.

Live actualGrams	Đã có đường dữ liệu	Firmware ghi runtime/feeding; app realtime listener đọc.
History/Statistics	Đã có trong app	Phụ thuộc feedingHistory do firmware sinh.
AI Detection	Đã có app + firmware code	Firmware ghi runtime/latestDetection; Dashboard/Statistics đọc.
In-app Notifications	Đã có	Suy ra từ snapshot/history và lưu read-state trong phiên.
FCM background push	Chưa triển khai	FIREBASE_MESSAGING_ENABLED=false.

4.4.3. Kết quả tích hợp với firmware

Bảng 10: Các tham số tích hợp firmware liên quan đến Firebase và ứng dụng

Thông số	Giá trị trong source	Vai trò
DEVICE_ID	feeder-01	Khớp đường dẫn devices/feeder-01 của app.
FIRMWARE_VERSION	1.0.0	Được gửi trong status.
HEARTBEAT_INTERVAL_MS	10.000 ms	Chu kỳ firmware cập nhật heartbeat.
COMMAND_POLL_INTERVAL_MS	1.200 ms	Chu kỳ kiểm tra command pending.
AI_CONFIDENCE_THRESHOLD	0,80	Ngưỡng AI tự động kích hoạt feedSpecies().
AI_SCAN_INTERVAL_MS	5.000 ms	Chu kỳ chạy AI khi ENABLE_AI=true.
WEIGHT_SAMPLE_INTERVAL_MS	120 ms	Chu kỳ đọc Load Cell trong lúc cấp thức ăn.
FEEDING_TIMEOUT_MS	20.000 ms	Timeout an toàn cho một lần cấp.
NO_WEIGHT_PROGRESS_TIMEOUT_MS	5.000 ms	Dừng nếu khối lượng không tăng đủ lâu.
STOP_EARLY_TOLERANCE_G	2 g	Đóng servo khi tiến gần khối lượng mục tiêu.

Firmware hiện bật ENABLE_AI=true và sử dụng TensorFlow Lite Micro. ai_interface.cpp nhận ảnh camera, resize về 128×128×3, chuẩn hóa về [0,1], chạy inference hai lớp cat/dog và áp dụng Softmax. Khi kết quả hợp lệ và confidence đạt 0,80, loop ghi latestDetection rồi gọi feedSpecies() ở mode automatic. Do đó app hiện đã có đường dữ liệu để hiển thị kết quả AI. Tuy nhiên source AI loop chưa đọc autoFeedEnabled trước khi gọi feedSpecies(), đây là điểm cần sửa để cấu hình trên mobile có hiệu lực đầy đủ.

4.4.4. Đánh giá luồng Feed Now end-to-end

Ở mức mã nguồn, luồng Feed Now đã có đầy đủ hai chiều: app ghi command pending; ESP32 poll lệnh, đổi sang processing, mở servo, đo Load Cell và cập nhật runtime/feeding.actualGrams; khi hoàn tất firmware đóng servo, cập nhật completed hoặc failed, ghi cooldown và push feedingHistory; app nhận snapshot mới để cập nhật Feed Now, Dashboard, History và Statistics. Điểm cần bổ sung trong báo cáo thực nghiệm là ảnh/log chứng minh từng trạng thái này xảy ra trên Firebase thật và thiết bị thật.

Bảng 11: Các dấu hiệu cần quan sát khi kiểm thử Feed Now

Giai đoạn	Dữ liệu quan sát trên Firebase/app
Trước lệnh	Dashboard báo thiết bị sẵn sàng; heartbeat mới; runtime không bận.
App gửi	command xuất hiện với requestId mới và status=pending.
ESP32 nhận	status chuyển processing; runtime/feeding chuyển feeding.
Đang cấp	runtime/feeding.actualGrams thay đổi theo số đo Load Cell.
Hoàn tất	runtime status completed hoặc failed; command có kết quả tương ứng.
Sau feed	cooldown theo loài được cập nhật và feedingHistory có record mới.
Trên mobile	History/Statistics/Dashboard nhận snapshot mới không cần refresh thủ công.

4.4.5. Những phần chưa được xem là đồng bộ hoàn toàn

Thứ nhất, Schedule hiện mới hoàn thiện CRUD trên React Native và lưu xuống config/schedules; firmware không có scheduler đọc node này, vì vậy chưa thể kết luận

hệ thống tự cho ăn đúng giờ. Thứ hai, autoFeedEnabled được app lưu nhưng AI loop chưa kiểm tra cờ trước auto-feed. Thứ ba, dailyLimitGrams được app dùng để chặn Feed Now dựa trên history nhận được, nhưng firmware chưa có lớp kiểm tra hạn mức ngày thứ hai. Thứ tư, Notifications mới là thông báo trong ứng dụng, chưa phải push notification khi app đóng.

Ngoài ra, command hiện chưa có cơ chế expiry/TTL. Trong điều kiện mạng chập chờn, một lệnh pending cũ có thể cần được firmware đánh giá tuổi lệnh trước khi thực thi. Security Rules cũng cần được siết theo vai trò app/firmware trước triển khai thực tế. Các giới hạn này là hướng hoàn thiện tiếp theo, đồng thời giúp phần kết quả của báo cáo phản ánh đúng mức độ triển khai hiện tại thay vì khẳng định vượt quá bằng chứng trong source.

4.4.6. Kết luận phần Firebase và ứng dụng

Phần mobile đã hình thành một kiến trúc tương đối hoàn chỉnh gồm Authentication, realtime subscription, cấu hình theo loài, CRUD lịch, Feed Now có transaction chống tranh chấp, theo dõi runtime, history, statistics và hiển thị AI detection. Firebase đóng vai trò bus dữ liệu chung giữa app và ESP32-S3, trong khi repository và device contract giúp kiểm soát schema ở phía mobile. Luồng Feed Now là phần tích hợp rõ nhất giữa giao diện và firmware; các hạng mục còn thiếu chủ yếu nằm ở scheduler, thực thi autoFeedEnabled/dailyLimit ở firmware, push notification và phân quyền production.

KẾT LUẬN

Đề tài tập trung xây dựng hệ thống cho thú cưng ăn thông minh dựa trên sự kết hợp giữa Internet vạn vật (IoT), Edge AI và hệ thống nhúng, nhằm hỗ trợ quá trình cho ăn tự động, kiểm soát khẩu phần và giám sát hoạt động từ xa. Hệ thống được thiết kế với ESP32-S3 CAM làm bộ xử lý trung tâm, thực hiện thu nhận hình ảnh và suy luận mô hình AI để phân loại loài thú cưng. Kết quả nhận diện được sử dụng để lựa chọn cấu hình và bộ cấp thức ăn phù hợp.

Quá trình cấp thức ăn được thực hiện thông qua servo và được kiểm soát theo vòng kín bằng Load Cell kết hợp HX711. Thay vì chỉ điều khiển servo theo một khoảng thời gian cố định, hệ thống theo dõi sự thay đổi khối lượng thực tế và dừng cấp khi đạt khẩu phần mục tiêu. Bên cạnh đó, các cơ chế như giới hạn thời gian thực hiện, phát hiện khối lượng không thay đổi và khoảng thời gian chờ giữa hai lần cho ăn được tích hợp nhằm hạn chế các trường hợp kẹt thức ăn, cấp thức ăn thất bại hoặc cấp lặp lại trong thời gian ngắn. Ngoài khả năng xử lý trực tiếp trên thiết bị, hệ thống sử dụng Firebase Realtime Database để đồng bộ cấu hình, lệnh điều khiển, trạng thái thiết bị và lịch sử hoạt động. Ứng dụng di động được phát triển bằng React Native, cho phép người dùng quản lý cấu hình cho ăn, thực hiện lệnh cho ăn thủ công, theo dõi trạng thái và kiểm tra lịch sử từ xa. Kiến trúc này tạo thành một quy trình liên tục từ thu nhận hình ảnh, nhận diện loài, ra quyết định, điều khiển cơ cấu cấp thức ăn đến lưu trữ và hiển thị dữ liệu cho người dùng. Việc thực hiện suy luận AI trực tiếp trên ESP32-S3 CAM cũng cho thấy tiềm năng ứng dụng Edge AI trong các hệ thống IoT có tài nguyên tính toán hạn chế. Tuy nhiên, mô hình cần được tối ưu về kích thước, bộ nhớ và thời gian suy luận để phù hợp với phần cứng nhúng. Các kỹ thuật như lựa chọn kiến trúc mạng nhẹ và lượng tử hóa INT8 có thể được sử dụng để giảm chi phí tính toán trong quá trình triển khai.

Trong giai đoạn tiếp theo, hệ thống cần được kiểm thử trên thiết bị thực để đánh giá các chỉ tiêu như độ chính xác phân loại, thời gian suy luận, sai số lượng thức ăn thực tế so với khẩu phần đặt trước, thời gian phản hồi của hệ thống, độ ổn định của kết nối Firebase và khả năng xử lý các tình huống lỗi. Kết quả thực nghiệm này sẽ là cơ sở để đánh giá đầy đủ hiệu quả của hệ thống và tiếp tục tối ưu trước khi triển khai trong môi trường sử dụng thực tế.

Nhìn chung, đề tài xây dựng được định hướng cho một hệ thống cho thú cưng ăn thông minh có khả năng kết hợp nhận diện bằng AI, điều khiển phần cứng theo phản hồi cảm biến và giám sát từ xa thông qua IoT. Kiến trúc đề xuất không chỉ hướng đến tự động hóa quá trình cho ăn mà còn tạo nền tảng để mở rộng các chức năng quản lý, thống kê và theo dõi thói quen ăn uống của thú cưng trong tương lai.

TÀI LIỆU THAM KHẢO

[1] K. Baldwin, J. Bartges, T. Buffington, L. M. Freeman, M. Grabow, J. Legred, and D. Ostwald Jr., "AAHA nutritional assessment guidelines for dogs and cats," *Journal of the American Animal Hospital Association*, vol. 46, no. 4, pp. 285–296, 2010, doi: 10.5326/0460285.

[2] L. Freeman, I. Becvarova, N. Cave, C. MacKay, P. Nguyen, B. Rama, G. Takashima, R. Tiffin, H. Tsjimoto, and P. van Beukelen, "WSAVA nutritional assessment guidelines," *Journal of Small Animal Practice*, vol. 52, no. 7, pp. 385–396, 2011, doi: 10.1111/j.1748-5827.2011.01079.x.

[3] O. E. Castillo-Arceo, R. U. Renteria-Flores, and P. C. Santana-Mancilla, "Design and development of a smart pet feeder with IoT and deep learning," *Engineering Proceedings*, vol. 82, no. 1, Art. no. 63, 2024, doi: 10.3390/ecsa-11-20487.

[4] M. X. Leu and N. F. N. Rosely, "Development of machine learning wildlife camera using ESP32 CAM for small mammals," *Pertanika Journal of Science & Technology*, vol. 33, no. 4, 2025, doi: 10.47836/pjst.33.4.05.

[5] Espressif Systems, "ESP-Detection: Lightweight real-time object detection on ESP series chips based on Ultralytics YOLOv11," software repository and technical documentation, accessed Oct. 5, 2026.

[6] R. David *et al.*, "TensorFlow Lite Micro: Embedded machine learning for TinyML systems," in *Proceedings of Machine Learning and Systems (MLSys)*, vol. 3, 2021.

[7] Espressif Systems, *ESP32-S3 Series Datasheet*, Espressif Systems, 2026.

[8] Google, "Firebase Realtime Database," *Firebase Documentation*, accessed Oct. 5, 2026.

[9] Meta Platforms, "React Native Documentation," accessed Oct. 5, 2026.